

EN STUDIE AV LLM- MODELLER FÖR ANALYS AV PENETRATIONSTESTRAPP ORTER OCH KÄLLKOD

Högskoleingenjörsutbildning i IT-ingenjör
Digital Infrastruktur och cybersäkerhet

Aiman Ibrahim Belhadj
Mohamed Sharmarke Said
Brian Muhiwa



HÖGSKOLAN I BORÅS

Program:	IT-ingenjörsprogrammet med inriktning digital infrastruktur och cybersäkerhet
Svensk titel:	En studie av LLM-modeller för analys av penetrationstestrapporter och källkod
Engelsk titel:	A study of LLM models for analyzing penetration test reports and source code
Utgivningsår:	2026
Författare:	Aiman Ibrahim Belhadj, Brian Muhirwa, Mohamed Sharmarke Said
Handledare:	Oscar Lisnell
Examinator:	Nitin Harale
Nyckelord:	Penetrationstestning, Penetrationstestrapporter, Artificiell intelligens (AI), Sårbarhetsanalys, Large Language Models (LLM), Root Cause Analysis (RCA), LM Studio, Open WebUI, Lokala språkmodeller, Säkerhetsanalys, Säkerhetssårbarheter.

SAMMANFATTNING

Detta examensarbete undersöker hur en lokal AI-baserad lösning kan användas för att analysera penetrationstestrappor till sammans med källkod i syfte att identifiera grundorsaker till säkerhetssårbarheter. En proof of concept utvecklades och utvärderades med hjälp av fyra lokala LLM-modeller och två metoder: LM Studio samt Open WebUI. Totalt genomfördes 40 analyser fördelade på fem testfall.

Resultaten visar att lokala AI-modeller kan ge tekniskt korrekta analyser för korrekt klassificerade sårbarheter men modellernas prestanda var starkt beroende av kvaliteten på indata.

I testfallet där rapporten innehöll en felaktig klassificering fick samtliga modeller och metoder 0 av 6 möjliga poäng. LM Studio-metoden presterade generellt bättre än Open WebUI-metoden. I Open WebUI-metoden misslyckades retrieval-steget i flera fall med att hämta relevant källkod. Lokala modeller erbjuder dessutom en säkerhetsfördel eftersom känsligt material stannar inom organisationens infrastruktur. Studien visar att AI kan fungera som ett stöd vid säkerhetsanalys men kan inte ersätta mänsklig granskning. AI-baserade verktyg har även använts som stöd för språklig granskning, stavningskontroll och grammatiska förbättringar, men inte för att generera studiens resultat, bedömningar eller slutsatser.

ABSTRACT

This thesis investigates how a local AI-based solution can be used to analyze penetration testing reports together with source code in order to identify root causes of security vulnerabilities. A proof of concept was developed and evaluated using four local LLM models and two methods: LM Studio and Open WebUI. A total of 40 analyses were conducted across five test cases.

The results show that local AI models can produce technically accurate analyses for correctly classified vulnerabilities, but are entirely dependent on the quality of the input data.

In the test case where the report contained an incorrect classification, all models and methods scored 0 out of 6. LM Studio generally performed better than the Open WebUI method, where the retrieval step in several cases failed to retrieve relevant source code. Local models also offer a security advantage since sensitive material remains within the organization's infrastructure. The study shows that AI can function as a support tool in security analysis but cannot replace human review. AI-based tools were also used to support language review, spelling correction and grammatical improvements, but not to generate the study's results, assessments or conclusions.

ORDLISTA

AI: Artificiell Intelligens – Samlingsbegrepp för datorbaserade system som utför uppgifter som normalt kräver mänsklig intelligens såsom analys och problemlösning.

Angular: Ett JavaScript-ramverk för att bygga webbapplikationer.

Chunk: Ett mindre textsegment som skapas när dokument delas upp inför indexering i en RAG-pipeline

CWE: Common Weakness Enumeration – Ett standardiserat system för klassificering av mjukvarusårbarheter.

Hallucination: När en AI-modell genererar information som verkar trovärdig men saknar faktagrund eller är felaktig

OWASP Juice Shop: Det är en avsiktligt sårbar webbapplikation skapad av OWASP för utbildning och säkerhetstestning.

Ollama: Ett verktyg för att hantera och köra lokala språkmodeller.

Penetrationstestning: Det är en metod för att identifiera säkerhetsbrister genom att simulera verkliga attacker mot ett system.

Prompt Engineering: Tekniken att formulera instruktioner till en AI-modell för att påverka och förbättra dess svar.

Top-k: Det är en inställning i RAG-system som avgör hur många kodsegment eller dokument som hämtas vid varje fråga.

1.	INLEDNING	1
1.1	Bakgrund	1
1.2	Problemformulering	1
1.3	Syfte och forskningsfrågor	1
1.4	Avgränsningar	2
2.	TEORI	3
2.1	Penetrationstestning och metodik.....	3
2.2	Root Cause Analysis.....	4
2.3	Artificiell intelligens och stora språkmodeller (LLM).....	5
2.3.1	Arkitekturer och kategorisering	5
2.3.2	Anpassningstekniker för säkerhetsuppgifter	5
2.4	Retrieval Augmented Generation	5
3.	METOD	1
3.1	Utveckling av Proof of Concept.....	2
3.1.1	RAG-baserad metod i Open WebUI	2
3.1.2	LM Studio-metoden	3
3.1.3	Utvärdering av systemens särdrag.....	4
3.2	Bedömningsgrund och rättningsmall	4
3.3	Klassificering av sårbarhet.....	5
3.3.1	DOM-XSS	5
3.3.2	SQL Injection	5
3.3.3	Directory Listing Enabled	6
4.	RESULTAT	7
4.1	Login Bypass via Error-based SQL Injection.....	7
4.2	Application is vulnerable to SQL Injection	8
4.3	M1: Cross-Site Scripting (XSS) - Reflected	10
4.4	DOM-Based Cross Site Scripting (XSS).....	12
4.5	M3: Directory Listing Enabled.....	14
4.6	Sammanställning av resultat	16
4.7	Analys av prestandamönster	16

5. DISKUSSION	18
5.1 Tolkning av resultat	18
5.2 AI:ns beroende av rapportens information.....	18
5.3 Metodens begränsningar.....	19
5.4 Implikationer	19
5.5 Relevans i ett större sammanhang	20
5.6 LM Studio-metoden kontra Open WebUI-metoden	20
5.7 Framtida arbete.....	21
6. SLUTSATSER OCH VIDARE FORSKNING.....	22
6.1 Slutsats.....	22
6.2 Framtida forskning	22
REFERENSLISTA	24
BILAGOR.....	27

1. INLEDNING

1.1 Bakgrund

Organisationer genomför regelbundet penetrationstester för att upptäcka säkerhetssårbarheter ur en angripares perspektiv (McKinnel et al. 2019; Sarker et al. 2023). Resultaten dokumenteras vanligtvis i omfattande penetrationstestrapporter där identifierade sårbarheter, teknisk reproduktion, möjlig påverkan och rekommenderade åtgärder beskrivs (Scarfone et al. 2008). För att minska risken för att samma brister återkommer krävs en grundorsaksanalys (Root Cause Analysis, RCA) som inte enbart fokuserar på den synliga sårbarheten utan också undersöker varför den uppstod från början (Lehtinen et al. 2011). Denna typ av analys är till stor del manuell och bygger på säkerhetsexpertis, vilket gör processen både tidskrävande och svår att skala i takt med att IT-miljöer blir allt mer komplexa (Sarker et al. 2023).

Utvecklingen inom artificiell intelligens har öppnat nya möjligheter för analys av både text och källkod, särskilt genom LLM:er (Large Language Models). Genom att kombinera penetrationstestrapporter med tillhörande källkod kan AI-baserade lösningar användas för att stödja analysen av sårbarheter och bidra med förslag på möjliga åtgärder (Du et al. 2026).

Detta examensarbete undersöker hur AI kan användas för att stödja analys av penetrationstestrapporter samt vilka begränsningar som finns med en sådan lösning. I dagsläget saknas tydliga riktlinjer för hur väl AI kan identifiera rimliga grundorsaker till sårbarheter utifrån denna typ av underlag.

1.2 Problemformulering

Med det ökade behovet av effektiv säkerhetsautomation och bristen på standardiserade format för penetrationstestrapporter (Sarker et al. 2023) uppstår ett behov av att undersöka i vilken utsträckning lokala AI-modeller kan analysera sådana rapporter i kombination med källkod. Särskilt viktigt är att utvärdera modellernas förmåga att identifiera grundorsaker till sårbarheter och föreslå tekniska åtgärder utan att känsligt material behöver lämna organisationens infrastruktur.

1.3 Syfte och forskningsfrågor

Syftet med examensarbetet är att undersöka hur en AI-baserad lösning kan användas för att analysera penetrationstestrapporter i kombination med tillhörande källkod i syfte till att identifiera rimliga grundorsaker till upptäckta sårbarheter, föreslå möjliga åtgärder samt utvärdera lösningens användbarhet.

- I vilken utsträckning kan en lokal AI-baserad lösning identifiera rimliga grundorsaker, förklaringar och åtgärdsförslag till sårbarheter baserat på penetrationstestrapporter och källkod?
- Vilka fördelar och begränsningar kan identifieras vid användning av lokala AI-modeller för säkerhetsanalys av penetrationstestrapporter och källkod med avseende på prestanda och säkerhet?
- Vilka skillnader i prestanda finns mellan Open WebUI-metoden och LM Studio-metoden vid analys av penetrationstestrapporter och källkod?

För att besvara dessa frågor utvecklades och utvärderades en proof of concept bestående av två metoder: LM Studio och Open WebUI. Genom att testa fyra lokala AI-modeller på fem testfall kunde modellernas förmåga att identifiera grundorsaker, formulera förklaringar och föreslå tekniska åtgärder analyseras och jämföras.

1.4 Avgränsningar

Lösningen utformas på en agnostisk modell där AI-komponenten kan ersättas av antingen lokala modeller eller API-baserade tjänster. Inom ramen för detta arbete implementeras och utvärderas endast lokala modeller medan API-baserade lösningar diskuteras teoretiskt.

2. TEORI

2.1 Penetrationstestning och metodik

Penetrationstestning/pentestning är en strukturerad metod för att identifiera säkerhetsbrister i ett system från ett angriparperspektiv genom att simulera attacker (Scarfone, Souppaya, Cody & Orebaugh 2008).

Till skillnad från passiv sårbarhetsskanning innebär penetrationstestning ett aktivt försök att utnyttja svagheter som har hittats. Det ger en mer realistisk bild av hur säkert ett system faktiskt är. Metoden används brett inom organisationer som ett proaktivt verktyg för att bedöma säkerheten i IT-infrastruktur. Metoden låter organisationer utvärdera hur bra befintliga försvarsmekanismer fungerar och identifiera brister innan de utnyttjas av faktiska angripare (McKinnel et al. 2019; Sarker, Yunus & Deraman 2023).

För att standardisera hur tester genomförs har flera ramverk utvecklats. National Institute of Standards and Technology skriver i sin tekniska guide att testningsprocessen består av flera faser som omfattar planering, informationsinsamling, sårbarhetsanalys, exploatering och rapportering (Scarfone et al. 2008; Sarker et al. 2023). Två av de mest använda ramverken i industri och forskning är OWASP Web Security Testning Guide (WSTG) och Penetration Testing Execution Standard (PTES). OWASP WSTG v4.2 är ett ramverk specifikt gjort för webapplikationer och strukturerar testprocessen i autentisering, sessionshantering och inputvalidering (Scarfone et al. 2008; OWASP Foundation 2026d). PTES erbjuder ett bredare perspektiv och definierar en komplett testlivscykel i sju steg från förhandskommunikation, informationsinsamling, exploatering och rapportering med syftet att skapa konsistens och transparens i offensiva säkerhetsgranskningar (Sarker et al. 2023).

I planeringsfasen definieras testets omfattning (Scope), mål och regelverk för testet. Därefter genomförs informationsinsamling (reconnaissance) och angreppsytan kartläggs genom att samla all tillgänglig information om målsystemet. I exploateringsfaserna samt sårbarhetsanalysen identifieras och testas sårbarheter aktivt, varefter resultaten sammanställs i en penetrationsrapport (Scarfone et al. 2008).

Rapportering är en central del av processen och innehåller typiskt en beskrivning av sårbarheten, teknisk reproduktion, påverkan samt föreslår åtgärder (Scarfone et al. 2008). Trots att rapportering är central har penetrationstestrapporter inget gemensamt format godkänt av säkerhetsorganisationer. Vilket innebär att rapportens struktur, terminologi och detaljeringsnivå i praktiken varierar beroende på testare, organisation och vilket ramverk som tillämpats (Sarker et al. 2023). En konsekvens är att två testare som följer samma ramverk kan producera mycket olika rapporter baserat på erfarenhet, verktygsval och personliga bedömningar. Denna variabilitet är problematisk ur ett automatiseringsperspektiv. En AI-modell som analyserar penetrationsrapporter måste kunna hantera avsaknaden av ett standardformat, då rapporternas struktur och innehåll varierar beroende på författare och vilket ramverk som tillämpas. Automatiserade verktyg presterar ofta sämre när sårbarheter kräver en djupare förståelse av kontext och flera steg i resonemang. Denna typ av sårbarheter kräver egenskaper som erfarna manuella tester har men som är svåra att återskapa (Sarker et al. 2023; Scarfone et al. 2008). Det är i detta gap mellan automatiseringsskalbarhet och den manuella expertisens djup som AI-baserade lösningar av den typ som undersöks i detta examensarbete har sin potential. Frågan som kvarstår är dock vilken grad nuvarande LLM-baserade system har av den kontextuella

förståelsen som behövs för att hantera den metodologiska variationen som präglar riktiga penetrationstestrapporter, vilket detta arbete syftar till att utforska.

2.2 Root Cause Analysis

Root Cause Analysis (RCA) är en metodisk teknik designad för att identifiera de underliggande orsakerna till ett problem, med syftet att säkerställa att det inte upprepas. Till skillnad från att bara hantera symtomen fokuserar RCA på att analysera de bakomliggande orsaker till varför ett problem har uppstått och de faktorer som bidrog till dess uppkomst. Inom mjukvaruutveckling används RCA för att identifiera orsaken till fel, brister i säkerheten och systemproblem. Syftet med metoden är även att förbättra både utvecklingsprocessen och koden (Lehtinen et al. 2011).

Det finns ingen enhetlig definition av begreppet "root cause" men inom mjukvaruindustrin betraktas det vanligtvis som en faktor vars åtgärd kan sänka risken för att liknande problem uppstår igen (Lehtinen et al. 2011). En vanlig RCA-process består av tre viktiga steg: (1) att tydligt definiera och begränsa problemet, (2) att upptäcka och strukturera möjliga orsaker och (3) att utveckla åtgärder för att korrigera situationen.

En välkänd teknik inom mjukvaruutveckling är Defect Causal Analysis (DCA), som systematiskt analyserar defekter för att identifiera återkommande mönster och orsaker kopplade till processerna (Card 1998). Målet sträcker sig bortom att bara att åtgärda specifika misstag, det handlar också om att långsiktigt minska antalet defekter genom att förbättra processerna. Evidensbaserade riktlinjer betonar betydelsen av en välstrukturerad och noggrant dokumenterad utförande av orsaksanalysen (Kalinowski et al. 2012).

En annan metod som är relevant är Orthogonal Defect Classification (ODC), vilken grupperar defekter i definierade kategorier. Denna strategi möjliggör en kvantitativ analys av bristerna i utvecklingsprocessen (Chillarege et al. 1992). Genom att använda systematisk klassificering kan organisationer identifiera mönster bakom defekter, vilket i sin tur leder till förbättringar av både utvecklingsprocessen och kvalitetssäkringen.

För att skapa kopplingar mellan orsaker används ofta visuella och analytiska verktyg, såsom Ishikawa-diagram som även kallas för fishbone-diagram. Denna teknik hjälper till att identifiera faktorer som påverkar områden som design, implementering, testning och kravspecifikation (Ishikawa 1986).

Inom ramen för säkerhetsanalys innebär RCA att en identifierad sårbarhet, som kan ha framkommit i en penetrationstestrapport, analyseras mer ingående än bara dess synliga tecken. En SQL-injektionssårbarhet kan till exempel uppstå på grund av bristande validering av användardata, felaktig hantering av databasanrop eller avsaknad av säkerhetsstandarder i kodningen. Den grundläggande orsaken kan därför relateras både direkt till koden och de processer som används.

I detta examensarbete används RCA som teoretisk bas för att bedöma om ett AI-baserat analysverktyg kan identifiera rimliga orsaksrelationer mellan en angiven sårbarhet och den bakomliggande källkoden. Målet är inte att fastställa en entydig grundorsak, utan snarare att utvärdera om verktygets resonemang är tekniskt hållbart, konsekvent och praktiskt. RCA-teorin fungerar därmed som en strukturerande modell för att förstå och bedöma analyser och rekommendationer som AI-modellen producerar.

2.3 Artificiell intelligens och stora språkmodeller (LLM)

Stora språkmodeller (Large Language Models, LLM) markerar ett betydande framsteg inom området för artificiell intelligens. Dessa modeller baseras ofta på Transformer-arkitekturen, som använder en uppmärksamhetsmekanism (attention mechanism) för att identifiera relevanta delar i en sekvens (Beljulji & Gür 2025, s.3). Dessa modeller tränas på stora mängder textdata för att lära sig rika språkliga representationer som kan anpassas för specifika uppgifter, bland annat cybersäkerhet (Xu et al. 2024, s.2).

2.3.1 Arkitekturer och kategorisering

I studier av cybersäkerhet kategoriseras LLM arkitekturer oftast i tre kategorier beroende på deras användningsområde (Xu et al. 2024, s.22).

- Encoder-only: Modeller som BERT fokuserar på att förstå och koda information vilket är användbart för att klassificera sårbarheter (Xu et al. 2024, s.22).
- Encoder-decoder: Modeller som T5 är strukturerade för sekvens-till-sekvens-uppgifter vilket gör dem passande för automatiserad programreparation (Xu et al. 2024, s.22, 26).
- Decoder-only: Denna arkitektur används i GPT-serien och Llama, genererar text token för token och dominerar marknaden tack vare sin förmåga till avancerat resonemang (Xu et al. 2024, s.23).

Decoder-only-modeller anses vara speciellt kraftfulla inom säkerhetsområdet eftersom deras breda kontextfönster gör det möjligt att analysera både sårbarhetsrapporter och källkod samtidigt (Xu et al. 2024, s.23).

2.3.2 Anpassningstekniker för säkerhetsuppgifter

För att anpassa redan tränade basmodeller till specifika säkerhetsändamål används främst tre tekniker (Xu et al. 2024, s.26):

1. Fine-tuning: Innebär vidare träning på domänspecifika datamängder för att justera modellens interna parametrar (Xu et al. 2024, s.26).
2. Prompt Engineering: Handlar om att utforma instruktioner (prompter) som styr modellens beteende (Tamberg & Bahsi 2025, s.3). Exempel inkluderar Chain-of-Thought (CoT), där modellen vägleds att resonera steg-för-steg (Beljulji & Gür 2025, s.23).

2.4 Retrieval Augmented Generation

Stora språkmodeller stöter på problemet att deras kunskap är fast och begränsad till träningsdata som användes under utbildningen. Språkmodellernas analys kan bli begränsad eftersom de saknar tillgång till specifik kontext och uppdaterad information. Detta gör dem olämpliga för att analysera domänspecifika material som penetrationstestrappor och källkod som de aldrig har kommit i kontakt med (Klesel & Wittmann 2025).

Retrieval Augmented Generation (RAG) adresserar detta problem genom att kombinera modellens förmåga att generera text med ett icke-parametriskt minne. Ett system som i realtid hämtar relevant information från externa datakällor, dit modellen kan vända sig för att hämta den nödvändiga informationen vid behov (Klesel & Wittmann 2025).

En typisk RAG-pipeline består av tre steg: sökning, berikning och generering. I sökprocessen omvandlas den inkommande frågan till en vektor som jämförs med vektoriserade texter i en extern databas för att hitta den mest relevanta informationen (Klesel & Wittmann 2025; Pratama et al. 2024). Berikning är steget där den insamlade informationen integreras i en promptmall

tillsammans med den ursprungliga frågan. Det ger modellen instruktioner att använda sin interna kunskap som grund. I det avslutande steget genererar modellen ett svar som baseras på både fråga och det insamlade underlaget. Detta är en process som ofta benämns grounding. Detta minskar risken för hallucinationer och gör det möjligt att spåra svarets ursprung till konkreta källor (Klesel & Wittmann 2025).

Det är dock viktigt att poängtera att RAG inte fungerar som en universallösning. Studier inom sårbarhetsdetektering har visat att den traditionella RAG som endast fokuserar på rå källkod ofta underpresterar eftersom den inte tar hänsyn till semantiska skillnader mellan sårbar kod och kod där sårbarheten har åtgärdats (Du et al. 2026). Därför rekommenderas ibland kunskapsbaserad RAG (knowledge-level RAG), som använder strukturerad och avancerad information om sårbarhetsbeteenden för att styra modellens resonemang (Du et al. 2026).

3. METOD

Testerna genomfördes med två penetrationstestrappor riktade mot den sårbara webbapplikationen OWASP Juice Shop. Genom att använda samma applikation i båda rapporterna blev det möjligt att jämföra resultaten och identifiera skillnader utan att påverkas av variationer i systemarkitektur eller implementering. Den första rapporten identifierade tre specifika sårbarheter medan den andra rapporten listade två sårbarheter av samma typ som de i den första. Den andra rapporten innehöll dock en annan struktur och terminologi. Sammanlagt utvecklades fem testfall. Varje test granskades med hjälp av fyra olika AI-modeller samt två distinkta metoder: LM Studio och Open WebUI tillsammans med RAG. Detta resulterade i sammanlagt genomfördes 40 analyser (5 testfall x 4 modeller x 2 metoder).

Den oberoende variabeln i studien var hur tillgången till information varierade mellan de olika metoderna:

- Open WebUI-metoden: här användes hela kodbasen tillsammans med en kunskapsdatabas och en rapport om penetrationstest.
- LM Studio-metoden: denna metod involverade manuellt utvalda kodfiler tillsammans med samma penetrationstestrappor.

Den beroende variabeln var modellernas förmåga att identifiera grundorsaken till sårbarheten, presentera en teknisk förklaring och föreslå möjliga lösningar. För att säkerställa jämförbara testvillkor användes identiska modeller, uppmaningar och hårdvara i båda metoderna.

Val av AI-modeller

I denna studie användes och utvärderades endast lokala språkmodeller vilket var en del av arbetets ramar. Modellerna valdes baserat på deras prestanda, minnesbehov samt deras förmåga att analysera kod och bearbeta både svenska och engelska texter.

De modeller som ingick i studien är:

- qwen2.5-coder:7b
- qwen2.5-coder:14b
- qwen3-coder:9b
- Ministral 3 14B Reasoning

Hårdvara

Alla tester utfördes på identisk hårdvara för att säkerställa att resultaten inte blev påverkade av variationer i systemets prestanda. Testmiljön omfattade följande komponenter:

- Processor: AMD Ryzen 5 7600 6-Core Processor
- RAM: 32 GB DDR5
- Operativsystem: Microsoft Windows 11 Pro (version 10.0.26200)

Systemprompt

För att säkerställa att modellerna fick samma instruktioner oavsett testinställningar användes identiska systemprompter i både metoderna. I Open WebUI skapades en custom-agent i Open webUI:s Workspace under namnet "Juice Shop Security".

Du agerar som en cybersäkerhetsanalytiker. Nedan finns en beskrivning från en penetrationstestrappor och ett kodutdrag från den berörda applikationen.

Din uppgift är att analysera informationen och identifiera den mest sannolika grundorsaken till sårbarheten. Förklara varför sårbarheten uppstår baserat på rapporten och koden. Föreslå en möjlig åtgärd eller säkerhetsförbättring.

Svara i följande format:

Grundorsak:

[Förklaring av vad som orsakar sårbarheten]

Förklaring:

[Beskriv hur sårbarheten uppstår baserat på rapport och kod]

Föreslagen åtgärd:

[Föreslå en möjlig lösning eller förbättring]

3.1 Utveckling av Proof of Concept

En proof of concept (PoC) är en avgränsad implementering av en teknisk lösning som syftar till att utvärdera om ett koncept fungerar i praktiken istället för att skapa ett helt färdigt system.

I examensarbetet skapades två olika Proof of Concept-metoder för att undersöka hur lokala språkmodeller kan analysera penetrationstestrappporter och källkod. Syftet var att identifiera grundorsakerna till säkerhetssårbarheter.

Den första metoden genomfördes i Open WebUI med en RAG-baserad struktur, där modellerna kunde utnyttja hela kodbasen via en kunskapsdatabas. Den andra implementeringen genomfördes i LM Studio, där modellerna istället använde manuellt utvalda kodfiler tillsammans med penetrationstestrappporten. Genom att analysera de två metoderna möjliggjordes en analys av olika sätt att få tillgång till informationen påverkade modellernas förmåga att göra analyser.

3.1.1 RAG-baserad metod i Open WebUI

Den första PoC-metoden utvecklades i Open WebUI och implementerades som en RAG-baserad miljö. Målet var att ge modellerna möjligheten att granska penetrationstestrappporter i kombination med den relevanta källkoden vilken automatiskt hämtades från en kunskapsdatabas.

En viktig designprincip var att inställningarna för modellerna förblev konstant under experimentets gång medan tillgången till information varierade beroende på de olika metoderna. Detta gjorde det möjligt att studera hur retrieval-processen påverkade modellernas analysförmåga och resonemang.

3.1.1.1 Val av plattform

Open WebUI valdes eftersom plattformen:

- Stödjer dokumentbaserad RAG med hjälp av kunskapsbaser.
- Möjliggör användning av lokala språkmodeller via Ollama
- Automatiserar embedding, chunking och vektorsökning
- Tillåter skapandet av en reproducerbar arbetsmiljö

En custom-agent med namnet "Juice Shop Security" skapades och användes som standardmiljön för alla tester.

3.1.1.2 Kunskapsbas och RAG-pipeline

För att möjliggöra sökningar i källkoden skapades en kunskapsbas som innehöll hela OWASP Juice Shops kodbas, bestående av 955 filer.

Vid uppladdningen av filerna blev de automatiskt indexerade i Open WebUI. Varje fil delades upp i mindre textsegment för att möjliggöra effektiv vektorsökning. När modellen ställdes inför en fråga om en sårbarhet, omvandlades den frågan till en vektor som jämfördes med kodsegmenten i databasen. De kodsegment som ansågs vara mest relevanta skickades sedan tillsammans med penetrationsrapporten till modellen.

Följande standardinställningar användes:

- Chunk-storlek: 1000 tecken
- Overlap: 100 tecken
- Embedding-modell: all-MiniLM-L6-v2
- Databas: Chroma
- Top-K: 3

Alla parametrar behövs på sina standardvärden för att undvika manuella justeringar och för att skapa en mer konsekvent testmiljö.

3.1.1.3 Implementeringsmiljö

Open WebUI kördes i en miljö baserad på containrar och Docker version 29.2.1 användes. De lokala språkmodellerna hanterades genom Ollama version 0.20.2.

Containern startades med detta kommando:

```
docker run -d -p 3000:8080 --add-host=host.docker.internal:host-gateway -v open-webui:/app/backend/data --name open-webui --restart always ghcr.io/open-webui/open-webui:main
```

Efter uppstart användes de vanliga inställningarna i Open WebUI för att hantera dokument och utföra retrieval. Inga manuella justeringar gjordes av parametrarna för chunking, embedding eller retrieval.

3.1.2 LM Studio-metoden

Den andra PoC-metoden skapades i LM Studio och användes för att jämföra med en RAG-baserade lösning i Open WebUI.

I motsats till Open WebUI saknade LM Studio tillgång till en kunskapsbas eller möjligheten att automatiskt hämta kodsegment. Modellerna arbetade istället med manuellt utvalda kodfiler tillsammans med penetrationstestrapporten.

Målet med denna metod var att studera modellernas prestation när den relevanta kontexten valdes manuellt, snarare än att automatiskt hämta den genom en RAG-pipeline.

3.1.2.1 Implementeringsmiljö

LM Studio användes som en lokal plattform för att köra och testa språkmodellerna utan att använda retrieval eller externa kunskapskällor. Samma hårdvara, modeller och systemprompt

användes som i Open WebUI-metoden för att säkerställa att resultaten i första hand påverkades av skillnader i informationsåtkomst istället för tekniska skillnader mellan de olika miljöerna.

LM Studio fungerade som en lokal plattform där modellerna kördes direkt på den stationära datorn utan att använda containerisering eller extern hantering av databaser.

3.1.2.2 Genomförande i LM Studio

I LM Studio fick modellerna tillgång till penetrationstrappporten samt manuellt utvalda kodfiler som relaterade till den specifika sårbarheten.

Till skillnad från Open WebUI-metoden genomfördes ingen automatisk hämtning av kodsekvenser från hela kodbasen. Den tekniska kontexten begränsades istället till de filer som ansågs mest relevanta för varje testfall.

Målet med detta upplägg var att undersöka hur modellerna presterade när de fick ta emot en mindre men mer fokuserad mängd information. Det gjorde det möjligt att jämföra om manuellt utvald kontext gav bättre resultat än automatiserad retrieval från hela kodbasen.

3.1.3 Utvärdering av systemens särdrag

Genom att jämföra LM Studios begränsade filhantering med Open WebUI:s mer omfattande kunskapsbas skapades ett underlag för att undersöka vilka begränsningar som kan uppstå vid användning av lokala AI-modeller. Detta upplägg gjorde det möjligt att diskutera betydelsen av olika arkitekturella val, exempelvis användningen av en RAG-pipeline jämfört med modellernas egen förmåga att resonera utifrån en mer begränsad och specifik kontext.

3.2 Bedömningsgrund och rättningsmall

I denna studie bedöms modellsvar utifrån en fast rättningsmall för sårbarhetsidentifiering och åtgärdsförslag. Mallen bygger på etablerade principer från OWASP WSTG och MITRE CWE samt utgår från faktisk källkod i applikationen. Rättningsmallen utvecklades för att säkerställa att bedömningen är rättvis, konsekvent och oberoende av vilket format modellens svar presenteras i. För att ett svar ska anses fullständigt korrekt måste modellen uppfylla samtliga följande kriterier:

- Identifiera den verkliga grundorsaken i koden inte enbart sårbarhetens yttre symtom.
- Förklara hur sårbarheten kan utnyttjas steg för steg.
- Peka ut var i applikationen problemet uppstår. Den ska ta upp specifik kodrad, variabel eller mekanism.
- Föreslå en tekniskt korrekt åtgärd med rätt syntax för det aktuella systemet.
- Korrekt avgöra om sårbarheten finns på server- eller klientsidan.
- Undvika tekniska fel såsom fel programmeringsspråk, fel ramverk eller generiska lösningar som inte är anpassade till applikationens tekniska kontext.

Poängsättningen baseras på tre dimensioner: grundorsak, förklaring och åtgärd. Varje dimension ges ett värde mellan 0 och 2. Vilket ger ett maxvärde om 6 poäng per testfall. Denna struktur gör det möjligt att jämföra modellernas prestanda både per dimension och totalt. Det gör möjligt att avgöra om en modell förstår problemet men misslyckas med lösningen eller om felet uppstår redan i analysfasen.

Den fullständiga rättningsmallen som användes vid bedömningen av samtliga modellsvar återfinns i Bilaga A (A1). Mallen användes konsekvent vid alla tester för att säkerställa att poängsättningen var reproducerbar och oberoende av modell och metod.

3.3 Klassificering av sårbarhet

3.3.1 DOM-XSS

En av de två granskade rapporterna klassificerar XSS-sårbarheten i sökfunktionen (/#/search?q=) som Reflected XSS. Den klassificeringen är tekniskt felaktig, vilket påverkar hur modellernas svar poängsätts. Därför förklaras skillnaden här innan resultaten presenteras.

Enligt OWASP (2026a) handlar skillnaden mellan DOM-baserad och Reflected XSS främst om var exekveringen sker (OWASP Foundation 2026a; OWASP Foundation 2026b). Vid Reflected XSS skickas användarens indata till servern, som sedan returnerar datan osanerad i HTTP-svaret. Webbläsaren tolkar därefter innehållet och kör koden. I Juice Shops sökfunktion sker inte detta. Applikationen använder hash-routing i Angular, vilket innebär att allt efter # i URL:en hanteras lokalt i webbläsaren och aldrig skickas till servern. Det finns därför inget serversvar som innehåller den skadliga koden.

Istället hämtar JavaScript på klientsidan värdet från URL-parametern och renderar det med hjälp av [innerHTML] tillsammans med bypassSecurityTrustHtml(). Funktionen kringgår Angulars inbyggda säkerhetsskydd och gör att innehållet kan tolkas och köras som kod i webbläsaren. Kombinationen av URL-parameter och osäker DOM-rendering motsvarar enligt OWASP WSTG (WSTG-CLNT-01) en DOM-baserad XSS-sårbarhet (OWASP Foundation 2026d).

Anledningen till att detta förtydligas är att klassificeringen påverkar vilket svar som bedöms som korrekt i rättningsmallen. Om modellen utgår från att sårbarheten är Reflected XSS kommer den sannolikt att föreslå åtgärder på serversidan, trots att problemet egentligen finns i klientkoden. Som resultaten visar är det också precis detta som händer i samtliga analyser för detta testfall.

3.3.2 SQL Injection

En av de granskade rapporterna klassificerar sårbarheten som "Login Bypass via Error-based SQL Injection". Vid genomgång av källkoden framgår att denna klassificering är korrekt. Det är dock viktigt att förtydliga den faktiska grundorsaken, eftersom det påverkar vilket modellsvar som bedöms som korrekt.

Sårbarheten handlar om en SQL-injection (CWE-89, WSTG-INPV-05) som finns på serversidan av inloggningsfunktionen (OWASP Foundation 2026d; OWASP Foundation 2026e). Problemet uppstår eftersom applikationen bygger databasfrågan med stränginterpolering. Värdet från req.body.email placeras därmed direkt i SQL-kommandot utan att först behandlas som data. Databasen kan därför inte skilja användarinmatning från själva SQL-kommandot.

Det är avgörande att förstå att de felmeddelanden som syns i HTTP-svaret bara är ett tecken på problemet och inte själva orsaken. Den verkliga orsaken ligger i bristen på parametriserade frågor. En modell som uteslutande inriktar sig på att hantera fel och föreslår att dölja felmeddelanden har visserligen upptäckt ett underliggande problem, men den missar den grundläggande orsaken. Detta resulterar i poängavdrag enligt rättningsmallen (MITRE Corporation 2026c; OWASP Foundation 2026e).

Sårbarheten är en specifik sårbarhet på serversidan. För att åtgärda den korrekt måste fokus ligga på hur databasfrågan i login.ts byggs upp snarare än hur felen visas för användaren.

3.3.3 Directory Listing Enabled

Rapporten beskriver denna sårbarhet som ”Directory Listing Enabled” och pekar ut / ftp som den berörda slutpunkten. Klassificeringen är korrekt och överensstämmer med CWE-548 samt OWASPs kategori Forced Browsing (OWASP Foundation 2026c).

Sårbarheten finns på serversidan och orsakas inte av en typisk bugg utan av en aktiv konfiguration. Middleware-funktionen serveindex aktiveras för sökvägen /ftp. Vilket gör att servern visar mappens innehåll som en klickbar lista med filer i webbläsaren. Denna funktion är avsedd för utvecklings- och testmiljöer men används här i applikationen utan någon autentisering eller åtkomstkontroll.

Det som särskiljer denna sårbarhet från de andra testfallen är att den grundläggande orsaken ligger i en specifik konfigurationsrad snarare än i applikationslogiken eller hanteringen av indata. En modell som har förståelse för detta bör kunna identifiera det aktuella serveindex-anropet på servern. Den direkta orsaken är att antingen ta bort raden eller säkra sökvägen med autentisering. Modeller som föreslår lösningar för andra webbutvecklingsservrar, såsom Apache eller Nginx, tyder på att de inte har kopplat sin analys till den verkliga kodbasen, vilket resulterar i poängavdrag.

4. RESULTAT

Resultaten i detta kapitel bygger på en manuell bedömning av modellernas svar med stöd av rättningsmallen som beskrivs i avsnitt 3.3. Modellernas fullständiga svar återfinns i Bilagor (2–3) och utgör det råmaterial som bedömningen baseras på.

4.1 Login Bypass via Error-based SQL Injection

Tabell 1: Jämförelse mellan LM Studio och Open WebUI (RAG) för testfallet Login Bypass via Error-based SQL Injection.

Testfall	Modell	Metod	Grundorsak score	Förklaring score	Åtgärd score	Total score
SQLi	Ministral 3 14B Reasoning	LM Studio	2	2	2	6
SQLi	Ministral 3 14B Reasoning	Open WebUI (RAG)	2	2	1	5

LM Studio-metoden fick bäst resultat med 6/6 i jämförelse med RAG-metoden (Open WebUI) som fick 5/6.

Ministral 3 14B via LM Studio levererar ett komplett svar. Den identifierar rätt kodvariabler, visar hur en angripare kan manipulera inloggningen och ger lösningen med korrekt syntax anpassad till systemet.

Ministral 3 14B via Open WebUI (RAG) är nästan lika stark. Förklaringen är mycket detaljerad och modellen visar steg för steg hur attacken fungerar. Den tappar ett poäng på att lösningsförslaget presenterar fel verktyg som huvudalternativ. Den korrekta lösningen finns med men hamnar sist bland flera alternativa förslag och framhävs inte tydligt.

Tabell 2: Jämförelse mellan LM Studio och Open WebUI (RAG) för testfallet Login Bypass via Error-based SQL Injection.

Testfall	Modell	Metod	Grundorsak score	Förklaring score	Åtgärd score	Total score
SQLi	Qwen 3.5 - 9B	LM Studio	2	2	1	5
SQLi	Qwen 3.5 - 9B	Open WebUI (RAG)	2	2	1	5

Både metodernas resultat är 5/6. Det är jämnt utfall men av lite olika anledningar.

Qwen 3.5-9B som kördes av LM Studio förstår problemet mycket väl och kopplar förklaringen tydligt till den aktuella koden i applikationen. Modellen föreslagna åtgärden bygger på rätt säkerhetsprincip men innehåller brist i implementationen. Modellen använder syntaxen **bind:** trots att det aktuella ramverket kräver replacements.

Qwen 3.5-9B som kördes av Open WebUI-metoden identifierar rätt kodplats och ger en lika korrekt analys men använder en generell syntax med platshållare ? som inte är överensstämmer med det bibliotek som används i systemet. Även i detta fall är den teoretiska lösningen korrekt, men den praktiska anpassningen till miljön är ofullständig.

Tabell 3 Jämförelse mellan LM Studio och Open WebUI (RAG) för testfallet Login Bypass via Error-based SQL Injection.

Testfall	Modell	Metod	Grundorsak score	Förklaring score	Åtgärd score	Total score
SQLi	Qwen 2.5-coder-14b	LM Studio	2	2	2	6
SQLi	Qwen 2.5-coder-14b	Open WebUI (RAG)	1	1	1	3

LM Studio-metoden uppnådde högst resultat med 6/6 medan RAG-metoden i Open WebUI fick 3/6.

Qwen 2.5-coder-14B via LM Studio identifierade den korrekt kodrad i applikationen därefter beskrevs tydligt hur sårbarheten kan utnyttjas och föreslog en åtgärd med korrekt syntax anpassad till det bibliotek som faktiskt används i systemet.

Qwen 2.5-coder-14B via Open WebUI med RAG tappade den fokus på grundorsaken. Istället för att uppmärksamma att användarinmatning infogas direkt i databaskommandot fastnade modellen vid att applikationen visar felmeddelanden. Åtgärdsförslaget skrevs dessutom i Java, trots att applikationen är utvecklad i JavaScript, vilket visar att modellen inte lyckades koppla lösningen till rätt teknisk kontext.

Tabell 4 Jämförelse mellan LM Studio och Open WebUI (RAG) för testfallet Login Bypass via Error-based SQL Injection.

Testfall	Modell	Metod	Grundorsak score	Förklaring score	Åtgärd score	Total score
SQLi	Qwen 2.5-coder-7b	LM Studio	2	2	1	5
SQLi	Qwen 2.5-coder-7b	Open WebUI (RAG)	1	1	1	3

LM Studio-metoden fick 5/6 medan Open WebUI-metoden fick 3/6.

Qwen 2.5-coder-7b via LM Studio metoden visar god teknisk förståelse. Den identifierar rätt kodrad samt demonstrerar hur sårbarheten kan utnyttjas och föreslår en korrekt typ av åtgärd. Poängavdraget beror enbart på att syntaxen i lösningen inte helt stämmer med det faktiska ramverket trots att principen är korrekt.

Qwen 2.5-coder-7b via Open WebUI-metoden håller en mer generell nivå. Modellen identifierar att det handlar om SQL Injection men kopplar inte analysen tydligt till den faktiska koden. Lösningsförslaget är dessutom skrivet i Python trots att applikationen byggd i JavaScript. Vilket visar modellen saknar korrekt teknisk kontext.

4.2 Application is vulnerable to SQL Injection

Tabell 5: Jämförelse mellan LM Studio och Open WebUI (RAG) för testfallet Application is vulnerable to SQL Injection.

Testfall	Modell	Metod	Grundorsak score	Förklaring score	Åtgärd score	Total score
----------	--------	-------	------------------	------------------	--------------	-------------

SQLi	Ministral 3 14B Reasoning	LM Studio	2	2	2	6
SQLi	Ministral 3 14B Reasoning	Open WebUI (RAG)	2	1	1	4

Ministral 3 14B via LM Studio levererar väldigt starkt svar. Den citerar rätt kodrad, visar hur attacken fungerar steg för steg och presenterar lösningen med exakt rätt syntax för systemet. Den nämner till och med båda de korrekta alternativen sida vid sida. Det är konsekvent och pålitligt.

Ministral 3 14B via Open WebUI tappar poäng på ett avslöjande sätt. Modellen skriver själv att "koden inte är bifogad i kontexten" och använder därför en påhittad exempelkodrad istället för den verkliga. Modellen vet att den saknar rätt information men ändå att svara, fast på fel grund.

Tabell 6: Jämförelse mellan LM Studio och Open WebUI (RAG) för testfallet *Application is vulnerable to SQL Injection*.

Testfall	Modell	Metod	Grundorsak score	Förklaring score	Åtgärd score	Total score
SQLi	Qwen 3.5-9B	LM Studio	2	2	1	5
SQLi	Qwen 3.5-9B	Open WebUI (RAG)	2	1	1	4

LM Studio-metoden fick bäst resultat med 5/6 i jämförelse med RAG-metoden fick 4/6.

Qwen 3.5-coder-9B via LM Studio levererar ett starkt svar. Den hittar rätt kodrad i applikationen. Den visar ett korrekt exempel på hur en angripare kan utnyttja sårbarheten och ger en lösning som är nära nog rätt. Det enda som drar ner betyget är att det primära lösningsförslaget använder en syntax som tillhör ett annat databassystem.

Qwen 3.5-coder-9B med Open WebUI-metoden är något svagare. Den identifierar grundorsaken korrekt men förklaringen stannar på ytlig nivå, den hittar aldrig den faktiska kodraden och visar inget konkret exempel på hur attacken fungerar i praktiken. Lösningsförslaget har samma problem som LM Studio-metoden men med mer generisk syntax som inte är anpassad för det applikationen faktiskt använder.

Tabell 7: Jämförelse mellan LM Studio och Open WebUI (RAG) för testfallet *Application is vulnerable to SQL Injection*.

Testfall	Modell	Metod	Grundorsak score	Förklaring score	Åtgärd score	Total score
SQLi	Qwen 2.5-coder-7b	LM Studio	2	2	2	6
SQLi	Qwen 2.5-coder-7b	Open WebUI (RAG)	1	0	0	1

LM Studio-metoden fick bäst resultat med 6/6 i jämförelse med RAG metoden som fick 1/6.

Qwen 2.5-coder-7b via LM Studio levererar det starka svaret på SQL Injection bland alla testade modeller och metoder. Den hittar rätt fil, rätt kodrad, rätt variabler, visar ett korrekt attackexempel och ger lösningen i sitt faktiska kodsammanhang med exakt rätt syntax.

Qwen 2.5-coder-7b via Open WebUI är däremot det svagaste SQL Injection-svaret. Förklaringen saknar all koppling till källkoden. Värst av allt, lösningsförslaget saknas helt, texten avslutas mitt i en mening.

Tabell 8: Jämförelse mellan LM Studio och Open WebUI (RAG) för testfallet *Application is vulnerable to SQL Injection*.

Testfall	Modell	Metod	Grundorsak score	Förklaring score	Åtgärd score	Total score
SQLi	Qwen 2.5-coder-14b	LM Studio	1	1	1	3
SQLi	Qwen 2.5-coder-14b	Open WebUI (RAG)	1	1	0	2

LM Studio-metoden fick bäst resultat med 3/6 i jämförelse med RAG metoden som fick 2/6.

Qwen 2.5-coder-14b via LM studio förstår att det handlar om SQL Injection och pekar i rätt riktning men stannar på en ytlig nivå genom hela svaret. Den citerar aldrig den faktiska kodraden, visar inget konkret attack exempel och lösningsförslaget använder fel i bibliotek.

Qwen 2.5-coder-14b via Open WebUI gör ett mer allvarligt misstag. Lösningsförslaget är skrivet i Python med ett SQLite-bibliotek. Applikationen är byggd i JavaScript med ett helt annat databasbibliotek. Det är fel programmeringsspråk, fel databaskoppling och fel miljö, allt på en gång.

4.3 M1: Cross-Site Scripting (XSS) - Reflected

Tabell 9: Jämförelse mellan LM Studio och Open WebUI (RAG) för testfallet *M1: Cross-Site Scripting (XSS) – Reflected*.

Testfall	Modell	Metod	Grundorsak score	Förklaring score	Åtgärd score	Total score
XSS	Ministral 3 14B Reasoning	LM Studio	0	0	0	0
XSS	Ministral 3 14B Reasoning	Open WebUI (RAG)	0	0	0	0

Båda metoderna fick 0/6. Ingen av dem lyckas identifiera den faktiska sårbarheten

Ministral 3 14B via LM Studio och via Open WebUI RAG gör samma fel som övriga modeller: sårbarheten tolkas som Reflected XSS istället för DOM XSS. Att detta sker trots att Ministral är en uttalad reasoning-modell visar att förmågan till djupare resonemang inte hjälper när utgångspunkten är fel.

Open WebUI-metoden kommer något närmare genom att identifiera rätt parameter i applikationen (?q=) men bygger fortfarande hela sin analys på fel premiss och missar de kodrader i klientkoden som orsakar sårbarheten.

Tabell 10: Jämförelse mellan LM Studio och Open WebUI (RAG) för testfallet M1: Cross-Site Scripting (XSS) – Reflected.

Testfall	Modell	Metod	Grundorsak score	Förklaring score	Åtgärd score	Total score
XSS	Qwen 3.5 - 9B	LM Studio	0	0	0	0
XSS	Qwen 3.5 - 9B	Open WebUI (RAG)	0	0	0	0

Båda metoderna fick samma resultat av 0/6.

Qwen 3.5-9B som kördes med LM Studio och Open WebUI gör samma grundläggande felbedömning. Både tolkar sårbarheten som reflected XSS istället för DOM XSS. Vilket leder analysen i fel riktning från början.

Den faktiska sårbarheten finns i klientkod i webbläsaren där applikationen själv inaktiverar sitt inbyggda skydd. Ingen av metoderna identifierar detta. Istället föreslår båda åtgärder på serversidan. Vilket inte adresserar den verkliga orsaken.

Detta tyder på båda metoderna utgick från en felaktig klassificering i rapporten och byggde hela sin analys på den grunden utan att verifiera mot den faktiska koden.

Tabell 11: Jämförelse mellan LM Studio och Open WebUI (RAG) för testfallet M1: Cross-Site Scripting (XSS) – Reflected.

Testfall	Modell	Metod	Grundorsak score	Förklaring score	Åtgärd score	Total score
XSS	Qwen 2.5-coder-14b	LM Studio	0	0	0	0
XSS	Qwen 2.5-coder-14b	Open WebUI (RAG)	0	0	0	0

Båda metoderna fick samma resultat av 0/6.

Qwen 2.5-coder-14b som kördes med LM Studio och Open WebUI gör samma grundläggande felbedömning. Både tolkar sårbarheten som reflected XSS istället för DOM XSS. Vilket från början leder analysen i fel riktning.

Den faktiska sårbarheten finns i klientkod i webbläsaren där applikationen själv inaktiverar sitt inbyggda skydd. Ingen av metoderna identifierar detta. Istället föreslår båda åtgärder på serversidan. Vilket inte adresserar den verkliga orsaken.

Detta tyder på båda metoderna utgick från en felaktig klassificering i rapporten och byggde hela sin analys på den grunden utan att verifiera mot den faktiska koden.

Tabell 12: Jämförelse mellan LM Studio och Open WebUI (RAG) för testfallet M1: Cross-Site Scripting (XSS) – Reflected.

Testfall	Modell	Metod	Grundorsak score	Förklaring score	Åtgärd score	Total score
XSS	Qwen 2.5-coder-7b	LM Studio	0	0	0	0
XSS	Qwen 2.5-coder-7b	Open WebUI (RAG)	0	0	0	0

Båda metoderna fick samma resultat 0/6. Ingen av metoderna lyckades identifiera eller korrekt analysera denna sårbarhet.

Open WebUI-metoden försöker ge en teknisk förklaring men föreslår en lösning i Java och Kotlin trots att applikationen är skriven i JavaScript. Detta visar att modellen inte använder kodbasen för sin analys utan genererar ett generiskt svar om XSS utan kontextuell anpassning. LM Studio-metoden är ännu ytlig och nämner enbart XSS utan vidare teknisk förklaring, kodreferens eller konkret åtgärdsförslag. Svaret saknar både identifiering av specifik kodrad och resonemang kring varför sårbarheten uppstår.

4.4 DOM-Based Cross Site Scripting (XSS)

Tabell 13: Jämförelse mellan LM Studio och Open WebUI (RAG) för testfallet DOM-Based Cross Site Scripting (XSS).

Testfall	Modell	Metod	Grundorsak score	Förklaring score	Åtgärd score	Total score
XSS	Qwen 3.5-9B	LM Studio	2	2	2	6
XSS	Qwen 3.5-9B	Open WebUI (RAG)	1	0	0	1

LM Studio-metoden fick bäst resultat med 6/6 i jämförelse med RAG metoden som fick 1/6.

Qwen 3.5-9B via LM Studio levererar väldigt bra svar på DOM XSS. Modellen hittar rätt kodrader i rätt filer, förklarar exakt hur skyddet stängs av, visar ett konkret attackexempel och presenterar lösningen i sitt verkliga kodsammanhang.

Qwen 3.5-9B via Open WebUI generisk beskrivning utan koppling till den faktiska koden och en lösning som inte ens är anpassad för Angular. Modellen skriver om DOM XSS i allmänhet men analyserar aldrig applikationen.

Tabell 14: Jämförelse mellan LM Studio och Open WebUI (RAG) för testfallet DOM-Based Cross Site Scripting (XSS).

Testfall	Modell	Metod	Grundorsak score	Förklaring score	Åtgärd score	Total score
XSS	Qwen 2.5-coder-7b	LM Studio	1	0	0	1

XSS	Qwen 2.5-coder-7b	Open WebUI (RAG)	0	0	0	0
------------	-------------------	------------------	---	---	---	---

LM Studio fick 1/6 och Open WebUI fick 0/6. Ingen av metoderna presterade bra.

Qwen 2.5-coder-7b via LM Studio hittar Angular-koden och den sårbara funktionen men pekar på fel sårbarhet i samma fil. Den analyserar produktbeskrivningar istället för sökfältet. Ännu allvarigare är att den föreslår “bypassSecurityTrustHtml()” som lösningen. Vilket är exakt det som orsakar problemet.

Qwen 2.5-coder-7b via Open WebUI håller sig på en helt generisk nivå utan någon koppling till den faktiska koden. Lösningsförslaget kombinerar fel metoder på ett som fortfarande lämnar applikationen sårbar.

Tabell 15: Jämförelse mellan LM Studio och Open WebUI (RAG) för testfallet DOM-Based Cross Site Scripting (XSS).

Testfall	Modell	Metod	Grundorsak score	Förklaring score	Åtgärd score	Total score
XSS	Qwen 2.5-coder-14b	LM Studio	1	1	1	3
XSS	Qwen 2.5-coder-14b	Open WebUI (RAG)	0	0	0	0

LM Studio-metoden fick bäst resultat med 3/6 i jämförelse med RAG metoden som fick 0/6.

Qwen 2.5-coder-14b via LM Studio hittade den rätta funktionen “bypassSecurityTrustHtml()” och förstår varför den är farlig. Problemet är att den pekar på fel ställe i koden. Den analyserar produktbeskrivningar istället för sökfältet. Lösningen blir därför ofullständig och riktad mot fel del av applikationen.

Qwen 2.5-coder-14b via Open WebUI nämner inte den relevanta källkoden och ger generiska råd utan koppling till Angular. Åtgärdsförslaget kombinerar fel tekniker på ett sätt som inte löser problemet och nämner dessutom RegEx-sanitering som en del av grundorsaken. Något som inte alls stämmer med verkligheten.

Tabell 16: Jämförelse mellan LM Studio och Open WebUI (RAG) för testfallet DOM-Based Cross Site Scripting (XSS).

Testfall	Modell	Metod	Grundorsak score	Förklaring score	Åtgärd score	Total score
XSS	Ministral 3 14B Reasoning	LM Studio	2	2	2	6
XSS	Ministral 3 14B Reasoning	Open WebUI (RAG)	0	0	0	0

LM Studio-metoden fick bäst resultat med 6/6 i jämförelse med Open WebUI-metoden fick 0/6.

Ministral 3 14B via LM Studio levererar ett perfekt svar. Det andra i testserien på denna sårbarhet. Den identifierar rätt funktion, förklarar exakt varför Angular-skyddet kringgås, visar ett konkret attackexempel och presenterar båda de korrekta lösningarna i sitt faktiska kodsammanhang. Den påpekar dessutom korrekt varför detta är DOM XSS och inte Reflected XSS.

Ministral 3 14B via Open WebUI gör ett allvarligt misstag och föreslår ``bypassSecurityTrustHtml()`` som lösning, vilket är exakt den funktion som orsakar problemet. Det är ett allvarligare fel än att bara svara fel, eftersom en utvecklare som följer rådet aktivt skulle göra applikationen mer sårbar.

4.5 M3: Directory Listing Enabled

Tabell 17: Jämförelse mellan LM Studio och Open WebUI (RAG) för testfallet M3: Directory Listing Enabled.

Testfall	Modell	Metod	Grundorsak score	Förklaring score	Åtgärd score	Total score
Directory listing	Ministral 3 14B Reasoning	LM Studio	1	1	1	3
Directory listing	Ministral 3 14B Reasoning	Open WebUI (RAG)	1	1	1	3

Båda metoderna fick samma resultat 3/6.

Ministral 3 14B Reasoning via båda metoderna visar god förståelse för vad directory listing innebär och varför det är en säkerhetsrisk. Däremot identifierar ingen av dem den specifika kodrad i server.ts som orsakar sårbarheten.

LM Studio-metoden föreslår en med `express.static` och ett visst inställningsvärde. Förslaget framstår som rimligt men åtgärdar inte den faktiska orsaken i applikationens konfiguration.

Open WebUI-metoden nämner autentisering som ett alternativ lösning. Vilket delvis överensstämmer med en av de korrekta åtgärderna men utan koppling till den aktuella kodbasen.

Tabell 18: Jämförelse mellan LM Studio och Open WebUI (RAG) för testfallet M3: Directory Listing Enabled.

Testfall	Modell	Metod	Grundorsak score	Förklaring score	Åtgärd score	Total score
Directory listing	Qwen 3.5 - 9B	LM Studio	1	1	1	3
Directory listing	Qwen 3.5 - 9B	Open WebUI (RAG)	1	1	1	3

Båda metoderna fick samma resultat 3/6.

Qwen 3.5-9B via LM studio via LM Studio och via Open WebUI gör i princip samma misstag och landar på samma poäng. Både förstår vad directory listning är och kan förklara varför det är farligt men ingen av dem hittar den faktiska kodraden som orsakade problemet.

LM Studio-metoden föreslår lösningar för Nginx och Apache och nämner även egen middleware-lösningar men inget av dem träffar rätt.

Open WebUI-metoden håller sig till Apache och IIS, men nämner åtminstone autentisering som ett alternativ för legitima användningsfall. Vilket delvis stämmer med en av de korrekta lösningarna i facit. Det är ett plus men inte tillräckligt för att skilja metoderna åt.

Tabell 19: Jämförelse mellan LM Studio och Open WebUI (RAG) för testfallet M3: Directory Listing Enabled.

Testfall	Modell	Metod	Grundorsak score	Förklaring score	Åtgärd score	Total score
Directory listing	Qwen 2.5-coder-14b	LM Studio	1	1	1	3
Directory listing	Qwen 2.5-coder-14b	Open WebUI (RAG)	1	0	0	1

LM Studio-metoden fick bäst resultat med 3/6 i jämförelse med RAG-metoden (Open WebUI) som fick 1/6.

Qwen 2.5-coder-14b som kördes av LM Studio-metoden visar förståelse för vad sårbarheten innebär och förklarar den på ett begripligt sätt. Dock föreslår den lösningar för helt andra typer av webbservrar än den som faktiskt används i applikationen. För att förstå vad detta innebär så är det som att föreslå ett reservdäck till en cykel när bilen har punktering, det vill säga att man fattat problemet men ger fel verktyg/lösning.

Open WebUI-metoden gör ett mer allvarligt fel. Den hänvisar till en konfigurationsfil "web.config" som inte existerar i källkoden. När analysen bygger på en komponent som saknas i kodbasen faller både förklaring och åtgärdsförslag oavsett hur rimliga de kan verka i teorin. Detta leder till ett betydligt lägre resultat.

Tabell 20: Jämförelse mellan LM Studio och Open WebUI (RAG) för testfallet M3: Directory Listing Enabled.

Testfall	Modell	Metod	Grundorsak score	Förklaring score	Åtgärd score	Total score
Directory listing	Qwen 2.5-coder-7b	LM Studio	1	1	1	3
Directory listing	Qwen 2.5-coder-7b	Open WebUI (RAG)	1	1	1	3

Båda metoderna fick samma resultat 3/6.

Open WebUI-metoden föreslår den lösningar för helt andra typer av webbservrar än den som faktiskt används i applikationen. Modellen visar därmed förståelse för vad directory listing innebär men kopplar problemet till fel teknisk miljö.

LM Studio-metoden är tekniskt närmare den korrekta lösningen räcker den inte för ett Övergripande förståelse utan den precision som krävs.

4.6 Sammanställning av resultat

Tabell 21: Sammanställning av totalpoäng för samtliga testfall, modeller och metoder (max 6 poäng per analys)

Testfall	Modell	LM Studio	Open WebUI (RAG)
SQLi – Login Bypass	Ministral 3 14B	6	5
SQLi – Login Bypass	Qwen 3.5-9B	5	5
SQLi – Login Bypass	Qwen 2.5-coder-14b	6	3
SQLi – Login Bypass	Qwen 2.5-coder-7b	5	3
SQLi – App Vulnerable	Ministral 3 14B	6	4
SQLi – App Vulnerable	Qwen 3.5-9B	5	4
SQLi – App Vulnerable	Qwen 2.5-coder-7b	6	1
SQLi – App Vulnerable	Qwen 2.5-coder-14b	3	2
XSS – Reflected (M1)	Ministral 3 14B	0	0
XSS – Reflected (M1)	Qwen 3.5-9B	0	0
XSS – Reflected (M1)	Qwen 2.5-coder-14b	0	0
XSS – Reflected (M1)	Qwen 2.5-coder-7b	0	0
XSS – DOM-Based	Ministral 3 14B	6	0
XSS – DOM-Based	Qwen 3.5-9B	6	1
XSS – DOM-Based	Qwen 2.5-coder-14b	3	0
XSS – DOM-Based	Qwen 2.5-coder-7b	1	0
Directory Listing	Ministral 3 14B	3	3
Directory Listing	Qwen 3.5-9B	3	3
Directory Listing	Qwen 2.5-coder-14b	3	1
Directory Listing	Qwen 2.5-coder-7b	3	3
Totalt		70	38

4.7 Analys av prestandamönster

Resultat i tabell 21 visar tydliga mönster och inte slumpmässiga variationer.

Indatakvalitet är den starkaste faktorn

Det mest tydliga mönstret är att samtliga modeller och metoder fick 0/6 på testfallet M1 (Reflected XSS), där rapporten innehöll en felaktig klassificering. Ingen modell ifrågasatte klassificeringen eller verifierade den mot källkoden. I stället byggde alla vidare på den felaktiga utgångspunkten och föreslog lösningar på serversidan för ett problem som i själva verket fanns i klientkoden. Detta visar att indata-kvalitet har större påverkan på analysförmågan än både modellval och metod.

LM Studio presterar bättre när kontexten är direkt och specifik

LM Studio-metoden nådde totalt 70 poäng medan Open WebUI fick 38 poäng.

Skillnaden kan till stor del förklaras av hur kontexten tillhandahölls. I LM Studio bifogades relevanta källkodsfiler, vilket gav modellen direkt tillgång till den kod som var kopplad till sårbarheten. Open WebUI:s RAG-pipeline hämtade i stället de tre kodsegment som hade högst semantisk likhet med frågan. I flera fall räckte detta inte för att inkludera den kodrad som faktiskt låg bakom problemet. I ett av testfallen påpekade modellen själv att relevant kod saknades i kontexten men genererade ändå ett svar baserat på en påhittad kodrad. Det är ett tydligt exempel på hallucination, trots att RAG är avsett för att minska just denna typ av fel.

Sårbarheters natur påverkar metodernas resultat

Vid SQL Injection, där grundorsaken är tydligt avgränsad till en specifik kodrad presterade LM Studio väldigt bra och flera modeller nådde 6/6. För Directory Listing, där problemet snarare handlar om en konfigurationsinställning än applikationslogik, var resultaten mer jämförbara mellan metoderna och landade oftast på 3/6. Detta tyder på att RAG fungerar bättre i situationer där analysen inte kräver identifiering av en specifik kodrad utan i högre grad bygger på en mer generell förståelse av konfigurationsprinciper.

Modellstorlek och specialisering ger inte alltid bättre resultat

Minstral 3 14B Reasoning och Qwen 3.5-9B presterade generellt bäst, men mönstret är inte helt entydigt. Qwen 2.5-coder-7B nådde exempelvis 6/6 i ett av SQL Injection-testfallen via LM Studio, medan den större Qwen 2.5-coder-14B stannade på 3/6 i samma testfall via Open WebUI. Detta tyder på att tillgång till rätt kontext väger tyngre än modellstorlek i sig. En mindre modell med direkt åtkomst till relevant källkod kan därmed prestera bättre än en större modell där retrieval-processen inte lyckas tillhandhålla tillräckligt relevant underlag.

5. DISKUSSION

Syftet med examensarbetet var att undersöka hur väl en lokal AI-baserad lösning kan analysera penetrationstestrapporter tillsammans med källkod för att identifiera grundorsaker, förklaringar och förslag på åtgärder för säkerhetssårbarheter. Genom att utveckla en proof of concept och testa fem olika testfall med fyra modeller och två metoder gick det att dra slutsatser om lösningens styrkor, begränsningar och möjliga användningsområden.

5.1 Tolkning av resultat

Det mest betydelsefulla resultatet är inte att modellerna ibland misslyckas, utan snarare orsaken till dessa misslyckanden. Felen uppvisar ett tydligt mönster: när indata är korrekta och kontexten enkelt kan nås, fungerar modellerna effektivt. När något av dessa villkor inte uppfylls försämras analyserna snabbt och på ett tydligt sätt. Det är särskilt betydelsefullt att samtliga modeller och metoder fick betyget 0/6 i det felklassificerade XSS-testfallet. Det handlar inte om att modellerna är omedvetna när det XSS; flera av dem visade faktiskt en bra förståelse för DOM XSS-testfallet.

Problemet ligger istället i att de okritiskt accepterar klassificeringen i rapporten utan att själva kontrollera den mot koden. Det tyder på att modellerna i sin nuvarande utformning fungerar mer som analytiska verktyg än som kritiska granskare. De kan följa ett resonemang men startar inte egna. Denna skillnad är viktig för hur lösningen ska tillämpas i praktiken. En organisation som inför ett liknande system bör vara medveten om att kvaliteten på analyserna beror direkt på kvaliteten på de rapporter som skickas in. AI:n förstärker informationen som den får oavsett om denna är korrekt eller felaktig.

5.2 AI:ns beroende av rapportens information

Modellerna fungerar som analytiker som utgår från att den informationen de får är korrekt, snarare än att själva ifrågasätta och kontrollera om underlaget faktiskt stämmer. I säkerhetskritiska sammanhang blir den här skillnaden särskilt betydelsefull. Om en organisation automatiserar sin sårbarhetsanalys med ett liknande system finns det risk att felaktiga åtgärdsförslag uppfattas som riktiga, just eftersom svaren ofta är välformulerade och tekniskt övertygande. Problemet är att slutsatserna i sådana fall bygger på information som redan från början är felaktig.

Risken blir ännu större eftersom verkliga penetrationsrapporter ofta innehåller mänskliga misstag, otydliga beskrivningar eller varierande terminologi beroende på vem som har skrivit rapporten.

För att minska risken för liknande fel i framtiden skulle AI-baserade säkerhetsanalysverktyg kunna utvecklas så att de aktivt jämför rapportens påståenden med den underliggande källkoden innan analysen genomförs. Om skillnader mellan rapport och kod upptäcks, skulle systemet kunna markera detta för användaren. Eftersom de lokala modellerna i denna studie inte visade någon tydlig förmåga att utföra en sådan kontroll utan vägledning, behövs ytterligare forskning om hur sådana verifieringsfunktioner kan implementeras och utvärderas.

5.3 Metodens begränsningar

Studien genomfördes som en experimentell jämförelse mellan två olika metoder för AI-baserad säkerhetsanalys: en RAG-baserad lösning i Open WebUI och en mer manuellt styrd lösning i LM Studio. En tydlig styrka i upplägget var att samma modeller, prompts och hårdvara användes i båda miljöerna. På så sätt minskade risken att resultaten skulle påverkas av skillnader i testmiljön, vilket gjorde det enklare att undersöka vilken betydelse informationsåtkomsten faktiskt hade.

Ytterligare en styrka av studiens metod var att rättningsmallen utvecklades av examensarbetets författare med stöd av OWASP WSTG, MITRE CWE samt den faktiska källkoden. Samma bedömningsmetod användes genom hela studien för att skapa en så konsekvent utvärdering som möjligt. Trots detta finns fortfarande ett visst inslag av subjektiv tolkning i bedömningen av modellsvaren, särskilt i de fall där resonemangen endast var delvis korrekta. En mer standardiserad och externt validerad bedömningsmall hade sannolikt kunnat ytterligare stärka studiens tillförlitlighet.

Studien omfattade också endast fem testfall baserade på OWASP Juice Shop. Därför går resultaten inte utan vidare att generalisera till andra typer av applikationer, programmeringsspråk eller produktionsmiljöer. Eftersom det är en medvetet sårbar applikation som tagits fram för utbildning och säkerhetstestning kan vissa sårbarheter dessutom vara tydligare och pedagogiskt utformade än de vanligtvis är i verkliga system.

En annan begränsning var att Open WebUI användes med standardinställningar för retrieval, chunkning och embedding. Syftet med detta var att undvika manuella optimeringar och samtidigt skapa en mer reproducerbar testmiljö. Det innebär dock att resultaten inte nödvändigtvis speglar den högsta möjliga prestandan för den RAG-baserade metoden. Andra retrieval-strategier eller embedding-modeller hade sannolikt kunnat påverka utfallet.

Studien avgränsades även till lokala språkmodeller. Kommersiella modeller som GPT-4 eller Claude inkluderades inte inom ramen för arbetet, vilket innebär att resultaten inte kan användas för att jämföra lokala modeller med större molnbaserade AI-system.

Sammantaget innebär dessa begränsningar att resultaten bör tolkas med viss försiktighet. De fyra modeller som testades representerar endast en liten del av lokala språkmodeller som finns tillgängliga idag. Det är möjligt att modeller med annan arkitektur, större parameterstorlek eller domänspecifik träning hade presterat annorlunda. Resultaten visar därför främst hur de testade modellerna presterade under studiens förutsättningar. Det ska inte generaliseras till alla lokala språkmodellers förmåga att analysera sårbarheter. Fältet utvecklas i snabb takt, vilket innebär att modeller som lanseras efter denna studie kan ha förbättrade förmågor att hantera de utmaningar som identifierats här.

5.4 Implikationer

Resultaten från studien visar att lokala AI-modeller kan fungera som ett stöd vid analys av penetrationstestrappor och källkod. Det här kan vara särskilt relevant för organisationer som arbetar med känslig information och därför vill undvika att dela källkod eller säkerhetsrapporter med externa molntjänster. Genom att köra modellerna lokalt går det att behålla kontrollen över datahanteringen och samtidigt minska risker kopplade till sekretess och olika regelverk.

Samtidigt framgår det att AI-baserad säkerhetsanalys fortfarande har tydliga begränsningar när det gäller tillförlitlighet. I flera av fallen genererade modellerna lösningar som vid första inblick verkade tekniskt rimliga, men som i praktiken visade sig vara felaktiga. Detta blev extra tydligt när retrieval-steget inte fungerade som tänkt eller när rapporterna innehöll felklassificerade sårbarheter. AI bör därför främst ses som ett verktyg som kan stötta säkerhetsanalytiker i deras arbete, snarare än något som kan ersätta mänsklig kompetens helt och hållet.

Studien pekar även på att de arkitekturella valen påverkar resultaten i stor utsträckning. En större mängd information innebär inte automatiskt bättre analysförmåga. I vissa fall gav en mindre och mer noggrant utvald kontext bättre resultat än en automatiserad retrieval-process som hämtade information från en hel kodbas. Det här är något som kan vara relevant för organisationer som vill implementera AI-baserade säkerhetsverktyg i praktiken.

Ur ett forskningsperspektiv bidrar studien med empiriska resultat kring hur lokala LLM-modeller presterar vid säkerhetsanalys av penetrationstestrappor och källkod. Området är fortfarande ganska nytt, och jämfört med större molnbaserade AI-system finns det än så länge relativt begränsat med forskning om lokala modeller.

5.5 Relevans i ett större sammanhang

Resultaten är relevanta eftersom AI-verktyg blir allt vanligare inom säkerhetsarbete. Studien visar att lokala LLM-modeller kan ge tekniskt korrekta analyser för tydliga och korrekt klassificerade sårbarheter, men att de inte kan ersätta mänsklig expertis. Detta gäller särskilt vid bedömning av rapportklassificeringar och teknisk kontext.

Den konfigurationsagnostiska designen gör dessutom lösningen flexibel, eftersom AI-komponenten kan bytas ut mellan lokala modeller och API-baserade tjänster beroende på organisationens behov och krav.

En tydlig fördel med lokala modeller är att känsligt material aldrig lämnar organisationens infrastruktur. Penetrationstestrappor och källkod innehåller ofta detaljerad information om sårbarheter i produktionssystem, och om sådant material skickas till externa API-tjänster finns en risk för informationsläckage. Lokala modeller minskar den risken och kan därför vara ett mer lämpligt alternativ i säkerhetskänsliga miljöer.

5.6 LM Studio-metoden kontra Open WebUI-metoden

Ett tydligt mönster i studien var att LM Studio-metoden presterade bättre än eller lika bra som Open WebUI-metoden med RAG. Skillnaden märktes främst i testfall där retrieval-steget inte lyckades hämta rätt källkod.

I ett av fallen noterade modellen själv att rätt kod saknades i kontexten och skapade istället en påhittad kodrad som grund för analysen. Det är ett tydligt exempel på hallucination trots att RAG är tänkt att minska just sådana problem. LM Studio-metoden undvek detta eftersom modellen fick direkt tillgång till rätt källkodsfil och kunde ge mer precisa analyser.

För Directory Listing var resultaten däremot likvärdiga mellan metoderna. Alla modeller fick 3/6 oavsett metod. Det tyder på att RAG fungerar bättre när analysen inte är lika beroende av specifik kod utan mer av generell konfigurationsinformation. Valet av metod verkar därför bero på vilken typ av sårbarhet som analyseras.

5.7 Framtida arbete

Det finns flera möjligheter att bygga vidare på studien. Ett naturligt nästa steg är att optimera RAG-konfigurationen. Genom att testa högre Top-K-värden, bättre chunk-strategier och reranking-metoder för att förbättra retrieval-steget. Eftersom standardinställningar användes i denna studie finns det sannolikt utrymme för förbättring.

Det hade också varit intressant att jämföra lokala modeller med API-baserade modeller som GPT-4 eller Claude för att undersöka skillnader i prestanda när säkerhet och datakonfidentialitet prioriteras. En sådan jämförelse hade kunnat ge organisationer bättre underlag vid val av AI-lösning

Framtida studier bör även undersöka hur lösningen fungerar i mer komplexa system där analysen kräver spårning av dataflöden mellan flera filer och tjänster. Det hade gett en tydligare bild av AI-modellers faktiska styrkor och begränsningar i verkliga penetrationstestmiljöer.

6. SLUTSATSER OCH VIDARE FORSKNING

I detta kapitel sammanfattas examensarbetets viktigaste slutsatser utifrån forskningsfrågorna och resultaten från experimenten. Studien undersökte hur en lokal AI-baserad lösning kan användas för att analysera penetrationstestrapporter tillsammans med källkod för att identifiera grundorsaker till säkerhetssårbarheter.

6.1 Slutsats

Studien utgick från följande forskningsfrågor:

1. I vilken utsträckning kan en lokal AI-baserad lösning identifiera rimliga grundorsaker, förklaringar och åtgärdsförslag till sårbarheter baserat på penetrationstestrapporter och källkod?
2. Vilka fördelar och begränsningar finns vid användning av lokala AI-modeller för säkerhetsanalys med fokus på prestanda och säkerhet?
3. Vilka skillnader i prestanda finns mellan Open WebUI-metoden och LM Studio-metoden vid analys av penetrationstestrapporter och källkod?

Resultaten visar att lokala AI-modeller kan identifiera rimliga grundorsaker och åtgärdsförslag för tydliga och korrekt klassificerade sårbarheter. För SQL Injection lyckades flera modeller ge tekniskt korrekta analyser anpassade till systemets implementation. Samtidigt misslyckades alla modeller helt i testfallet där rapporten innehöll en felaktig klassificering i rapporten. Ingen modell verifierade informationen mot koden, vilket visar att analysförmågan är starkt beroende av kvaliteten på indata.

Forskningsfrågorna kan sammanfattas enligt följande:

1. Lokala AI-modeller kan ge relevanta analyser och åtgärdsförslag för korrekt klassificerade sårbarheter, men de är känsliga för felaktigt underlag och kan inte självständigt verifiera rapportens innehåll mot koden.
2. Lokala modeller ger en säkerhetsfördel eftersom känsligt material stannar inom organisationens infrastruktur. Samtidigt kräver större och mer avancerade modeller högre krav på systemresurser och prestanda, vilket kan vara en begränsning för vissa organisationer.
3. LM Studio-metoden presterade generellt bättre än eller lika bra som Open WebUI-metoden. Retrieval-steget visade sig vara en kritisk och ibland opålitlig del av processen, vilket i vissa fall ledde till hallucinationer och sämre analyser.

Sammantaget visar studien att lokala AI-modeller kan fungera som ett stöd vid säkerhetsanalys men inte som ett helt självständigt system. Mänsklig granskning behövs fortfarande för att säkerställa att analyserna är korrekta och relevanta.

6.2 Framtida forskning

Eftersom studien genomfördes med ett begränsat antal testfall och rapporter från samma applikation finns det flera möjligheter att vidareutveckla forskningen. Framtida studier bör inkludera fler typer av penetrationstestrapporter och system för att göra resultaten mer generaliserbara.

Det vore även intressant att undersöka hur RAG-systemets fulla potential kan utnyttjas genom att inte enbart ge modellen tillgång till källkod, utan även komplettera med relevanta

säkerhetsstandarder och riktlinjer som en del av kunskapsbasen. Vid analys av exempelvis XSS-sårbarheter hade modellen kunnat få tillgång till OWASP:s regler, rekommendationer och mallar för säker hantering av XSS-sårbarheter. Det hade kunnat ge modellen bättre kontext vid analysen och samtidigt minska risken för felaktiga eller alltför generella åtgärdsförslag.

Slutligen bör framtida forskning undersöka hur lösningen fungerar i större och mer komplexa system samt jämföra lokala modeller med API-baserade modeller. Det kan bidra till en bättre förståelse för vilken typ av AI-lösning som lämpar sig bäst i olika säkerhets- och organisationsmiljöer.

REFERENSLISTA

- Bau, J., Bursztein, E., Gupta, D. & Mitchell, J. (2010). State of the Art: Automated Black-Box Web Application Vulnerability Testing. *I 2010 IEEE Symposium on Security and Privacy*. Oakland, CA, USA 16-19 maj 2010, s. 332-345. ki10.1109/SP.2010.27
- Beljulji, E. & Gür, G. (2025). Large Language Models in security code review and testing. *Journal of Systems Research (JSys)*, [förhandspublicerad online].
- Card, D. N. (1998). Learning from our mistakes with defect causal analysis. *IEEE Software*, 15(1), s. 56-63. doi:10.1109/52.646883
- Chillarege, R., Bhandari, I. S., Chaar, J. K., Halliday, M. J., Moebus, D. S., Ray, B. K. & Wong, M.-Y. (1992). Orthogonal defect classification—A concept for in-process measurements. *IEEE Transactions on Software Engineering*, 18(11), s. 943–956. doi:10.1109/32.177364
- Ciavatta, J. A. S., Bermejo Higuera, J. R., Bermejo Higuera, J., Sicilia Montalvo, J. A., Sureda Riera, T. & Pérez Melero, J. (2026). Integration of Large Language Models (LLMs) and Static Analysis for Improving the Efficacy of Security Vulnerability Detection in Source Code. *Computers, Materials & Continua*, 86(3), s. 1-40. doi:10.32604/cmc.2025.074566
- Du, X. et al. (2026). Vul-RAG: Enhancing LLM-based Vulnerability Detection via Knowledge-level RAG. *ACM Transactions on Software Engineering and Methodology*, [publicerad online 20 februari 2026]. doi:10.1145/3797277
- Hack & Fix Academy (2025). *Juice Shop – Web Application Security Assessment Report*. <https://academy.hackandfix.com/wp-content/uploads/2025/06/Juice-Shop-Web-Application-Security-Assessment-Report.pdf>
- Ishikawa, K. (1986). *Guide to quality control*. Asian Productivity Organization. doi:10.1002/qre.4680010318
- Kalinowski, M., Card, D. & Travassos, G. (2012). Evidence-based guidelines to defect causal analysis. *IEEE Software*, 29(4), s. 16–18. doi:10.1109/MS.2012.72
- Khare, A., Dutta, S., Li, Z., Solko-Breslin, A., Alur, R. & Naik, M. (2023). Understanding the Effectiveness of Large Language Models in Detecting Security Vulnerabilities. *arXiv*, [förhandspublicerad online]. doi:10.48550/arXiv.2311.16169
- Klesel, M. & Wittmann, H. F. (2025). Retrieval-Augmented Generation (RAG). *Business & Information Systems Engineering*, 67(4), s. 551–561. doi:10.1007/s12599-025-00945-3
- Lehtinen, T., Mäntylä, M. & Vanhanen, J. (2011). Development and evaluation of a lightweight root cause analysis method. *Journal of Systems and Software*, 84(4), s. 607–620. doi:10.1016/j.infsof.2011.05.005
- McKinnel, D. R., Dargahi, T., Dehghantanha, A. & Choo, K.-K. R. (2019). A systematic literature review and meta-analysis on artificial intelligence in penetration testing and vulnerability assessment. *Computers and Electrical Engineering*, 75, s. 175–188. doi:10.1016/j.compeleceng.2019.02.022

MITRE Corporation (2026a). CWE-548: *Exposure of Information Through Directory Listing*. <https://cwe.mitre.org/data/definitions/548.html> [2026-05-10].

MITRE Corporation (2026b). CWE-79: *Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')*. <https://cwe.mitre.org/data/definitions/79.html> [2026-05-10].

MITRE Corporation (2026c). CWE-89: *Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')*. <https://cwe.mitre.org/data/definitions/89.html> [2026-05-10].

OWASP Foundation (2026a). *Cross Site Scripting (XSS)*. <https://owasp.org/www-community/attacks/xss/> [2026-05-10].

OWASP Foundation (2026b). *DOM Based XSS Prevention Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/DOM_based_XSS_Prevention_Cheat_Sheet.html [2026-05-10].

OWASP Foundation (2026c). *Forced Browsing*. https://owasp.org/www-community/attacks/Forced_browsing [2026-05-10].

OWASP Foundation (2026d). *OWASP Web Security Testing Guide*. <https://owasp.org/www-project-web-security-testing-guide/> [2026-02-27].

OWASP Foundation (2026e). *SQL Injection*. https://owasp.org/www-community/attacks/SQL_Injection [2026-05-10].

OWASP Foundation (2026f). *SQL Injection Prevention Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html [2026-05-10].

Pratama, D., Suryanto, N., Adiputra, A. A., Le, T.-T.-H., Kadiptya, A. Y., Iqbal, M. & Kim, H. (2024). CIPHER: Cybersecurity Intelligent Penetration-Testing Helper for Ethical Researcher. *Sensors*, 24(6878). doi:10.3390/s24216878

Ridoy, S. Z., Shaon, M. S. H., Cuzzocrea, A. & Akter, M. S. (2024). EnStack: An Ensemble Stacking Framework of Large Language Models for Enhanced Vulnerability Detection in Source Code. *arXiv*, [förhandspublicerad online]. doi:10.48550/arXiv.2411.16561

Sarker, K. U., Yunus, F. & Deraman, A. (2023). Penetration Taxonomy: A Systematic Review on the Penetration Process, Framework, Standards, Tools, and Scoring Methods. *Sustainability*, 15(13), 10471. doi:10.3390/su151310471

Scarfone, K., Souppaya, M., Cody, A. & Orebaugh, A. (2008). *Technical Guide to Information Security Testing and Assessment (NIST Special Publication 800-115)*. National Institute of Standards and Technology. doi:10.6028/NIST.SP.800-115

Schutzwirk GmbH (2021). *Web Application Penetration Test Report*. <https://www.schutzpunkt.com/reports/pentest-webapplication-en.pdf>

Tamberg, K. & Bahsi, H. (2025). Harnessing Large Language Models for Software Vulnerability Detection: A Comprehensive Benchmarking Study. *IEEE Access*, 13, s. 29698-29717. doi:10.1109/ACCESS.2025.3541146

Xu, H., Wang, S., Li, N., Wang, K., Zhao, Y., Chen, K., Yu, T., Liu, Y. & Wang, H. (2024). Large Language Models for Cyber Security: A Systematic Literature Review. *arXiv*, [förhandspublicerad online]. doi:10.48550/arXiv.2405.04760

BILAGOR

Bilaga 1. Login Bypass via Error-based SQL Injection (Schutzwerk GmbH rapport)

4 Findings

H1: Login Bypass via Error-based SQL Injection (SQLi)

Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:H/A:L
State	Open
Immediate Action recommended	Yes !!
Quick Win to Fix	Yes ✨
Tags	WSTG-INPV-05, CWE-89, ATT&CK-T1190
Affected Components	https://juice-shop.lab/rest/user/login

Summary

The application exposes database error information when processing untrusted input, enabling error-based SQL injection. Attackers can trigger SQL interpreter errors and use the returned error messages to bypass the login.

Impact

This allows any visitor to the website to log in as an administrator using this vulnerability.

Furthermore, disclosure of detailed database errors enables targeted inference of query structure and schema. Attackers can use error content to extract sensitive data, modify records, or bypass logical controls that depend on query outcomes. Persistent exposure of SQL errors increases the probability of full data compromise for affected tables and related entities.

Recommendation

Enforce safe handling of all user-controlled inputs and stop exposing database error details to end users.

Technical Description

Untrusted input is concatenated into SQL queries without parameterization, allowing malformed SQL to reach the database engine. When evaluation fails, the database emits detailed error messages that the application returns in its HTTP responses or UI, leaking internal query context. Error-based SQL injection leverages these disclosed errors to iteratively learn the structure of the underlying queries and schema. Disclosed content commonly includes SQLSTATE codes, driver names, exception class and stack traces, table or column identifiers, function names, and type mismatch details. These signals enable precision adjustments of input values, revealing columns, types, and constraints until meaningful data retrieval or modification becomes feasible. The root cause is string-based query construction and permissive error handling that surfaces low-level database exceptions to end users.

Secondary factors include inconsistent input validation and lack of enforced parameterized data access patterns across the codebase. The example below demonstrates a generic server response that reveals a database error without exposing environment-specific details.

```
HTTP/1.1 500 Internal Server Error
Content-Type: text/plain; charset=utf-8

Error executing query on /items?id=<value>
SQLSTATE[42000]: Syntax error or access violation: 1000: actual DB error excerpt (e.g.,
column not found, type mismatch)
Driver: 1000: driver/version if disclosed | Query fragment: 1000: fragment if disclosed
```

Typical indicators of error-based SQL injection include database error codes in responses, leaked identifiers and data types, and traces linking directly to query construction paths. When present, these signals materially reduce uncertainty for an attacker, speeding the path to data compromise.

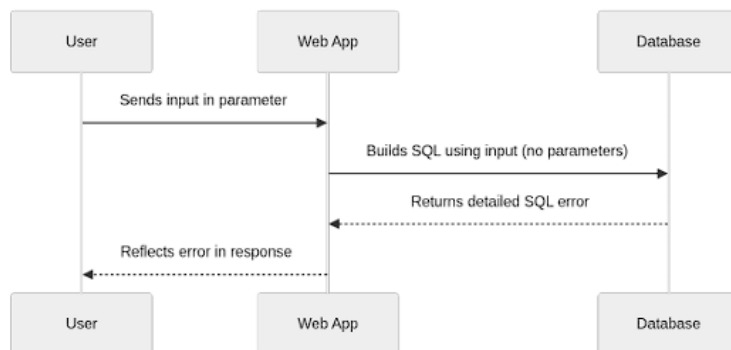


Figure 2 - Error-based SQLi signal flow

Evidence

In Figure 3, you can see that an SQL error is triggered if the email address contains a `'`.

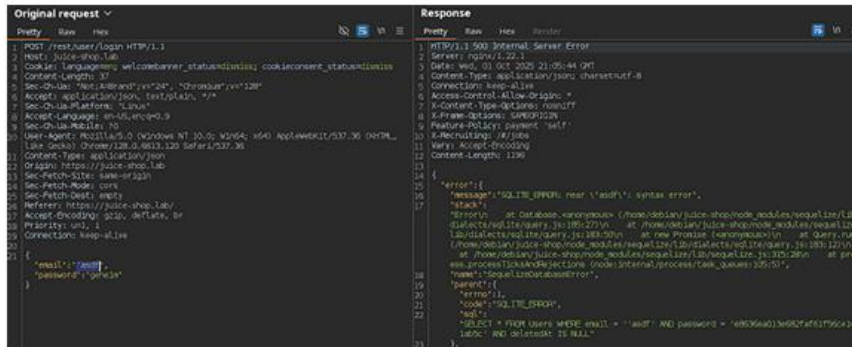


Figure 3 - SQL error in email

By cleverly choosing the username ' or 1 = 1; -- ', an SQL injection occurs, allowing the login to be bypassed. Finally, you are logged in as an administrator (see Figure 3).

The following happens in the background when the query is assembled:

1. ' ' closes the email address field.
2. Afterward the inserted SQL code is then used.
3. or 1 = 1 is a true statement.
4. ; terminates the SQL query.
5. The rest of the original statement is commented out with --.

```
SELECT * FROM Users WHERE email = '' or 1 = 1; --' AND password =
'e8636ea013e682faf61f56ce1cb1ab5c' AND deletedAt IS NULL
```

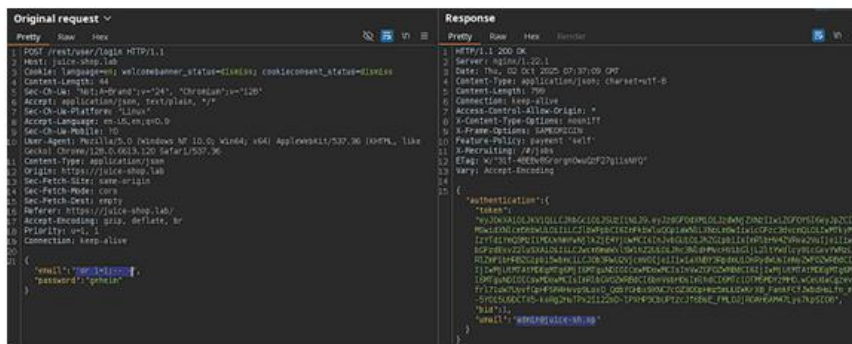


Figure 4 - SQL injection login bypass

This then allows successful access to the administration interface at <https://juice-shop.lab/#/administration> (see Figure 5). Here, for example, all other users can be viewed and reviews can be deleted.

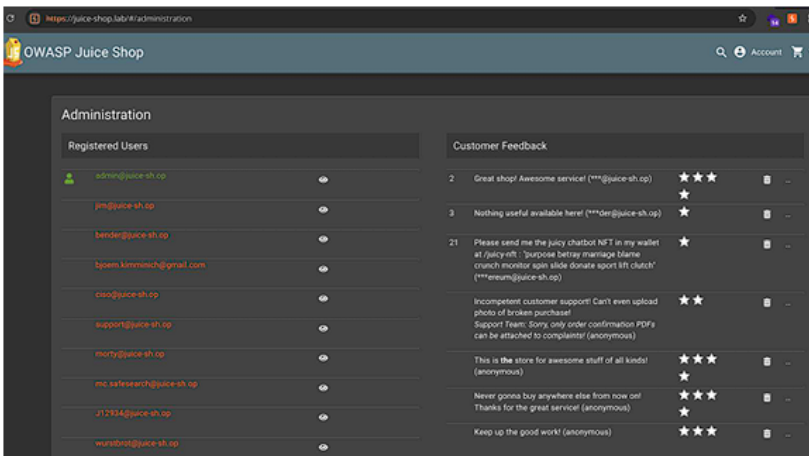


Figure 5 - Access to administration interface

Technical Recommendation

Replace string-concatenated SQL with prepared statements and bind variables for every query that uses user-controlled data. Enforce parameterization via a shared data access layer or ORM configuration and prohibit ad-hoc query construction in code reviews and CI checks. Implement generic error handling that maps database exceptions to standardized responses (e.g., HTTP 500) without revealing SQLSTATE, identifiers, or stack traces, and log full details server-side with correlation IDs. Disable verbose error pages and detailed exception messages in production configurations for the web framework and database drivers. Validate and normalize inputs according to strict type and length constraints before reaching the data layer, rejecting unexpected formats early. Inventory and refactor all endpoints that include user input in queries, adding tests to assert that errors are not reflected and that queries are parameterized. Add static analysis or lint rules to detect string-built SQL and enforce prepared statement usage across the codebase. Example (conceptual):

```
// Before: vulnerable
String sql = "SELECT * FROM items WHERE id = " + id_variable; // concatenation
stmt.executeQuery(sql);

// After: parameterized
PreparedStatement ps = conn.prepareStatement("SELECT * FROM items WHERE id = ?");
ps.setInt(1, id_variable);
ps.executeQuery();
```

References

- https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/07-Input_Validation_Testing/05-Testing_for_SQL_Injection
- https://owasp.org/www-community/attacks/SQL_Injection
- <https://cwe.mitre.org/data/definitions/89.html>
- <https://portswigger.net/web-security/sql-injection>

Bilaga 2. M1: Cross-Site Scripting (XSS) - Reflected (Schutzwerk GmbH rapport)

6.1 M1: Cross-Site Scripting (XSS) - Reflected

Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N
State	Open
Immediate Action recommended	No
Quick Win to Fix	Yes ✳
Tags	WSTG-CLNT-01, CWE-79
Affected Components	https://juice-shop.lab/#/search?q=

Summary

The application reflects user-supplied input into the response without proper context-aware output encoding. This enables an attacker to craft a link or request that executes attacker-controlled script in the victim's browser within the application. The issue is limited to transient responses and requires user interaction, but consequences can include account compromise, data exposure, and unauthorized actions performed via the victim's session.

Impact

Successful exploitation allows execution of attacker-controlled script in the victim's browser in the application. Potential outcomes include session misuse if tokens are accessible to script, unauthorized actions performed on behalf of the user, exposure of on-page sensitive data, and trusted UI manipulation that facilitates convincing in-origin phishing. The attack requires user interaction (for example, following a crafted link or submitting data), and impact can extend to any user who engages with the crafted input.

Recommendation

Apply strict context-aware output encoding for all reflected data and enforce a restrictive Content Security Policy if possible.

Technical Description

Reflected Cross-Site Scripting occurs when user-controlled data is immediately echoed back in a server response without appropriate output encoding for its rendering context. Typical sources include query parameters, path segments, form fields, or headers that are inserted into HTML, attributes, URLs, or inline scripts via string concatenation. The root cause is the absence or bypassing of context-aware output encoding and the use of unsafe rendering patterns that treat untrusted data as trusted markup or script. Framework auto-escaping may be disabled, misconfigured, or circumvented by unsafe templating constructs. Improper content handling, such as returning HTML with untrusted data while using a permissive response type, can further increase exposure, though the core flaw is the lack of correct encoding. A restrictive Content Security Policy (CSP) can limit impact but does not remediate the underlying issue. The example below shows a parameter reflected into HTML content without encoding, illustrating how untrusted input is rendered by the browser:

```
GET /search?q=<untrusted_input> HTTP/1.1
Host: TODO: host
User-Agent: TODO
Accept: text/html
```

```
<!doctype html>
<html>
  <body>
    <p>Search for: <untrusted_input></p>
  </body>
</html>
```

Evidence

- Affected endpoint and parameter:
 - URL: <https://juice-shop.lab/#/search>
 - Parameter: *q*

In Figure 6, you can see that the input `asdf` in the search field is mirrored in the Search Results context of the web application.

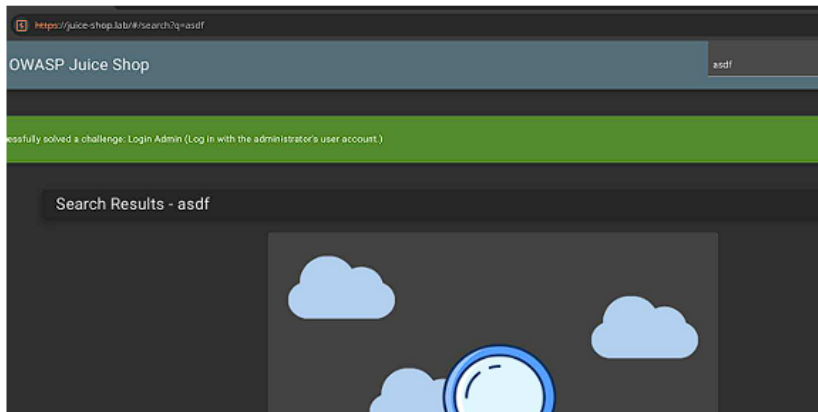


Figure 6 - Reflected input from search field

Entering the following JavaScript code displays a pop-up message confirming the execution of this code (see Figure 7).

```
<iframe%20src%3D"javascript:alert('Protection point%20XSS')">
```

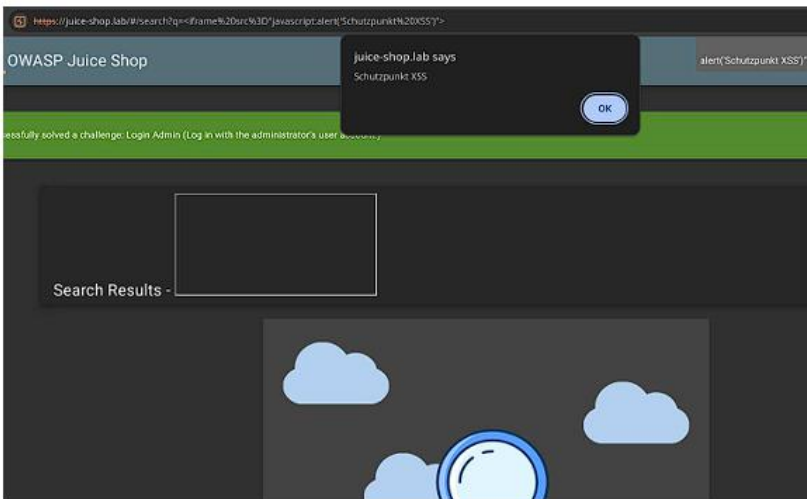


Figure 7 - Reflected XSS

Figure 8 shows the affected location in the frontend code of the application.

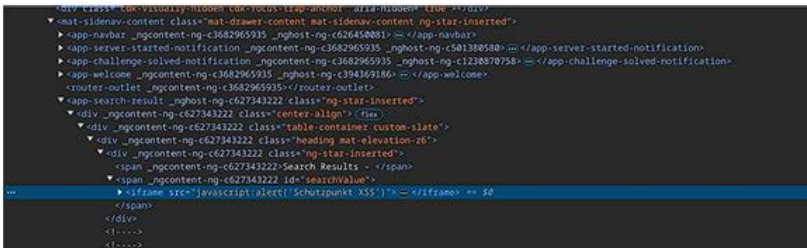


Figure 8 - Location in the code with XSS

Technical Recommendation

Identify every location where untrusted input is reflected into responses and ensure context-aware output encoding is applied before rendering. Use framework features that auto-escape by default, and avoid concatenating untrusted data into HTML, attributes, URLs, CSS, or JavaScript contexts. Apply the correct encoder for the specific context, such as HTML text, HTML attribute, URL parameter, or JavaScript string, instead of relying on generic or blacklist filters. Avoid rendering untrusted input as HTML; use safe templating and DOM APIs that handle text nodes rather than markup. Deploy a restrictive CSP that disallows inline script and limits script sources to vetted origins to reduce impact if an encoding gap remains. Add unit and integration tests that verify reflected fields are encoded and that dangerous characters are rendered harmless in their specific contexts.

Example (illustrative):

```
// Template or server-side rendering
String safe = org.owasp.encoder.Encode.forHtml(userInput);
out.print(safe);
```

References

- <https://owasp.org/www-community/attacks/xss/>
- https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/07-Input_Validation_Testing/01-Testing_for_Reflected_Cross_Site_Scripting
- <https://cwe.mitre.org/data/definitions/79.html>

Bilaga 3. M3: Directory Listing Enabled (Schutzwerk GmbH rapport)

5.3 M3: Directory Listing Enabled

Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
State	Open
Immediate Action recommended	No
Quick Win to Fix	Yes ✱
Tags	CWE-548, CWE-200
Affected Components	https://juice-shop.lab/ftp

Summary

The web server is configured to allow directory listing on one or more public paths. This exposes the names and structure of files and subdirectories to any visitor, even without authentication. File listings increase information disclosure and make reconnaissance and targeted attacks easier.

Impact

Unrestricted directory browsing can reveal sensitive files, configuration remnants, backups, and credentials stored within web-accessible paths. Exposed filenames and structure enable faster reconnaissance, targeted brute-force against discovered assets, and discovery of overlooked administrative endpoints. Information leakage can aid subsequent attacks such as credential harvesting from configuration files, exploitation of outdated components identified in listings, and unauthorized download of proprietary content. If search engines index these listings, exposure persists beyond the immediate audience and increases long-term risk.

Recommendation

Disable directory listing on all publicly accessible web paths.

Technical Description

Directory listing occurs when the server returns an automatically generated index page for a directory that lacks a default index file such as index.html. It typically results from enabling features like Apache mod_autoindex, Nginx autoindex, or IIS Directory Browsing, or from default configurations that are not hardened. When enabled, a request to a directory path (for example, /assets/) yields a page listing files, sizes, timestamps, and subdirectories. These listings reveal internal file naming, build artifacts, backups, temporary files, and configuration fragments that were not intended to be publicly accessible. Attackers and automated crawlers can iterate directories to map the application structure and locate sensitive files faster. Publicly reachable listings require no authentication and may be cached or indexed by search engines, increasing unintended exposure. This behavior is often accidental and persists until directory indexing is explicitly disabled or a default index file is provided.

Example request and response demonstrating an enabled listing.

```
GET /assets/ HTTP/1.1
Host: example.com
```



```

HTTP/1.1 200 OK
Content-Type: text/html

<!DOCTYPE html>
<html>
<head><title>Index of /assets/</title></head>
<body>
<h1>Index of /assets/</h1>
<a href="backup.zip">backup.zip</a>
<a href=".env">.env</a>
<a href="config.old">config.old</a>
</body>
</html>

```

Evidence

- Observed URLs with directory listing enabled:
 - <https://juice-shop.lab/ftp>

As can be seen in Figure 12, several sensitive files can also be read under the above path, such as:

- incident-support.kdbx
- suspicious_errors.yml
- coupons_2013.md.bak



Figure 12 - Directory Listing Enabled

Technical Recommendation

Turn off directory indexing at the web server level for all public sites and applications, and enforce least exposure for any path that must remain accessible. In Apache HTTP Server, disable indexes globally or per directory by removing autoindex and using a restrictive Options directive.

```

# httpd.conf or .htaccess
Options -Indexes
# Ensure mod_autoindex is not loaded if not needed
# LoadModule autoindex_module modules/mod_autoindex.so # comment or remove

```

In Nginx, disable autoindex for relevant locations or server blocks.

```

server {
    location / {
        autoindex off;
    }
}

```

```
}  
}
```

In IIS, disable Directory Browsing at the site or application level, or enforce it via web.config.

```
<!-- web.config -->  
<configuration>  
  <system.webServer>  
    <directoryBrowse enabled="false" />  
  </system.webServer>  
</configuration>
```

If a listing is required for a legitimate use case, restrict it to authenticated, role-limited users and isolate it from public routes. Validate that no sensitive files reside within web-accessible directories and deploy a default index page where appropriate to prevent auto-generated listings.

References

- <https://cwe.mitre.org/data/definitions/548.html>
- https://owasp.org/Top10/A05_2021-Security_Misconfiguration
- https://httpd.apache.org/docs/current/en/mod/mod_autoindex.html
- https://nginx.org/en/docs/http/nginx_http_autoindex_module.html

Bilaga 4. DOM-Based Cross Site Scripting (XSS) (Hack & Fix Academy rapport)

5.7. DOM-Based Cross Site Scripting (XSS)

 CVSS Score: 6.8	Likelihood Medium	Remediation Effort Easy
Risk Statement	Attackers might be able to exploit XSS attacks leading to user accounts compromise	
Category	Application	
Description	DOM-based XSS vulnerabilities usually arise when JavaScript takes data from an attacker-controllable source, such as the URL, and passes it to a sink that supports dynamic code execution. A successful XSS exploit would allow an attacker to rewrite web pages and insert malicious scripts which would alter the intended output. This could include HTML fragments, CSS styling rules, arbitrary JavaScript, or references to third-party code. An attacker could use this to steal users' passwords, collect personal data such as credit card details, provide false information, or run malware. From the victim's point of view, this is performed by the genuine website, and the victim would blame the site for incurred damage. An additional risk with DOM XSS is that unlike reflected or stored XSS, tainted values do not have to go through the server. Since the server is not involved in the sanitization of these inputs, server-side validation is not likely to not be aware XSS attacks have been occurring, and any server-side security solutions, such as a WAF, are likely to be ineffective in DOM XSS mitigation. The application creates web pages that include untrusted data, whether from user input, the application's database or from other external sources. The untrusted data is embedded directly in the page's HTML, causing the browser to display it as part of the web page. If the input includes HTML fragments or JavaScript, these are displayed too, and the user cannot tell that this is not the intended page. The vulnerability is the result of directly embedding arbitrary data without first encoding it in a format that would prevent the browser from treating it like HTML or code instead of plain text. When a DOM XSS occurs, it is the client-side code itself that manipulates the local web page's DOM and introduces malicious content.	
Risk and Business Impact	When attackers successfully exploit XSS vulnerabilities in a web application, they can insert a script that gives them access to end-users account credentials and browsers. In some special conditions, from a client-side type, the attack may become a server-side issue. Some of the most dangerous attacks carried out using XSS are Redirection, Clickjacking, URL Spoofing, Session hijacking, Cookie stuffing, Phishing, Ad Hijacking, CSRF/XSRF attacks, History stealing, XSS	

	defacement, Key & Mouse Logging, Browser hijacking, Port Scanning, DDoS, XSS Tunneling and others.
How to Reproduce	• Open the affected application • Insert the XSS payload in vulnerable parameters • Open the page where the XSS is reflected
Recommendation	Fully encode all dynamic data, regardless of source, before embedding it in output. Encoding should be context-sensitive. For example: • HTML encoding for HTML content • HTML Attribute encoding for data output to attribute values • JavaScript encoding for server-generated JavaScript It is recommended to use the platform-provided encoding functionality, or known security libraries for encoding output. • Implement a Content Security Policy (CSP) with explicit whitelists for the application's resources only. • As an extra layer of protection, validate all untrusted data, regardless of source (note this is not a replacement for encoding). Validation should be based on a whitelist: accept only data fitting a specified structure, rather than reject bad patterns. Check for: ◦ Data type ◦ Size ◦ Range ◦ Format ◦ Expected values • In the Content-Type HTTP response header, explicitly define character encoding (charset) for the entire page. • Set the HTTPOnly flag on the session cookie for "Defense in Depth", to prevent any successful XSS exploits from stealing the cookie.
Impacted Assets	http://localhost:3000/#/search?q=
Proof of Concept	The following evidence has been gathered to illustrate this vulnerability:
Access the search bar and enter the following payload: <pre><iframe src="javascript:alert('xss')"></pre>	

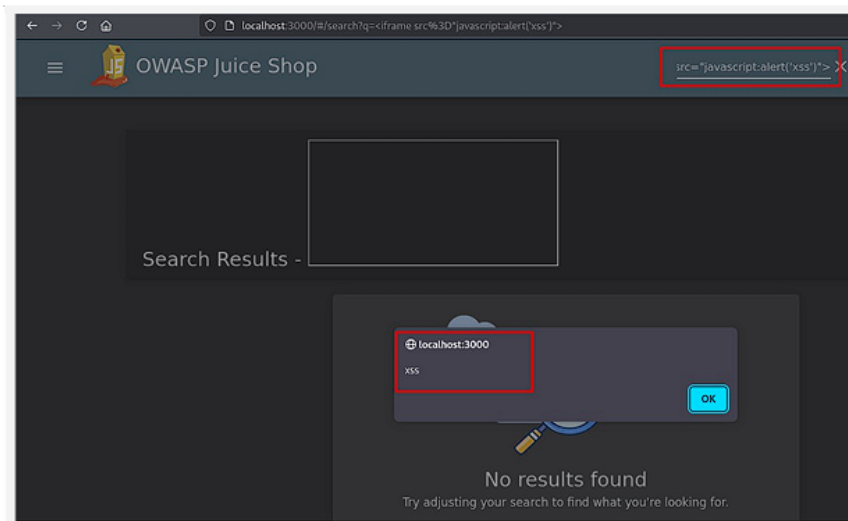


Figure 17 - The XSS payload is executed

An alert box with the message 'xss' should appear, indicating a successful DOM-based XSS attack.

Web Request

N/A

Web Response

N/A

References

- https://www.owasp.org/index.php/DOM_Based_XSS
- https://en.wikipedia.org/wiki/Cross-site_scripting

Bilaga 5. Application is vulnerable to SQL Injection (Hack & Fix Academy rapport)

5. Finding Details

5.1. Application is vulnerable to SQL Injection

 CVSS Score: 8.9	Likelihood High	Remediation Effort Moderate
Risk Statement	A successful SQL injection attack can result in unauthorized access to sensitive data, such as passwords, credit card details, or personal user information.	
Category	Application	
Description	The application constructs an SQL command using externally-influenced input from the application, but it does not neutralize special elements that could modify the intended SQL command when it is sent to the database component. SQL injection is a web security vulnerability that allows an attacker to interfere with the queries that an application makes to its database. It generally allows an attacker to view data that they are not normally able to retrieve. This might include data belonging to other users, or any other data that the application itself is able to access. In many cases, an attacker can modify or delete this data, causing persistent changes to the application's content or behavior. Without sufficient removal or quoting of SQL syntax in user-controllable inputs, the generated SQL query can cause those inputs to be interpreted as SQL instead of ordinary user data. This can be used to alter query logic to bypass security checks, or to insert additional statements that modify the back-end database, possibly including the execution of system commands. There are a wide variety of SQL injection vulnerabilities, attacks, and techniques, which arise in different situations. Some common SQL injection attacks include Classic SQLi, Blind or Inference SQL injection, Database management system-specific SQLi.	
Risk and Business Impact	An attacker can escalate the attack to compromise the underlying server or other back-end infrastructure or perform a denial-of-service attack. Many high-profile data breaches in recent years have been the result of SQL injection attacks, leading to reputational damage and regulatory fines. In some cases, an attacker can obtain a persistent backdoor into an organization's systems, leading to a long-term compromise that can go unnoticed for an extended period.	
How to Reproduce	• Submit the single quote character (') and look for errors or other anomalies. • Submit some SQL-specific syntax	

	• Submit Boolean conditions such as OR 1=1 and OR 1=2 and look for differences in the application's responses.
Recommendation	Most instances of SQL injection can be prevented by using parameterized queries (also known as prepared statements) instead of string concatenation within the query. With most development platforms, parameterized statements that work with parameters can be used (sometimes called placeholders or bind variables) instead of embedding user input in the statement. A placeholder can only store a value of the given type and not an arbitrary SQL fragment. Hence the SQL injection would simply be treated as a strange (and probably invalid) parameter value. Parameterized queries can be used for any situation where untrusted input appears as data within the query, including the WHERE clause and values in an INSERT or UPDATE statement. They can't be used to handle untrusted input in other parts of the query, such as table or column names, or the ORDER BY clause. Application functionality that places untrusted data into those parts of the query will need to take a different approach, such as white-listing permitted input values, or using different logic to deliver the required behavior. For a parameterized query to be effective in preventing SQL injection, the string that is used in the query must always be a hard-coded constant, and must never contain any variable data from any origin. Do not be tempted to decide case-by-case whether an item of data is trusted, and continue using string concatenation within the query for cases that are considered safe. It is all too easy to make mistakes about the possible origin of data, or for changes in other code to violate assumptions about what data is tainted.
Impacted Assets	http://localhost:3000/#/login
Proof of Concept	The following evidence has been gathered to illustrate this vulnerability:
Insert the following payload in the login form under the email field: <pre>' OR 1=1 --</pre> Use any random password, then click Login:	

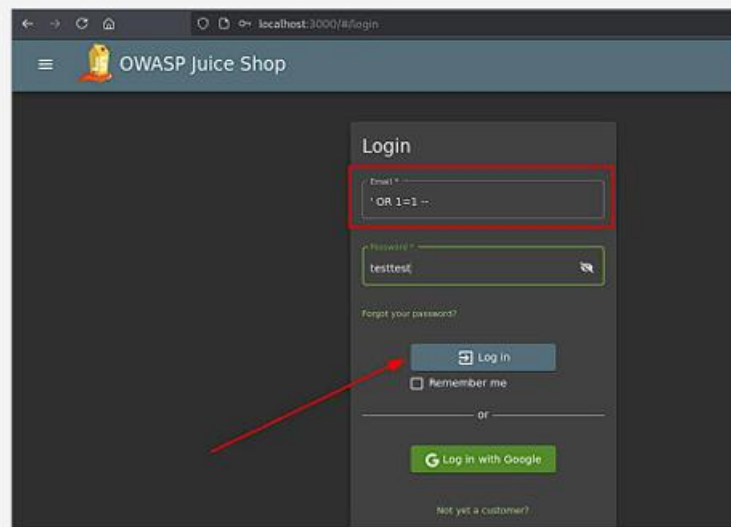


Figure 1 - SQL injection in Email field

After exploiting this vulnerability, the user will be authenticated as admin:

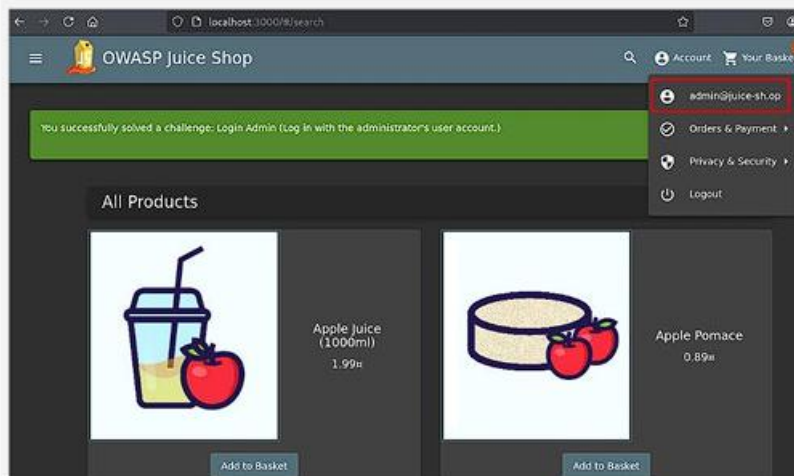


Figure 2 - Login as admin user

Using Burp Suite tool, the request was intercepted and saved locally in request.txt file:

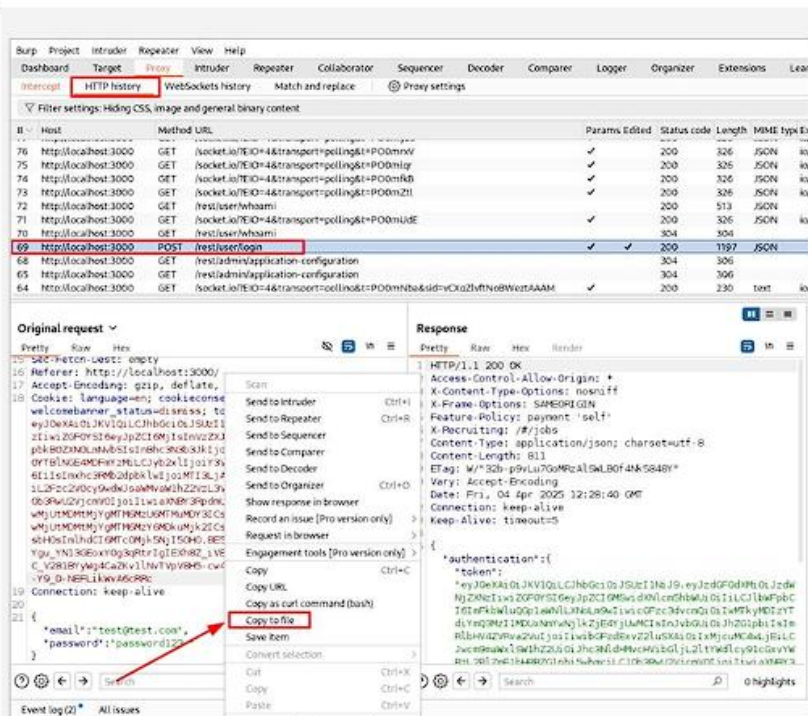


Figure 3 - Saving the login request in a local file

Further, automatic exploitation was performed using SQLMap tool:
 sqlmap -r request.txt -level=5 -risk=3 -batch -ignore-code 401

After execution, SQLMap shows that the JSON email parameter is vulnerable to SQL injection:

```
[15:51:15] [INFO] checking if the injection point on (custom) POST parameter 'JSON email' is a false positive
(custom) POST parameter 'JSON email' is vulnerable. Do you want to keep testing the others (if any)? [y/N] N
sqlmap identified the following injection point(s) with a total of 582 HTTP(s) requests:
Parameters: JSON email ((custom) POST)
  Type: boolean-based blind
  Title: OR boolean-based blind - WHERE or HAVING clause (NOT)
  Payload: ("email": "test@test.com" OR NOT 8988=8988 -- MeAE", "password": "password123 ")

  Type: time-based blind
  Title: SQLite > 2.0 OR time-based blind (heavy query)
  Payload: ("email": "test@test.com" OR 3890=LIKE(CHAR(65,66,67,68,69,70,71),UPPER(HEX(RANDOMBLOB(500000000/2))))-- THGA",
  d": "password123 ")

[15:51:15] [INFO] the back-end DBMS is SQLite
back-end DBMS: SQLite
[15:51:15] [WARNING] HTTP error codes detected during run:
401 (Unauthorized) - 370 times, 500 (Internal Server Error) - 202 times
[15:51:15] [INFO] fetched data logged to text files under "/home/kali/.local/share/sqlmap/output/localhost"
[*] ending @ 15:51:15 /2025-04-04/
```

Figure 4 - JSON email parameter is vulnerable to SQL injection

After performing reconnaissance to discover the database tables and columns, we dumped the content of columns email and password from table Users:

```
sqlmap -r request.txt -level=5 -risk=3 -batch -ignore-code 401 -T Users -C email,password --dump
```

```
Database: <current>
Table: Users
[27 entries]
+-----+-----+
| email | password |
+-----+-----+
| NULL | 0192c...18b500 (ac...) |
| NULL | e541c...3e5e45 (n...) |
| NULL | 0c36c...44a4ef |
| NULL | 6edd...757b8c |
| 2vDuG9dLetPVfKFRhK98BWF7EeP@acrku.com | 8619...81a8fb |
| J12934@juice-sh.op | 3869...36bc82 |
| accountant@juice-sh.op | f2f9...d6d9e8 |
| admin@juice-sh.op | b03f...953bc8 |
| amy@juice-sh.op | 3c2a...714b7d |
| bender@juice-sh.op | 9ad5...941de4 |
| bjoern.kimminich@gmail.com | 030f...de0473 |
| bjoern@juice-sh.op | 7f31...1d6810 |
| bjoern@owasp.org | 9283...62e466 |
| chris.pike@juice-sh.op | 10a7...f0095b |
| ciso@juice-sh.op | 963e...d636dc |
| demo | 05f9...b8f1af |
| emma@juice-sh.op | fe01...04e229 (d...) |
| ethereum@juice-sh.op | 0047...8e4d45 |
```

Figure 5 - Dumping users and password hashes

Web Request

N/A

Web Response

N/A

References

- https://www.owasp.org/index.php/SQL_Injection_Prevention_Cheat_Sheet
- <https://portswigger.net/web-security/sql-injection>

Bilaga 6. Rättningsmall för sårbarhetsidentifiering

SQL Injection – login

Grundorsak

Applikationen bygger sin databassökning genom att klistra in det användaren skriver direkt i själva databaskommandot. Det som användaren skriver i fälten req-body.email och req.body.password behandlas inte bara som vanligt text, utan blir en del av instruktionen som skickas till databasen. Enligt riktlinjer OWASP Foundation (2026e) ska användarens inmatning alltid behandlas som data och aldrig som en del av ett kommando. I det här fallet skiljs dessa inte åt, vilket skapar en möjlighet för angriparen att mata in innehåll som databasen tolkar som ett nytt kommando snarare än som vanlig text.

Förklaring

Sårbarheten uppstår genom följande händelsekedja:

1. Användaren skriver in e-postadress och lösenord i inloggningsfältet.
2. Dessa värden placeras sedan direkt in i SQL-strängen:

```
SELECT * FROM Users WHERE email = '${req.body.email}' AND password = '${hash(...)}'
```

Om en angripare skriver exempelvis:

' OR 1=1 –

kommer den faktiska SQL-frågan att se ut så här:

```
SELECT * FROM Users WHERE email = " OR 1=1 -- AND password = '...'
```

Uttrycket OR 1=1 gör att villkoret alltid blir sant, medan – kommenterar bort resten av frågan. Resultatet blir att databasen returnerar en användare utan att korrekt autentisering har genomförts.

Detta är ett klassiskt exempel på SQL Injection (MITRE Corporation 2026c; OWASP Foundation 2026e) där databasen tolkar angriparens indata som en del av SQL-språket istället för som vanlig text.

Föreslagen åtgärd

Följ rekommendationen från OWASP SQL Injection Prevention Cheat Sheet OWASP Foundation (2026f): Använd parametriserade queries (prepared statements) där data skickas separat från SQL-kommandot.

Den rekommenderade åtgärden enligt OWASP SQL Injection Prevention Cheat Sheet är att använda parametriserade queries, så kallade prepared statements, där användardata skickas separat från själva SQL-kommandot. På så sätt behandlas inmatningen som data istället för som en del av

databasinstruktionen.

I stället för att bygga SQL-frågan med stränginterpolering:

```
sequelize.query(  
  `SELECT * FROM Users WHERE email = '${email}' AND password = '${hashedPassword}'`  
)
```

bör parameterbindning användas via Sequelize:

```
sequelize.query(  
  'SELECT * FROM Users WHERE email = :email AND password = :password',  
  {  
    replacements: {  
      email: email,  
      password: hashedPassword  
    }  
  }  
)
```

Ett ännu säkrare alternativ är att använda ORM-metoden där frågan byggs av ramverket istället för manuellt:

```
UserModel.findOne({  
  where: {  
    email: email,  
    password: hashedPassword  
  }  
})
```

Genom att separera data från SQL-kommandot förhindras att databasen tolkar användarens inmatning som en del av SQL-språket, vilket minskar risken för SQL Injection.

DOM XSS (search)

Grundorsak

Sårbarheten uppstår eftersom applikationen hanterar användarens inmatning på ett osäkert sätt. Text läses in direkt från webbadressen utan kontroll eller rensning och visas därefter på sidan på ett sätt som gör att webbläsaren tolkar innehållet som kod i stället för vanlig text.

Detta möjliggörs genom två kodbeteenden som samverkar. I TypeScript-koden hämtas söktexten direkt från URL-parametern q, och i HTML-mallen visas innehållet med [innerHTML]. När [innerHTML] används instrueras webbläsaren att tolka innehållet som HTML eller JavaScript. Det är kombinationen av dessa två delar som skapar sårbarheten.

Förklaring

Sårbarheten uppstår genom följande händelseförlopp:

1. En angripare skriver in skadlig kod i sökfältet, exempelvis:
`<iframe src="javascript:alert('xss')">`
2. Den inmatade texten hamnar i webbläsarens adressfält som ?q=....
3. Applikationen hämtar värdet och stänger av sitt eget skydd med funktionen `bypassSecurityTrustHtml()`
4. Texten visas på sidan med [innerHTML], vilket gör att webbläsaren tolkar innehållet som faktiskt kod.
5. Webbläsaren exekverar koden och angriparen får därmed möjlighet att köra egen kod i offrets webbläsare.

Detta är ett typiskt exempel på DOM-baserad XSS (MITRE Corporation 2026b; OWASP Foundation 2026a) där exekveringen orsakas av klientens, alltså webbläsarens egen kodhantering.

Föreslagen åtgärd

För att eliminera sårbarheten behöver kedjan brytas på två nivåer.

1. Åtgärd i HTML – ta bort osäkra visningen

Visningen som tolkar innehållet som kod:

```
<span [innerHTML]="searchValue"></span>
```

Bör ersättas med en säker variant som enbart renderar text:

```
<span>{{ searchValue }}</span>
```

När texten visas på detta sätt försöker webbläsaren inte längre tolka innehållet som HTML eller JavaScript.

2. Åtgärd i TypeScript – undvik okontrollerad inläsning från URL

Istället för att kringgå Angulars inbyggda säkerhet med:

```
This.searchValue = this.sanitizer.bypassSecurityTrustHtml(queryParam)
```

Bör värdet användas direkt utan att skyddsmekanismen stängs av:

```
This.searchValue = queryParam
```

Genom att både undvika [innerHTML] och låta Angulars inbyggda skydd vara aktivt minskar risken för att användarinmatning tolkas och körs som kod i webbläsaren.

Directory Listing (/ftp)

Grundorsak

Sårbarheten uppstår eftersom servern är konfigurerad att visa innehållet i en servermapp direkt i webbläsaren. Detta sker genom att middleware-funktionen `serverIndex` är aktiverad för sökvägen `/ftp`.

Funktionen är i första hand avsedd för utveckling och testmiljöer, men i det här fallet är den aktiv i applikationen och gör det möjligt för alla som besöker adressen att se innehållet i mappen. Det strider mot OWASP:s rekommendationer kring Forced Browsing (OWASP Foundation 2026c) och motsvarar svagheten MITRE Corporation (2026a) (CWE-548 Information Exposure Through Directory Listing).

Förklaring

Sårbarheten uppstår genom följande händelseförlopp:

1. Användaren skriver in adressen:
`http://localhost:3000/ftp`
2. Servern innehåller följande kod:
`App.use('/ftp', serveIndex('/ftp', { icons: true } ))`
3. Denna kod instruerar servern att:
 - a. Öppna mappen `ftp` på servern
 - b. Visa hela innehållet i webbläsaren som en klickbar fillista
4. Webbläsaren tolkar detta som en vanlig webbsida där användaren kan:
 - a. Se samtliga filer
 - b. Klicka på och ladda ned dem
 - c. Identifiera filer som aldrig var avsedda att vara publikt tillgängliga

Konsekvensen blir att känslig eller interna filer exponeras utan att användaren behöver vara inloggad eller ha någon särskild behörighet.

Föreslagen åtgärd

Den bakomliggande orsaken till sårbarheten är att `serveIndex` har aktiverats. För att åtgärda problemet behöver funktionen antingen tas bort eller skyddas.

Alternativ 1 – ta bort funktionen helt

Ta bort denna rad:

```
App.use('/ftp', serveIndex('ftp', { icons: true })))
```

Alternativ 2 – skydda med autentisering och auktorisation

Om katalogen behöver vara tillgänglig bör den placeras bakom inloggning och behörighetskontroll så att endast auktoriserade användare kan få åtkomst till innehållet.

Bilaga 7. Sammanställning av testresultat – Open WebUI-metoden (RAG systemet)

Testtyp	Modell	Rapport	Svar (Grundorsak)	Svar (Förklaring)	Svar (Åtgärd förslag)
Login Bypass via Error-based SQL Injection (SQLi)	Qwen 2.5-coder-14b	Pentest rapport SchutzPunkt.pdf	Den mest sannolika grundorsaken till sårbarheten är att applikationen exponerar detaljerade databasfelinformationer när den bearbetar osäkra indata, vilket möjliggör felbaserad SQL-injection.	Sårbarheten uppstår eftersom applikationen inte hanterar SQL-frågor på ett säkert sätt. I källkoden ses att SQL-frågor konstrueras genom att osäkra användarinmatning direkt infogas i frågan, vilket gör det möjligt för en angripare att utnyttja detta för att injecta egna SQL-kod challenges.yml . I pentestrapporten beskrivs hur ett särskilt användarnamn (' or 1 = 1; --) kan användas för att skapa en felaktig SQL-fråga, vilket leder till att administratörsinloggningen omgår SECURITY.md . Databasfelinformationerna som exponeras hjälper angriparen att identifiera och utnyttja sårbarheterna.	En möjlig säkerhetsförbättring är att använda förberedda satser (prepared statements) istället för att konstruera SQL-frågor genom strängsammanfogning. Förberedda satser hjälper till att undvika SQL-injection genom att separera SQL-kod från användarinmatning. Här är ett exempel på hur detta kan implementeras Pentest rapport...d SQL).pdf : // Före: osäkert String sql = "SELECT * FROM items WHERE id = " + id_variable; // sammanfogning stmt.executeQuery(sql); // Efter: förberedda satser PreparedStatement ps = conn.prepareStatement("SELECT * FROM items WHERE id = ?"); ps.setInt(1, id_variable); ps.executeQuery(); Genom att använda detta mönster kan SQL-injection undvikas och säkerheten påverkas positivt.

Login Bypass via Error-based SQL Injection (SQLi)	Qwen 2.5-coder-7b	Pentest rapport SchutzPunkt.pdf	Det märkta felet är att applikationen inte korrigerar användarinmatningen på rätt sätt. Detta kan leda till SQL-injektion.	I kodutdraget ser vi att username och password hämtas direkt från användaren utan någon form av input-filtrering eller validering. Denna information används sedan i en SQL-fråga, vilket ger möjlighet för en attackerare att införa malicious SQL-kod.	Korrigeringsförslag skulle inkludera att använder parametriserade SQL-frågor eller ORM (Object Relational Mapping) som hjälper till att förhindra SQL-injektioner. Här är ett exempel på hur det kan göras: <code>query = "SELECT * FROM users WHERE username = ? AND password = ?"</code> <code>cursor.execute(query, (username, password))</code>
Login Bypass via Error-based SQL Injection (SQLi)	Qwen 3.5 - 9B	Pentest rapport SchutzPunkt.pdf	Sårbarheten orsakas av att användarstyrda ingångsvärden (username och password) direkt infogas i SQL-queries utan att använda parametrar (prepared statements). Detta kombinerats med att databasfelmeddelanden exponeras till användaren, vilket möjliggör "error-based SQL injection" enligt rapporten	Enligt penetrationstestrapporten exponerar applikationen databasfelinformation när den bearbetar otestad input, vilket möjliggör error-based SQL injection Pentest rapport...d SQL).pdf . När en användare skickar input som 'or 1 = 1; --' till login-formuläret, infogas detta direkt i SQL-queries utan parametrar. Databasen tolkar inputen som SQL-kod istället för som ett strängvärde. Felmeddelanden från databasen läcks ut vilket ger attacker möjlighet att analysera och utvinna information om databasscheman Pentest rapport...d SQL).pdf . Den specifika sårbara endpointen är <code>https://juice-shop.lab/rest/user/login</code> som kan användas för att logga in som administratör utan giltiga inloggningar Pentest rapport...d SQL).pdf	Den säkraste lösningen är att använda parametrar (prepared statements) för att hantera användarinput. Här är en föreslagen säker fix: Säker kod med parametrar: // SÄKER - Använder parametrar <code>const query = 'SELECT * FROM Users WHERE email = ? AND password = ? AND deletedAt IS NULL';</code> <code>const user = req.body.email;</code> <code>const pass = req.body.password;</code> <code>const users = await db.query(query, [user, pass]);</code> Ej säker (nuvarande kod): // Osäker - Direkt concatenation <code>const query = `SELECT * FROM Users WHERE email = '\${user}' AND password = '\${pass}' AND deletedAt IS NULL`;</code> <code>const users = await db.query(query);</code> Extra säkerhetsåtgärder: Avaktivera exponering av felmeddelanden till användaren - returnera generiska fel istället för databasfel Implementera input-validering före query-exekverering

					Överväg att använda ORM-bibliotek som automatiskt använder parametrar Implementera Content Security Policy (CSP) för att begränsa skadlig output Enligt OWASP SQL Injection Prevention Cheat Sheet bör alla användarinput hanteras med parametrar eller whitelist
Login Bypass via Error-based SQL Injection (SQLi)	Minis tral 3 14B Reasoning	Pentest rapport SchutzPunkt.pdf	Sårbarheten beror på bristande inputsanitizing och användning av direkt SQL-konkatenering i applikationens logik för inloggning. Applikationen exponerar dessutom databasfelmeddelanden till användaren, vilket möjliggör error-based SQL-injektion. Detta sker genom att ohanterad användarinput direkt inkluderas i SQL-frågor utan parameterisering eller validering.	Ohanterad input i SQL-frågor: När en användare matar in ' or 1 = 1; -- - som användarnamn, stänger citattecknet (') det ursprungliga fältet (email), och den efterföljande SQL-koden (or 1 = 1) gör att villkoret alltid blir sant. Punkten och bindestrecket (; -- -) kommenterar bort resten av den ursprungliga SQL-frågan, vilket gör att inloggningen går igenom utan att lösenordet kontrolleras Pentest rapport...zPunkt.pdf Exempel på den manipulerade SQL-frågan: SELECT * FROM Users WHERE email = " or 1 = 1; -- -' AND password = '...' Detta resulterar i att angriparen kan logga in som administratör utan giltigt lösenord.	För att åtgärda sårbarheten bör följande åtgärder vidtas: 1. Använd preparerade anrop (parameterized queries) Ersätt direkt SQL-konkatenering med preparerade anrop. Här är ett exempel på hur det kan se ut i Node.js med mysql2-paketet (eller motsvarande för andra språk/databaser): Otrygg kod (vulnerabel): // Vulnerabel kod (direkt konkatenering) const email = req.body.email; // Ohanterad input! const password = req.body.password; const query = `SELECT * FROM Users WHERE email = '\${email}' AND password = '\${password}' AND deletedAt IS NULL`; connection.query(query, (err, results) => { ... }); Säker kod (med preparerade anrop): // Säker kod (parameterized query) const email = req.body.email; const password = req.body.password; const query = `SELECT * FROM Users WHERE email = ? AND password = ? AND deletedAt IS NULL`; connection.execute(query, [email, password], (err, results) => { ...

				Exponering av databasfel: När SQL-frågan misslyckas (t.ex. på grund av felaktig syntax), returnerar databasen detaljerade felmeddelanden till användaren. Dessa meddelanden kan innehålla information om tabeller, kolumner, funktioner och databasschema, vilket underlättar för angriparen att utforska och manipulera databasen ytterligare Pentest rapport...zPunkt.pdf Saknad parameterisering: Koden använder troligen direkt strängkonkatenering för att bygga SQL-frågor istället för preparerade anrop (parameterized queries) eller ORM (Object-Relational Mapping). Detta gör att ohanterad input kan injiceras och exekveras som maliciös SQL-kod.	<pre>});</pre> Varför detta fungerar: Databasen behandlar ? som en placeholder för värden som skickas separat. Input kan inte manipulera SQL-syntaxen eftersom den separeras från frågan. 2. Stäng av detaljerade databasfelmeddelanden Konfigurera databasen och webbservern för att inte exponera felmeddelanden till användaren. Exempel för MySQL och Node.js: Konfigurera MySQL: <pre>-- I my.cnf eller via kommando [mysqld] log_errors = /var/log/mysql/error.log log_error_verbosity = 2 skip-show-database</pre> Använd även try-catch i applikationen för att hantera fel på ett säkert sätt: <pre>connection.query(query, [email, password], (err, results) => { if (err) { console.error("Database error:", err); // Logga fel internt res.status(500).send("An error occurred. Please try again."); // Generiskt svar till användaren } else { res.json(results); } });</pre> Använd en felhanteringsmiddleware (Express.js): <pre>app.use((err, req, res, next) => { console.error(err.stack); res.status(500).send("Something broke!"); // Undvik att läcka detaljer });</pre> 3. Validera och sanitize input
--	--	--	--	---	--

					Lägg till validering av användarinput innan det når databasen: const { body, validationResult } = require('express-validator'); <pre> app.post('/login', body('email').trim().escape().isEmail().normalizeEmail(), body('password').trim().escape()), (req, res) => { const errors = validationResult(req); if (!errors.isEmpty()) { return res.status(400).json({ errors: errors.array() }); } // Fortsätt med säker databashantering... }); </pre> 4. Använd ORM eller säker bibliotek Om möjligt, byt till en ORM (t.ex. Sequelize, TypeORM) eller säkra bibliotek som automatiskt hanterar parameterisering: // Exempel med Sequelize const user = await User.findOne({ where: { email: req.body.email, password: req.body.password, deletedAt: null } }); Varför detta löser problemet: Ingen SQL-injektion: Input kan inte manipulera SQL-frågan. Inga läckta felmeddelanden: Användaren ser inte databasschema eller känsliga detaljer. Robusthet: Validering och sanitizing minskar risken för andra typer av attacker.
--	--	--	--	--	---

M1: Cross-Site Scripting (XSS) - Reflected	Qwen 2.5-coder-14b	Pentest rapport SchutzPunkt.pdf	Den mest sannolika grundorsaken till sårbarheten är att användarangivna data återges i ett webbsvar utan lämplig kontextbärande utdatakodning. Denna brist på korrekt kodning gör det möjligt för en attacker att skicka särskilda inmatningsdata som kan manipulera webbsidan, vilket i sin tur kan leda till att angivna skript körs i användarens webbläsare. Pentest rapport...ected).pdf . Detta orsakar att angivna skript kan köras i användarens webbläsare.	Sårbarheten uppstår eftersom applikationen återger användarangivna data i ett webbsvar utan att tillämpa lämplig kontextbärande utdatakodning. Denna brist på korrekt kodning gör det möjligt för en attacker att skicka särskilda inmatningsdata som kan manipulera webbsidan, vilket i sin tur kan leda till att angivna skript körs i användarens webbläsare. Pentest rapport...ected).pdf . Detta problem är begränsat till flytande svar och kräver användarinteraktion för att utnyttjas, men konsekvenserna kan inkludera sessionsoverfördelning, dataexponering och obehöriga åtgärder som utförs via användarens session. Pentest rapport...ected).pdf .	För att lösa denna sårbarhet rekommenderar vi att du använder stränga kontextbärande utdatakodningar för alla återflektade data och implementerar en restriktiv Content Security Policy (CSP) om möjligt. Pentest rapport...ected).pdf . Ett exempel på hur detta kan genomföras i kod: <pre>// Kontextbärande utdatakodning för HTML-text function encodeForHTML(text) { return text .replace(/&/g, '&amp;') .replace(/</g, '&lt;') .replace(/>/g, '&gt;') .replace(/"/g, '&quot;') .replace(/'/g, '&#039;'); } // Använd funktionen för att koda användarangivna data const userInput = "<script>alert('XSS');</script>"; const safeOutput = encodeForHTML(userInput); console.log(safeOutput); // &lt;script&gt;alert(&#039;XSS&#039;);&lt;/script&gt; // Implementera en restriktiv Content Security Policy app.use((req, res, next) => { res.setHeader('Content-Security-Policy', "default-src 'self'; script-src 'none';"); next(); });</pre> Denna kod implementerar två viktiga åtgärder: Kontextbärande utdatakodning: Funktionen encodeForHTML kodar användarangivna data för att undvika att dessa data tolkas som HTML eller JavaScript. Restriktiv Content Security Policy (CSP): CSP är inställd för att endast tillåta resurser från samma ursprung och inte tillåta inline skript, vilket ytterligare minskar risken för XSS-angrepp.
---	---------------------------	--	---	--	--

					Genom att använda dessa åtgärder kan du skydda din applikation mot reflekterade XSS-angrepp.
M1: Cross-Site Scripting (XSS) - Reflected	Qwen 2.5-coder-7b	Pentest rapport SchutzPunkt.pdf	Detta anfall är ett reflekterat Cross-Site Scripting (XSS) anfall. XSS uppstår när en webbapplikation tar emot data från en användare och omedelbart återger det till klientens webbläsare utan att verifiera eller koda det på rätt sätt.	I OWASP Juice Shop finns ett reflekterat XSS anfall på den sökfunktionaliteten. Ett attackerande länk kan skapas som innehåller JavaScript-kod, och när en användare klickar på detta länk kommer JavaScript-koden att exekveras i användarens webbläsare. Denna sårbarhet uppstår eftersom den inmatade söktermen inte har genomgången någon form av kodering eller validering före den återges till klientens webbläsare.	En möjlig lösning på detta anfall är att använda en template-engine som automatiskt kodar all utdata, istället för att manuellt koda varje individuell parameter. Ett exempel kan vara att använda JSR-305 (Bean Validation) eller OWASP Encoder. Ett konkret exempel på hur detta kan göras i Java/Kotlin med OWASP Encoder skulle kunna se ut så här:// Importera OWASP Encoder-biblioteket <pre>import org.owasp.encoder.Encode;</pre> // Exempel på kodbits som kodar användarinmatning för säkerhet <pre>String userInput = request.getParameter("query"); String safeOutput = Encode.forHtml(userInput);</pre> out.print("<div>Search Results for: " + safeOutput + "</div>"); Denna kod använder Encode.forHtml() för att säkerställa att alla HTML-tecken i användarinmatningen har kodats korrekt innan de återges till klienten. Detta förhindrar villkoren från att skriva JavaScript-kod till webbläsaren. Genom att implementera en sådan kodbarriär kan OWASP Juice

					Shop undvika reflekterade XSS-anfall, eftersom alla utdata automatiskt kommer att vara säkert kodade.
M1: Cross-Site Scripting (XSS) - Reflected	Qwen 3.5 - 9B	Pentest rapport SchutzPunkt.pdf	Den primära orsaken till sårbarheten är att applikationen reflekterar användarinput i responsinnehållet utan att tillämpa korrekt, kontextmedveten utmatning (output encoding). Detta gör att kod som ingår i inmatningen inte avfärdas som osäker, vilket tillåter skript att körs i offrets webbläsarkontext	Penetrationstestrappporten identifierar specifikt "M1: Cross-Site Scripting (XSS) - Reflected" i OWASP Juice Shop. Den angivna orsaken är att applikationen speglar användarinput i svarsinnehållet utan rätt utmatning Pentest rapport...ected).pdf . Särskilt noterbart är att sådana system ibland försöker skydda sig via "homegrown sanitization" eller reguljära uttryck (Regex), men det visas i rapporten att dessa ofta har brister och inte tillräckligt effektivt motverkar sårbarheten challenges.yml . Utan att tillhandahålla kontextmedveten koding (t.ex. HTML-encoderingsfunktioner) blir XSS-angrepp möjliga.	För att korrigera sårbarheten bör man undvika direkt reflektion av input och istället använda säker kodningsbiblioteksfunktioner eller säkerställa att ramverket utför automatisk encoding (t.ex. i EJS eller Handlebars templates). Nedan är en exempelkod på ett säkert sätt att hantera input i JavaScript/Node.js, samt rekommendationer om Content Security Policy (CSP). <pre>// Säker input hantering (Exempel i Node.js / JavaScript) function escapeHtml(str) { if (str === null typeof str !== 'undefined') return str; return .replace(/&/g, "&amp;") .replace(/</g, "&lt;") .replace(/>/g, "&gt;") .replace(/"/g, "&quot;") .replace(/'/g, "&#x27;"); } // Exempel på användning i en render-funktion const userInput = "inputFromUser"; const safeInput = escapeHtml(userInput); // Säkrar innan output</pre>

					<pre>// Eller använda CSP som en komplementär skydd const contentSecurityPolicy = { contentSecurityPolicy: "default-src 'self'; script-src 'self'; style-src 'self' }; const res = res.render('view', { safeInput, contentSecurityPolicy });</pre> Denna åtgärd säkerställer att osäker kontext inte körs, vilket eliminerar den direkta sårbarheten.
M1: Cross-Site Scripting (XSS) - Reflected	Minis tral 3 14B Reasoning	Pentest rapport SchutzPunkt.pdf	Sårbarheten M1: Cross-Site Scripting (XSS) - Reflected uppstår på grund av bristande kontextmedveten output-kodning av användarstyrt input i webbapplikationen Juice Shop. Applikationen reflekterar användarens input direkt i HTTP-svar utan att sanera eller kodomvandla det, vilket gör det möjligt för attacker att injicera skadlig JavaScript-kod som exekveras i offerets webbläsare	Reflekterad XSS betyder att skadlig kod injiceras via en URL eller parameter och reflekteras tillbaka i webbsvar till användaren. I Juice Shop är sårbarheten kopplad till sökfunktionen (t.ex. <a href="https://juice-shop.lab/#/search?q=<skadlig%20kod>">https://juice-shop.lab/#/search?q=<skadlig kod>), där användarens input (q-parametern) inte saneras eller kodomvandlas innan det skickas tillbaka i HTML-svaret Pentest rapport...zPunkt.pdf. Kodmässigt (baserat på kontexten) indikerar rapporten och källkodshinvisningar att: Applikationen använder egenimplementerad sanitering baserad på RegEx (regex), vilket ofta är otillräckligt för att blockera alla XSS-angrepp challenges.yml +1	För att åtgärda sårbarheten bör följande åtgärder vidtas: 1. Använd kontextmedveten output-kodning Kodomvandla all användarstyrt input innan det skickas tillbaka till klienten. För Juice Shop (som är skriven i JavaScript/TypeScript) kan detta göras med bibliotek som DOMPurify eller manuell HTML-entitetskodning. Exempel på fix för sökfunktionen (pseudokod): <pre>// I Juice Shop's search-komponent (t.ex. i /src/components/search/search.component.ts) import DOMPurify from 'dompurify'; function sanitizeInput(input: string): string { return DOMPurify.sanitize(input, { ALLOWED_TAGS: [], // Tillåt inga HTML-taggar ALLOWED_ATTR: [], }); } function handleSearch(query: string) {</pre>

				Content Security Policy (CSP) kan vara bristfällig eller felkonfigurerad, vilket minskar skyddet mot XSS challenges.yml +1 Ingen kontextmedveten kodning (t.ex. HTML-entitetskodning för <, >, &) appliceras på användarinput innan det renderas i klienten Pentest rapport...zPunkt.pdf Exempel på hur angriparen kan utnyttja detta: <code>https://juice-shop.lab/#/search?q=<script>alert('XSS')</script></code> Om applikationen reflekterar detta i svaret utan kodning, kommer skriptet att exekveras i offerets webbläsare.	<pre>const sanitizedQuery = sanitizeInput(query); // Använd sanitizedQuery i API-anrop eller rendering this.searchResults = await this.searchService.search(sanitizedQuery); }</pre> 2. Stärka Content Security Policy (CSP) Lägg till eller uppdatera CSP-headern för att begränsa källor och blockera inline-skript. Exempel på en strikt CSP: Content-Security-Policy: <pre>default-src 'self'; script-src 'self' https://trusted.cdn.com; style-src 'self' https://trusted.cdn.com; img-src 'self' data:; object-src 'none'; frame-src 'none'; base-uri 'self'; form-action 'self';</pre> 3. Undvik egenimplementerad sanitering med RegEx RegEx är inte tillräckligt säkert för att skydda mot XSS. Använd istället beprövade bibliotek som: DOMPurify (rekommenderas för JavaScript). OWASP ESAPI (för serversidiga språk som Java). Built-in-funktioner i ramverk som React (automatisk escapning i JSX). 4. Validera och begränsa input Begränsa vad som accepteras i inputfält (t.ex. bara bokstäver och siffror i sökfält) och använd whitelist-validering istället för blacklist.
--	--	--	--	---	---

M3: Directory Listing Enabled	Qwen 2.5-coder-14b	Pentest rapport SchutzPunkt.pdf	Den mest sannolika grundorsaken till sårbarheten är att directory browsing är aktiverat i webbservern. I pentestrapporten beskrivs att webbservern på OWASP Juice Shop är konfigurerad för att tillåta katalogvisning på en eller flera offentliga sökvägar. Detta exponerar namnen och strukturen på filer och underkataloger för alla besökare, även utan autentisering. Källkoden i web.config som ingår i rapporten visar att directory browsing är aktiverat: <pre><configuration> <system.webServer> <directoryBrowse enabled="true" /> </system.webServer> </configuration></pre> Sårbarheten uppstår eftersom directory browsing kan exponera känsliga filer, konfigurationsrestmaterial, säkerhetskopior och lagrade autentiseringsuppgifter. Detta utvidgar informationsexponeringen och gör rekonstruktion och målade angrepp lättare.	En möjlig lösning är att inaktivera directory browsing i web.config genom att ange enabled="false". Här är det förslagande koden: <pre><configuration> <system.webServer> <directoryBrowse enabled="false" /> </system.webServer> </configuration></pre> Med denna ändring kommer webbservern inte att visa filer i katalogerna och minska risken för informationsexponering. Om det är nödvändigt att ha katalogvisning aktiverad, bör man begränsa detta endast till autentiserade användare och undvika att exponera känsliga filer på webbservern.
----------------------------------	--------------------	---------------------------------	--	--

M3: Directory Listing Enabled	Qwen 2.5-coder-7b	Pentest rapport SchutzPunkt.pdf	Directory listing är aktiverad på publika sökvägar, vilket exponerar namn och struktur för filer och underkataloger för alla besökare.	Directory listing är aktiverad i OWASP Juice Shop eftersom den inte är korrekt konfigurerad. Servern är inställd att visa kataloginnehåll när en användare navigerar till en katalog utan en index.html-fil. Detta gör det möjligt för angripare att se vilka filer som finns tillgängliga på servern och potentiellt lokalisera känslig information som säkerhetskopior, konfigurationsfiler och annan konfidentiell data.	För att åtgärda detta problem behöver du inaktivera kataloglistning för alla publika sökvägar i webbserverns konfiguration. Om du använder Apache HTTP Server kan du uppnå detta genom att lägga till eller modifiera följande rader i din httpd.conf- eller .htaccess-fil: <pre><Directory "/var/www/html/"> Options -Indexes </Directory></pre> För Nginx, lägg till följande rad i din serverblocks-konfiguration: <pre>location / { autoindex off; }</pre> Alternativt kan du framtvunga kataloglistning via en web.config-fil om du använder Microsoft IIS. Här är ett exempel på hur man gör det på ett säkert sätt: <pre><configuration> <system.webServer> <directoryBrowse enabled="false" /> </system.webServer> </configuration></pre> Denna konfiguration säkerställer att katalogbläddring är inaktiverad, vilket förbättrar säkerheten i din applikation genom att förhindra att angripare kan se känsliga filstrukturer och innehåll.
----------------------------------	-------------------	---------------------------------	--	---	--

M3: Directory Listing Enabled	Qwen 3.5 - 9B	Pentest rapport SchutzPunk t.pdf	Webbservarens konfiguration tillåter oautentiserad katalogvisning (Directory Listing) på en eller flera offentliga sökvägar. Detta är en säkerhetskonnfigurationssårbarhet som tillåter att filnamn och struktur blir exponerade utan att kräva inloggning	Sårbarheten uppstår eftersom webbservaren (Apache eller motsvarande) är konfigurerad med inställningar som tillåter obehörig katalogbrowsing. Enligt penetrationstestrappporten exponeras detta via CVSS-värden för AV:N/AC:L/PR:N/UI:N Pentest rapport...abled).pdf . När katalogvisning är aktiverat kan besökare se: Sensitive filer Konfigurationsrester Backupper Autentiseringsuppgifter Information läckan underlättar ytterligare attacker såsom brute-force-attacker mot upptäckta tillgångar och avslöjar överlookade administratörspunkter	För att åtgärda sårbarheten behöver du konfigurera webbservaren så att katalogvisning stängs av: För Apache: # Lägg till i httpd.conf eller .htaccess Options -Indexes För IIS (web.config): <pre><configuration> <system.webServer> <directoryBrowse enabled="false" /> </system.webServer> </configuration></pre> Ytterligare rekommendationer: Säkerställ att inga känsliga filer befinner sig i webbtillgängliga mappar Pentest rapport...abled).pdf Implementera en standard-indexsida för att förhindra autogenererade listningar Om katalogvisning behövs för legitima användningsfall, begränsa den till autentiserade användare och isolera från offentliga routes
--	------------------	---	---	--	---

M3: Directory Listing Enabled	Ministry of Justice 314B Reasoning	Pentest report SchutzPunkt.pdf	Sårbarheten beror på att webbservern är konfigurerad med aktiverad directory listing (katalogvisning) på en eller flera publika sökvägar. Detta tillåter obehöriga att se filnamn, struktur och innehåll i en katalog utan att behöva autentisera sig. Sårbarheten är klassificerad under CWE-548 (Exposure of System File or Directory Listing) och CWE-200 (Information Exposure)	I rapporten framgår att sårbarheten uppstår när en webbserver (t.ex. Apache, Nginx eller IIS) är konfigurerad för att visa innehållet i en katalog automatiskt när en användare navigerar till en sökväg som saknar en standardindexfil (t.ex. index.html). Detta exponerar: Filnamn och struktur, vilket underlättar reconnaissance (förberedande spaning). Känsliga filer som konfigurationer, bakupper eller temporära filer. Möjligheten att ladda ner eller manipulera innehåll utan behörighet. I OWASP Juice Shop är detta en intentionell sårbarhet för utbildningssyfte, men i en produktionsmiljö skulle det utgöra en allvarlig säkerhetsrisk. Rapporten anger att sårbarheten finns på sökvägen https://juice-shop.lab/ftp	1. För IIS (Windows-server) I rapporten ges ett exempel på hur man stänger av directory listing i web.config: <pre><configuration> <system.webServer> <directoryBrowse enabled="false" /> </system.webServer> </configuration></pre> Varför? Detta avaktiverar automatisk katalogvisning på servernivå. 2. För Apache (Linux/Unix) I rapporten hänvisas till mod_autoindex. För att stänga av directory listing lägg till följande i Apache-konfigurationen (t.ex. /etc/apache2/sites-available/000-default.conf eller .htaccess): <pre><Directory "/var/www/html/ftp"> Options -Indexes # Avaktiverar indexering (katalogvisning) AllowOverride None Require all denied # Alternativt begränsa åtkomst till specifika IP:er </Directory></pre> Varför? Direktivet Options -Indexes förhindrar att servern genererar en automatisk lista över filer i katalogen. 3. För Nginx I rapporten nämns ngx_http_autoindex_module. För att stänga av directory listing redigera Nginx-konfigurationen (t.ex. /etc/nginx/sites-available/default): <pre>server { location /ftp { autoindex off; # Avaktiverar katalogvisning # Alternativt: return 403; # Blockera åtkomst helt } }</pre> Varför? Direktivet autoindex off förhindrar att Nginx visar innehållet i katalogen. 4. Generella rekommendationer (för alla webbserverar) Använd en standardindexfil: Se till att varje katalog har en index.html eller motsvarande som visas automatiskt.
-------------------------------	------------------------------------	--------------------------------	--	--	--

					Exempel: DirectoryIndex index.html index.htm Begränsa åtkomst: Om directory listing måste vara aktiv, begränsa åtkomsten till autentiserade användare <pre><Directory "/secure/ftp"> Options +Indexes Allow from env=REMOTE_USER # Endast autentiserade användare </Directory></pre> Ta bort känsliga filer: Granska katalogen och ta bort temporära filer, bakupper eller konfigurationer som inte behövs vara publika. OWASP Juice Shop är en öppet säkerhetsmedveten applikation, och sårbarheten är avsiktlig för utbildning. I en riktig produktionsmiljö bör directory listing alltid vara avstängd på publika sökvägar
Applicat ion is vulnerab le to SQL Injectio n	Qwen 2.5- coder- 14b	Juice Shop - Web Application Security Assessment Report	Den mest sannolika grundorsaken till sårbarheten [SQL Injection] i OWASP Juice Shop är att applikationen konstruerar SQL-kommandon från användarinmatning utan att neutralisera specialtecken som kan ändra den avsedda SQL-kommandot. Detta leder till möjlighet för en angreppare att manipulera dessa kommandon för att exekvera onödiga eller skadliga frågor mot databasen.	Enligt rapporten Juice-Shop-Web-...ction).pdf, konstruerar applikationen SQL-kommandon med hjälp av användarinmatning utan att korrekt neutralisera specialtecken. Detta gör det möjligt för en angreppare att infoga onödiga eller skadliga frågor via SQL-injection, vilket kan leda till obehörig åtkomst till känsliga data som lösenord eller personuppgifter. Källkoden visar att applikationen använder användarinmatning för att skapa SQL-frågor utan att använda parameteriserade frågor, vilket är en vanlig metod för att förhindra SQL-injection. Utan den här skyddsmekanismen kan ett angreppare skicka specifika indata som manipulerar SQL-kommandot	En möjlig lösning är att använda parameteriserade frågor (prepared statements) för att hantera användarinmatning i SQL-frågorna. Detta ser uttryckligen ut som den bästa praxisen för att förhindra SQL-injection. Här är ett exempel på hur detta kan implementeras med hjälp av parameteriserade frågor: <pre>python import sqlite3 def fetch_user_by_email(email): # Ansluta till databasen conn = sqlite3.connect('database.db') cursor = conn.cursor() # Använda en parameteriserad fråga för att skydda mot SQL-injection query = "SELECT * FROM Users WHERE email = ?" cursor.execute(query, (email,))</pre>

				och leder till onödiga eller skadliga frågor. <pre> # Hämta resultatet result = cursor.fetchone() # Stäng anslutningen till databasen conn.close() return result # Exempel på att använda funktionen med ett obehörigt inmatningsvärde user = fetch_user_by_email("admin' --") print(user) I det här exemplet använder vi en parameteriserad fråga för att hämta användaruppgifter baserat på e-postadress. Med hjälp av parameteriserade frågor, där specialtecken i användarinmatningen behandlas som vanlig data och inte som SQL-kod, undviker vi möjligheten för SQL-injection. Denna metod rekommenderas för att förhindra sårbarhet mot SQL-injection i applikationer. Detta ligger i linje med den rekommendation från rapporten Juice- Shop-Web-...ction).pdf, vilket påpekar att de flesta instanser av SQL-injection kan förhindras genom användning av parameteriserade frågor istället för strängkonkatenation. </pre>
--	--	--	--	--

Applicat ion is vulnerab le to SQL Injectio n	Qwen 2.5- coder- 7b	Juice Shop - Web Application Security Assessment Report	SQL-injektion uppstår när användarinmatning inte korrekt hanteras av applikationen, vilket gör att angripare kan införa SQL-kod i inputfält. Detta resulterar i inkorrekta frågor till databasen och potentiellt tillgång till känsliga data.	I OWASP Juice Shop är det sårbara om man försöker skicka SQL-instruktioner som inte är korrekt inmatade. Detta kan göras genom att infoga specialtecken eller strängar i inputfält, vilket resulterar i inkorrekta SQL-frågor till databasen Juice-Shop-Web-...ction).pdf . I rapporten beskrivs detta som en "web security vulnerability that allows an attacker to interfere with the queries that an application makes to its database"	
Applicat ion is vulnerab le to SQL Injectio n	Qwen 3.5 - 9B	Juice Shop - Web Application Security Assessment Report	Sårbarheten uppstår då applikationen konstruerar SQL-ansökan med externt influerad inmatning från användaren utan att neutralisera specialkaraktärer som kan modifiera den ursprungliga SQL-kommandon. Detta sker genom att användarinmatning direkt klibbas in i SQL-queries med string-konkatering istället för parametriserade queries	Enligt penetrationstrappporten är det JSON-email-parameter som är sårbar för SQL Injection Juice-Shop-Web-...ction).pdf . När en användare skickar in data (t.ex. i en search eller query-funktion) kombineras denna data direkt med SQL-kommandon utan att användarinmatning renvas eller valideras. Detta gör att en attacker kan manipulera databasfrågan genom att skicka in input som innehåller SQL-syntax, vilket möjliggör dataexfiltration. Rapporten rekommenderar användning av parametriserade queries (prepared statements) istället för string-konkatering	För att åtgärda sårbarheten bör ni implementera parametriserade queries. Här är ett exempel på hur ni kan förbättra koden: // INKORREKT (Sårbar) - Undvik detta: let query = `SELECT * FROM users WHERE email = '\${email}' AND password = '\${password}'`; // RÄTT (Säkert) - Använd parametriserade queries: const query = `SELECT * FROM users WHERE email = ? AND password = ?`; const result = await db.query(query, [email, password]); Om ni använder en ORM (t.ex. Sequelize eller TypeORM), se till att alltid använda .where() eller .findOne() med input direkt utan string-konkatering. Detta garanterar att användarinmatning behandlas som data, inte som SQL-kod. Referenser för ytterligare information: OWASP SQL Injection Prevention Cheat Sheet Juice-Shop-Web-...ction).pdf PortSwigger Web Security SQL Injection Juice-Shop-Web-...ction).pdf

Application is vulnerable to SQL Injection	Minimal 3 Reasoning	Juice Shop - Web Application Security Assessment Report	Sårbarheten beror på direkt string-konkatenering av användarstyrd input i SQL-frågor utan användning av parameteriserade frågor (prepared statements). Detta tillåter attacker att manipulera SQL-syntaxen genom att injicera ogiltiga SQL-kommandon, såsom OR 1=1, vilket bryter mot applikationens avsiktliga logik Rapporten beskriver att applikationen konstruerar SQL-frågor genom att direkt kombinera användarinput med statisk SQL-kod (t.ex. i formulärfält som e-postadress eller inloggningsuppgifter). När en attacker lägger till SQL-syntax (t.ex. admin' --) i ett fält som e-post, tolkas detta som en del av SQL-frågan istället för ren data. Exempelvis: -- Sårbar kod (hypotetisk, ej bifogad i kontexten): SELECT * FROM users WHERE email = '[user_input]'; Om användaren matar in admin' OR '1'=1, blir SQL-frågan: SELECT * FROM users WHERE email = 'admin' OR '1'=1'; Detta returnerar alla användare, inklusive admin, utan autentisering Juice-Shop-Web-...ction).pdf . Rapporten bekräftar att detta leder till olaglig åtkomst till känsliga data (t.ex. lösenord, kreditkort) och har en CVSS-score på 8.9	Använd parameteriserade frågor för att separera data från SQL-logik. Exempel på säker kod (i Node.js med mysql2 eller liknande bibliotek): <pre>// OSÄKER: String-konkatenering (ej rekommenderad) const userInput = req.body.email; // Användarstyrd input const query = `SELECT * FROM users WHERE email = '\${userInput}'`; // Sårbar!</pre> <pre>// SÄKER: Parameteriserad fråga (rekommenderad) const query = 'SELECT * FROM users WHERE email = ?'; const [rows] = await connection.execute(query, [req.body.email]);</pre> Varför detta fungerar: ? är en placeholder som ersätts med användarinput som data, inte SQL-kod. Databasen behandlar input som en sträng, inte som exekverbara kommandon. Applikerar på alla platser där användarinput används i SQL-frågor (t.ex. WHERE, INSERT, UPDATE). Ytterligare åtgärder: Använd ORM-bibliotek (t.ex. Sequelize, TypeORM) som automatiskt hanterar parameterisering. Input-sanitizing: Validera och begränsa input (t.ex. e-postformat) innan det når SQL. Använd WAF (Web Application Firewall): Konfigurera ModSecurity CRS för att blockera SQL-injektion
--	---------------------	---	---	--

DOM-Based Cross Site Scripting (XSS)	Qwen 2.5-coder-14b	Juice Shop - Web Application Security Assessment Report	Den mest sannolika grundorsaken till sårbarheten är att JavaScript i applikationen tar emot data från en attacker-controllable source (t.ex. URL) och passar den vidare till en sink som stöder dynamisk kodsprovning.	Sårbarheten uppstår eftersom Juice Shop-webbapplikationen saknar lämplig kod för att korrekt skapa och hantera DOM-element. Detta leder till att användarindata kan mängdvis bli oövervakade eller ej korrekt encoderade, vilket gör det möjligt för attacker att infoga krossningsplatsskript (XSS). I vissa fall använder applikationen en egen implementering av regelextrahering (Regex) som inte täcker alla scenarioer och kan därför vara obehindrad. Utifrån informationen i rapporten och källkoden challenges.yml, indikerar det att användning av ett dåligt utformat Regex för sanitering är en del av problemet, vilket gör det lättare för attacker att skapa XSS-payloads.	En möjlig lösning är att implementera robusta och kontextspecifika kodningsmekanismer för alla dynamiska data som visas på webbsidan. Detta inkluderar att använda plattformens inbyggda kodningsfunktioner eller kända säkerhetsbibliotek för att säkerställa att inget oönskat skript körs i klientsidan. Ett exempel på hur detta kan implementeras med hjälp av JavaScript-kod är: <pre>// Funktionsdefinition för att kodera HTML-innehåll function escapeHtml(unsafe) { return .replace(/&/g, "&amp;") .replace(/</g, "&lt;") .replace(/>/g, "&gt;") .replace(/"/g, "&quot;") .replace(/'/g, "&#039;"); } // Exempel på hur man använder funktionen för att göra data säkra const userInput = "<script>alert('xss')</script>"; const safeOutput = escapeHtml(userInput); // Infoga det säkra innehållet i DOM-elementet document.getElementById("output").innerHTML = safeOutput;</pre> Genom att använda funktionen escapeHtml för att kodera alla användarindata som ska visas på webbsidan, minskas risken för DOM-baserad XSS. Dessutom rekommenderas implementering av en Content Security Policy (CSP) med explicita vittlistor för applikationens resurser, vilket ger ytterligare skydd mot skriptinbäddning och andra sårbara punkter.
--------------------------------------	--------------------	---	---	--	---

DOM-Based Cross Site Scripting (XSS)	Qwen 2.5-coder-7b	Juice Shop - Web Application Security Assessment Report	En DOM-Based Cross-Site Scripting (XSS) sårbarhet uppstår när JavaScript-kod tar emot data från en användare kontrollerbar källa, som URL:en, och sedan passar det till ett mottagande punkt som stödjer dynamisk kodexekvering.	I OWASP Juice Shop kan XSS-sårbarheten uppstå om JavaScript-koden tar emot data från en URL-parameter och sedan använder det för att uppdatera en del av webbsidan, till exempel genom att ange innehållet i ett HTML-element. Om dessa parametrar inte rätt är koda eller hanterade på ett säkert sätt, kan attacker införa skadliga skript som sedan körs i användarens browser.	För att förhindra XSS-sårbarheten bör all innehåll från URL-parametrar eller andra användarinmatningar koda rätt innan det matas till en mottagande punkt. I JavaScript kan detta göras med hjälp av funktioner som encodeURIComponent() för HTML-kontext eller encodeURI() för URI-kontext. Ett exempel på hur man kan säkerställa att användarinmatningen koda rätt innan den matas till en mottagande punkt skulle kunna se ut så här: <pre>// Exempel på URL-parameterhantering i OWASP Juice Shop const userInput = new URLSearchParams(window.location.search).get('paramName'); const safeInput = encodeURIComponent(userInput); // Koda innehållet rätt</pre> // Använd det koda innehållet i en mottagande punkt document.getElementById('targetElement').innerHTML = `<div>\${safeInput}</div>`; Genom att använda funktioner som encodeURIComponent() eller liknande kan innehållet från URL-parametrar eller andra användarinmatningar koda rätt innan det matas till en mottagande punkt, vilket förhindrar möjliga XSS-sårbarheter.
--------------------------------------	-------------------	---	---	---	--

DOM-Based Cross Site Scripting (XSS)	Qwen 3.5 - 9B	Juice Shop - Web Application Security Assessment Report	DOM-baserad Cross Site Scripting (XSS) uppstår när JavaScript-koden tar data från en angriparens kontrollerbara källa, såsom URL-parametrar, och vidarebefordrar denna data till en "sink" (t.ex. document.write, innerHTML, eller eventhandlers) som stödjer dynamisk kodkörning utan att först koda om datan Enligt penetrationstrappporten från Juice Shop Security Assessment Juice-Shop-Web...(XSS)).pdf kan en attacker utnyttja denna sårbarhet genom att: Infoga skadlig kod (XSS payload) i sårbara parametrar Öppna sidan där XSS är reflekterad JavaScript tolkar datan som kod istället för text, vilket leder till: Account-compromise Session hijacking Data-exfiltration (t.ex. användarnamn, lösenord, kreditkortsinformation) Rapporten specificerar att DOM-baserad XSS "usually arises when JavaScript takes data from an attacker-controllable source, such as the URL, and passes it to a sink that supports dynamic code execution" Juice-Shop-Web...(XSS)).pdf . Koden använder oftast data direkt utan tillräcklig kodning eller validering, vilket gör den sårbar för utnyttjande.	<pre>// FÖRST - Felaktig kod (sårbar): let userInput = urlParams.get("q"); document.getElementById("output").innerHTML = userInput; // SÅRBAR! // ÅTGÄRD - Korrekt kodning (Context-Aware): // 1. Använd context-sensitiv kodning function encodeDataForOutput(context) { if (context === "html") { // HTML-kodning för HTML-innehåll return encodeURIComponent(userInput).replace(/"/g, "&quot;"); } else if (context === "attribute") { // HTML-attribute kodning return encodeURIComponent(userInput).replace(/"/g, "&quot;"); } else if (context === "javascript") { // JavaScript-kodning return encodeURIComponent(userInput) .replace(/"/g, "\\") .replace(/'/g, "\\'") .replace(/\n/g, "\\n"); } } // 2. Implementera Content Security Policy (CSP) // Lägg till i HTTP-responsen const cspHeader = "default-src 'self'; script-src 'self';"; document.write(cspHeader); // 3. Validera all oberoende data innan användning function validateAndSanitize(input) { // Validera formatet först</pre>
--------------------------------------	---------------	---	---	--

					<pre> if (!isValidFormat(input)) { throw new Error("Ogiltig dataformat"); } // Koda om efter validering const sanitized = encodeDataForOutput("html"); return sanitized; } // 4. Använd säkrare DOM-metoder // Istället för innerHTML, använd: document.getElementById("output").textContent = userInput; // Eller: document.getElementById("output").appendChild(document.createTextNode(userInput)); </pre>
DOM-Based Cross Site Scripting (XSS)	Minis-tral 3 14B Reasoning	Juice Shop - Web Application Security Assessment Report	Den mest sannolika grundorsaken till sårbarheten är bristande eller otillräcklig sanitering och encoding av användarstyrt data i JavaScript-kod, där data från attacker-kontrollerade källor (t.ex. URL-parametrar) direkt används i sinks som stöder dynamisk kodexekvering (t.ex. innerHTML, document.write, eller direkt i JavaScript-kontext). Sårbarheten uppstår specifikt när JavaScript tolkar och exekverar obearbetat användarinput utan att först validera, sanitera eller konvertera det till säkra format	Källa till sårbarheten: I Juice Shop är applikationen skriven i TypeScript/JavaScript med Angular, vilket innebär att DOM-baserad XSS kan uppstå när: Användarinput (t.ex. från URL-parametrar som #username, ?search=..., eller location.hash) tas emot och direkt används i JavaScript-kod utan sanitering. Exempelvis om koden använder sig av: document.getElementById("output").innerHTML = userInput; // Farligt! eller eval(userInput); // Extremt farligt! Rapporten beskriver att sårbarheten uppstår när JavaScript tar data från attacker-kontrollerade källor och	För att åtgärda DOM-baserad XSS i Juice Shop (eller liknande applikationer) bör följande åtgärder vidtas: 1. Använd kontext-baserad encoding av användarinput. Ersätt direkt användning av innerHTML eller outerHTML med säkra metoder som: HTML-encoding för text som ska visas i HTML-kontext. HTML-attribute-encoding för data som ska användas i HTML-attribut. JavaScript-encoding för data som ska användas i JavaScript-kontext. Exempel på säker kod (TypeScript/JavaScript): <pre> // Otryggt: Direkt användning av innerHTML document.getElementById("output").innerHTML = userInput; // XSS-risk! // Säkert: Använd textContent eller HTML-encoding document.getElementById("output").textContent = userInput; // Endast text, ingen HTML </pre>

				passar det till en "sink" (t.ex. innerHTML, document.write, eller location-objekt) som exekverar det dynamiskt Hur angriparen utnyttjar sårbarheten: Angriparen injicerar ett XSS-payload (t.ex. javascript:alert('xss') eller) i en parameter (t.ex. URL). När sidan laddas tolkar och exekverar JavaScript-koden som en del av DOM:en, vilket leder till att angriparens skript körs i användarens webbläsare. Exempel från rapporten visar att en lyckad attack resulterar i en alert('xss')-ruta, vilket bekräftar att skriptet exekveras Varför är det farligt? DOM-baserad XSS är lokaliserad till enskilda sidor (inte persisterad i databasen), men kan fortfarande stjäla session-cookies, logga tangenttryckningar, eller omdirigera användare till farliga sidor. Rapporten nämner risker som kontotöppning, DDoS, och XSS-tunneling Juice-Shop-Web-...(XSS)).pdf Specifikt exempel från Juice Shop: En av de kända sårbara funktionerna i Juice Shop är sanitizing baserat på RegEx, vilket ofta är otillräckligt för att blockera alla XSS-angrepp challenges.yml	// Om HTML måste visas, använd DOMPurify (rekommenderat för Angular/JS) <pre>import DOMPurify from 'dompurify'; const clean = DOMPurify.sanitize(userInput); document.getElementById("output").innerHTML = clean; // Saniterat!</pre> Alternativ för Angular (rekommenderat): // I en Angular-komponent, använd [innerHTML] med sanitizing <pre>import { DomSanitizer, SafeHtml } from '@angular/platform-browser'; constructor(private sanitizer: DomSanitizer) {} sanitizeHtml(input: string): SafeHtml { return this.sanitizer.bypassSecurityTrustHtml(DOMPurify.sanitize(input)); }</pre> Använd sedan i HTML: <pre><div [innerHTML]="sanitizeHtml(userInput)"></div></pre> 2. Undvik eval() och dynamisk kodexekvering Om din kod använder eval(), setTimeout(), eller Function() med användarinput, ersätt med säkra alternativ: <pre>// Otryggt: eval(userInput); // Aldrig använda!</pre> // Säkert: <pre>const safeFunction = () => { /* din kod */ }; safeFunction();</pre> 3. Implementera Content Security Policy (CSP) Lägg till en CSP-header i din server (Node.js/Express-exempel):
--	--	--	--	--	---

				Hints i rapporten pekar på att vissa inputfält inte valideras ordentligt eller inte persisteras på ett säkert sätt challenges.yml <pre> app.use((req, res, next) => { res.setHeader("Content-Security-Policy", "default-src 'self';" + "script-src 'self' 'unsafe-inline' https://trusted.cdn.com;" + "style-src 'self' 'unsafe-inline';" + "img-src 'self' data;"); next(); }); </pre> Rekommendationer från rapporten: Använd explicit whitelisting för tillåtna resurser. Undvik 'unsafe-inline' om möjligt (ersätt med nonce-baserade tillstånd) 4. Validera och sanitera input på klientsidan Om du måste använda RegEx för sanitering (som i Juice Shop), använd robusta bibliotek som DOMPurify istället för egna RegEx: <pre> // Otryggt: Egna RegEx (lätt att bypassa) const sanitized = userInput.replace(/<script.*?>.*?<\script>/gi, ""); // Säkert: Använd DOMPurify import DOMPurify from 'dompurify'; const clean = DOMPurify.sanitize(userInput); </pre> 5. Undvik direkt användning av location-objekt med användarinput Om din kod ser ut så här: <pre> // Otryggt: XSS-risk! window.location = userInput; // Säkert: </pre>
--	--	--	--	---

					<pre>const trustedUrl = new URL('/safe-path', window.location.origin); window.location = trustedUrl;</pre> 6. Test och åtgärda specifika fall i Juice Shop Enligt rapporten finns det hints om specifika sårbara funktioner i Juice Shop, t.ex.: Inputfält som inte valideras ordentligt challenges.yml Sanitizing baserat på RegEx (ersätt med DOMPurify) challenges.yml Exempel på åtgärd för Juice Shop: Om du hittar kod som liknar detta: <pre>// I Juice Shop (exempel från källkod) const userInput = window.location.hash.substring(1); // Tar input från URL document.getElementById("output").innerHTML = userInput; // XSS!</pre> Fixa det så här: <pre>const userInput = window.location.hash.substring(1); const cleanInput = DOMPurify.sanitize(userInput); document.getElementById("output").textContent = cleanInput; // Säkert!</pre>
--	--	--	--	--	---

Bilaga 8. Sammanställning av testresultat – LM Studio-metoden

Test	Modell	Rapport	Svar (Grundorsak)	Svar (Förklaring)	Svar (Åtgärd förslag)
------	--------	---------	-------------------	-------------------	-----------------------

Login Bypass via Error-based SQL Injection (SQLi)	Qwen 3.5 - 9B	Pentest rapport SchutzPunkt.pdf	Sårbarheten beror på att användarinput från HTTP-användarns anrop (specifikt e-postadress och lösenord) direkt infogas i en SQL-query-sträng utan parametrisk behandling eller sanitisering under autentiseringsprocessen, vilket möjliggör SQL-injektion.	Enligt penetrationstrappporten identifieras sårbarheten H1 som "Login Bypass via Error-based SQL Injection (SQLi)" med en CVSS-poäng på 8.6 och klassificeras som High-risk ("Immediate action"). Rapporten anger att resultaten är giltiga för Juice-Shop-applikationen under testperioden. Koden visar i login-funktionen att databasinteraktion sker via <code>models.sequelize.query</code>. Här används strängkonkatenering (<code>'\${req.body.email} '</code>) direkt inuti SQL-kommandot istället för parametriska variabler. Detta tillåter en angripare att manipulera queryn (t.ex. genom att infoga <code>' OR '1'='1</code> i fälten) för att omgå autentiseringslogiken och få åtkomst utan korrekt lösenord, eller utnyttja felmeddelanden från databasen ("Error-based") som rapporten nämner för att extrahera data.	Använd parametriska frågor (prepared statements) istället för strängkonkatenering i <code>sequelize.query</code>. I Sequelize bör man använda den inbyggda mekanismen för parametrisk binding genom att definiera variablerna som bindade värden, exempelvis via <code>{ where: { email: req.body.email } }</code> eller explicit med <code>bind: models.sequelize.query(sqlString, { model: UserModel, plain: true, bind: [req.body.email '', security.hash(req.body.password '')] })</code>
---	---------------	---------------------------------	---	---	--

Login Bypass via Error-based SQL Injection (SQLi)	Qwen 2.5-coder-7b	Pentest rapport SchutzPunkt.pdf	Sårbarheten uppstår på grund av att användardefinierade indata inte hanteras korrekt och ansluts till SQL-frågor utan parametrar. Detta gör det möjligt för angripare att införa ogulka SQL-kod, vilket leder till SQL-injektion.	I rapporten beskrivs sårbarheten som "Login Bypass via Error-based SQL Injection". Koden visar att SQL-frågan skapas genom att ansluta användardefinierade indata (email och lösenord) till en sträng utan parametrar. Detta görs i följande kodrad: <pre>models.sequelize.query(`SELECT * FROM Users WHERE email = '\${req.body.email}    '}' AND password = '\${security.hash(req.body.password    ')}' AND deletedAt IS NULL`, { model: UserModel, plain: true })</pre> När en angripare skickar ett specifikt användarnamn (t.ex. ' or 1 = 1; -- -') kan SQL-frågan bli: <pre>SELECT * FROM Users WHERE email = " OR 1 = 1; -- -' AND password = 'e8636ea013e682faf61f56ce1cb1ab5c' AND deletedAt IS NULL</pre> Detta leder till att frågan alltid returnerar alla användare eftersom '1 = 1' alltid är sant, vilket gör det möjligt för angriparen att logga in som en administratör.	För att förhindra SQL-injektion ska strängar som används i SQL-frågor byggas med parametrar snarare än genom string-concatenation. Detta kan göras genom att använda prepared statements och bind variables. Ett exempel på hur detta kan göras: <pre>const ps = models.sequelize.query('SELECT * FROM Users WHERE email = ? AND password = ? AND deletedAt IS NULL', { model: UserModel, plain: true }, [req.body.email "", security.hash(req.body.password "")])</pre> Genom att använda prepared statements och bind variables kan SQL-injektioner förhindras eftersom parametrar separeras från SQL-koden. Detta gör det svårare för angriparen att införa ogulka SQL-kod.
--	--------------------------	--	--	---	--

Login Bypass via Error-based SQL Injection (SQLi)	Ministral 3 14B Reasoning	Pentest rapport SchutzPunkt.pdf	Sårbarheten beror på att användarinmatning direkt integreras i SQL-queries utan tillräcklig sanitering eller parameterisering, vilket gör systemet sårbart för SQL-injektioner.	I koden syns att req.body.email och req.body.password sätts direkt in i en SQL-query med string-interpolation. Detta tillåter angriparen att manipulera SQL-syntaxen, t.ex. genom att lägga till additional clauses som OR 1=1, vilket gör det möjligt att logga in utan giltigt lösenord (login bypass). Rapporten bekräftar att detta är en kritisk sårbarhet med hög risk.	Ersätt den dynamiska SQL-strängen med parameteriserade queries eller använd databasens inbyggda metoder för säker hantering av användarinmatning. Exempelvis: models.sequelize.query('SELECT * FROM Users WHERE email = :email AND password = :password AND deletedAt IS NULL', { replacements: email: req.body.email ", } }
Login Bypass via Error-based SQL Injection (SQLi)	Qwen 2.5-coder-14b	Pentest rapport SchutzPunkt.pdf	Den mest sannolika grundorsaken till sårbarheten är att applikationen använder strängsammanfogning för att skapa SQL-frågor, vilket gör det möjligt för angreppare att infoga felaktigt formaterad SQL-kod som kan utnyttjas för att manipulera frågorna.	Sårbarheten uppstår eftersom applikationen inte använder parameteriserade frågor när den skapar SQL-frågor. I koden hittar vi följande rad: models.sequelize.query(`SELECT * FROM Users WHERE email = '\${req.body.email    ''}' AND password = '\${security.hash(req.body.password    '')}' AND deletedAt IS NULL`, { model: UserModel, plain: true }) Här sammanfogas användarinput direkt i SQL-frågan utan att den först parameteriseras. Detta gör det möjligt för en angreppare att skicka in speciellt formaterad input som kan ändra betydelsen av frågan, vilket leder till SQL-injection.	En möjlig lösning är att använda parameteriserade frågor för att skydda mot SQL-injection. Här är ett exempel på hur koden kan ändras: models.sequelize.query('SELECT * FROM Users WHERE email = :email AND password = :password AND deletedAt IS NULL', { replacements: { email: req.body.email "", password: security.hash(req.body.password "") }, type: models.sequelize.QueryTypes.SELECT, model: UserModel, plain: true }) I detta exempel används :email och :password som platshållare i SQL-frågan, och de ersätts med värdena från req.body.email och security.hash(req.body.password) via replacements. Detta säkerställer att användarinput behandlas korrekt och förhindrar SQL-injection.

M1: Cross-Site Scripting (XSS) - Reflected	Qwen 3.5 - 9B	Pentest rapport SchutzPunkt.pdf	Avsaknad av kontextmedveten output-encoding för användarstyrd data som speglas i serverns svar (Reflected Cross-Site Scripting / XSS). Detta motsvaras till CWE-79 och innebär att oberoende input returneras direkt utan sanitisering.	Enligt penetrationstrappporten uppstår sårbarheten när användardata från källor såsom query-parametrar, sökvägar eller formulärfält speglas tillbaka i en serverrespons utan att kodning utförs för sin specifika renderingskontext (t.ex. HTML, attribut eller URL). Detta tillåter en angripare att injicera skript som körs i offrets webbläsare när de interagerar med sidan. Även om utnyttjandet kräver användarinteraktion (som att klicka på ett manipulerat länk), kan konsekvenserna vara allvarliga, inklusive sessionssnattning och exponering av känslig data.	<pre>// FÖRSTÄLLD SÄKERHETSFORBÄTTRING - // Kontextmedveten output-encoding function encodeHTML(data) { const htmlEntities = { '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#x27;', }; return data.toString().replace(/[&<>"]/g, char => htmlEntities[char]); } // Exempel på korrekt användning i responsgenerering: const userInput = searchParams.get('q'); // Användardata // från query-parameter let safeResponse; if (userInput) { const encodedData = encodeHTML(userInput); // Korrekt HTML-rendering med encoding safeResponse = `     <div class="search-result">       <h3>Sökresultat: \${encodedData}</h3>     <!-- Användarens data är nu kodad och kan inte       injicera skript -->     </div>   `; } // Alternativt - använd frameworks built-in escaping om // tillgängligt (exempelvis React, Vue) const safeResponse = <SearchResult q={encodeHTML(userInput)} />;</pre>
---	--------------------------	--	--	--	--

					<pre>// Dessutom: Implementera Content Security Policy för // ytterligare skydd document.head.insertAdjacentHTML('beforeend', '<meta http-equiv="Content-Security-Policy" content="default-src \'self\'; script-src \'self\'">');</pre>
M1: Cross-Site Scripting (XSS) - Reflected	Qwen 2.5- coder-7b	Pentest rapport SchutzPunkt.pdf	Cross-Site Scripting (XSS)	Penetrationstestrapporten indikerar att applikationen har en reflekterad XSS-sårbarhet. Detta beror på att användarinmatning inte är korrekt kodade innan den returneras till klienten, vilket gör det möjligt för angripare att införa skript som körs i värvärldens webbläsare. Koden utdraget från rapporten visar att data från användaren (t.ex., queryparametrarna) inte är kodade innan de returneras till klienten, vilket är den källan till sårbarheten.	Använd strict context-aware output encoding för alla reflekterade data. Implementera en restriktiv Content Security Policy (CSP) om möjligt. Exempel på kod för att implementera CSP: <pre><meta http-equiv="Content-Security-Policy" content="default-src 'self'; script-src 'self' https://trusted.cdn.com; object-src 'none';"></pre> Detta hjälper till att begränsa vilka resurser som kan laddas in på sidan och förhindrar att skript från oskyddade källor körs.

M1: Cross-Site Scripting (XSS) - Reflected	Ministr al 3 14B Reasonin g	Pentest rapport SchutzPunkt.p df	Brist på kontextmedveten utdata-kodning av användarstyrt data. Applikationen reflekterar användarlevererat input i svaret utan att tillämpa lämplig kodning baserad på renderingssammanhanget (t.ex. HTML, JavaScript), vilket är den primära orsaken till Reflected Cross-Site Scripting (XSS).	Sårbarheten uppstår när applikationen tar emot användarstyrda parametrar (som URL-parameter eller formfält) och direkt integrerar dessa i HTTP-svar utan att först tillämpa säkerhetsmässigt korrekt kodning. Baserat på penetrationstestrappporten, som troligen beskriver hur användarinput reflekteras tillbaka i svaret, och den bifogade koden (som sannolikt visar direkt concatenation av användarinput i HTML/JS), kan angripare konstruera maliciösa länkar eller begäran som exekverar skadlig script i offerets webbläsare. Denna typ av XSS kräver användarinteraktion men kan leda till allvarliga konsekvenser som sessionstjuvning, dataexponering och oauktoriserade handlingar.	Förklaring: Sårbarheten uppstår när applikationen tar emot användarstyrda parametrar (som URL-parameter eller formfält) och direkt integrerar dessa i HTTP-svar utan att först tillämpa säkerhetsmässigt korrekt kodning. Baserat på penetrationstestrappporten, som troligen beskriver hur användarinput reflekteras tillbaka i svaret, och den bifogade koden (som sannolikt visar direkt concatenation av användarinput i HTML/JS), kan angripare konstruera maliciösa länkar eller begäran som exekverar skadlig script i offerets webbläsare. Denna typ av XSS kräver användarinteraktion men kan leda till allvarliga konsekvenser som sessionstjuvning, dataexponering och oauktoriserade handlingar. Föreslagen åtgärd med kod: Implementera strikt kontextmedveten utdata-kodning för all reflekterad data och överväg att införa en restriktiv Content Security Policy (CSP). Här är ett exempel på hur man korrekt kodar output i olika sammanhang: <pre>// Exempel på säker kodning i Node.js med Express app.get('/search', (req, res) => { // Säkert att koda alla reflekterade parametrar const safeResult = req.query.q ? escapeHtml(req.query.q) : ""; res.send(`     <html>     <body>     <h1>Sökresultat</h1>     <p>Du sökte efter: \${safeResult}</p>     </body>     </html>   `); }); // En enkel HTML-escaping funktion function escapeHtml(unsafe) { return unsafe</pre>
---	--	---	---	---	--

					<pre> .replace(/&/g, "&amp;") .replace(/</g, "&lt;") .replace(/>/g, "&gt;") .replace(/"/g, "&quot;") .replace(/'/g, "&#039;"); } </pre>
M1: Cross-Site Scripting (XSS) - Reflected	Qwen 2.5- coder-14b	Pentest rapport SchutzPunkt.p df	Den mest sannolika grundorsaken till sårbarheten är att programmet inte använder korrekt kontextkänslig utdatakodning när det återger användarangivna data i webbsidan.	Sårbarheten uppstår eftersom programmet återger användarangivna data direkt i HTML-svaret utan att tillämpa lämplig kontextkänslig utdatakodning. Denna brist på utdatakodning gör det möjligt för en attacker att skapa en länk eller begäran som kör skript styrd av angrepet i användarens webbläsare inom programmet. Sårbarheten begränsas till övergångssvar och kräver användarinteraktion, men konsekvenserna kan inkludera kompromiss av konton, dataexposition och obehöriga åtgärder som utförs via användarens session.	En möjlig lösning är att tillämpa strikt kontextkänslig utdatakodning för alla återflekta data. Här är ett exempel på hur detta kan implementeras i JavaScript: <pre> // Före: Direkt infogande av användarangivna data utan // utdatakodning document.getElementById('userInput').innerHTML = userInput; // Efter: Använd kontextkänslig utdatakodning för att // skydda mot XSS function escapeHtml(unsafe) { return unsafe .replace(/&/g, "&amp;") .replace(/</g, "&lt;") .replace(/>/g, "&gt;") .replace(/"/g, "&quot;") .replace(/'/g, "&#039;"); } document.getElementById('userInput').innerHTML = escapeHtml(userInput); </pre>

					Denna kod implementerar en funktion för att skapa en säker version av användarangivna data genom att ersätta specialtecken med deras motsvarande HTML-entiteter. Genom att använda denna funktion innan du återger användardata i webbsidan kan du skydda dig mot XSS-attacker.
M3: Directory Listing Enabled	Qwen 3.5 - 9B	Pentest rapport SchutzPunkt.pdf	Webbservarens konfiguration tillåter automatisk mappvisning (directory listing) på offentliga sökvägar, vilket exponerar filnamn och strukturen för alla besökare utan autentisering. Detta är en serverkonfigurationsfel där funktioner som Apache mod_autoindex, Nginx autoindex eller liknande inte har avaktiverats korrekt (CWE-548, CWE-200).	Enligt penetrationstestrappporten och koden uppstår sårbarheten när webbservaren returnerar en automatiskt genererad indexsida för mappar som saknar standardindexfil såsom index.html. Detta sker ofta vid icke-hårdade default-konfigurationer eller genom att funktioner för directory browsing är aktiverade utan medveten konfigurationsändring. När en användare gör en begäran till en sökväg (t.ex. /assets/), returneras information om filstorlekar, tidsstämplar och undermappar som inte var avsedda att vara offentligt tillgängliga. Detta exponerar interna filnamn, byggartefakter, backup-filer och konfigurationsfragment som kan användas för rekognosering, brute-force-attacker mot upptäckta	Serverkonfiguration - Avaktivera directory listing: // För Express.js/Node.js server, konfigureras detta ofta i webbservarens inställningar // Exempel för Nginx (i nginx.conf): location / { autoindex off; # Stäng av mappvisning } # Alternativt via .htaccess för Apache: Options -Indexes # Avaktivera directory listing Alternativ lösning med default indexfil: Om du vill behålla sökvägen men inte visa filer, skapa en standardindexfil: // Skapa en tom eller minimal index.html för varje mapp som kan bli exponerad const fs = require('fs'); const path = require('path'); function ensureIndexFile(dirPath) { const indexPath = path.join(dirPath, 'index.html'); if (!fs.existsSync(indexPath)) {

				assets eller obehörig nedladdning av proprietär innehåll (CWE-548).	<pre>// Skapa en minimal indexfil för att undvika directory // listing fs.writeFileSync(indexPath, '<!DOCTYPE html><html><head></head><body>403 Forbidden</body></html>'); } } // Kör detta vid start eller efter deployment ensureIndexFile('/assets'); Ytterligare säkerhetsförbättringar: // Lägg till middleware för att blockera directory listing // begärningar app.use((req, res, next) => { if (req.method === 'GET' && req.path.endsWith('/')) { // Kontrollera om mappen saknar indexfil och // returnera 403 istället const indexPath = path.join(process.cwd(), req.path); if (!fs.existsSync(indexPath + '/index.html') !fs.existsSync(indexPath + '/default.html')) { res.status(403).send('Directory listing is disabled'); return; } } next(); }); // Använd helmet för ytterligare säkerhetsheaders (som // redan finns i Citation 3) app.use(helmet.noSniff()) app.disable('x-powered-by')</pre>
--	--	--	--	--	---

M3: Directory Listing Enabled	Qwen 2.5-coder-7b	Pentest rapport SchutzPunkt.pdf	Directory Listing Enabled	Den mest sannolika grundorsaken till sårbarheten är att directory listing är aktiverat på den webbservern. Detta exponerar namnen och strukturen på filer och underkataloger för alla besökare, även utan autentisering. Fillistningar ökar informationsexponering och gör reconnaissance och målade angrepp lättare.	// Disable directory listing on all publicly accessible web paths. app.use(express.static('public', { dotfiles: 'ignore', index: false, redirect: false })); Detta konfigurerar Express för att inte exponera filer i den offentliga katalogen, vilket inaktiverar directory listing.
M3: Directory Listing Enabled	Ministr al 3 14B Reasoning	Pentest rapport SchutzPunkt.pdf	The root cause is that directory listing is enabled on public paths, likely due to default server configurations not being hardened (e.g., mod_autoindex in Apache or similar features left enabled).	The vulnerability arises when the web server automatically generates and serves an index page for directories that lack a default file like index.html. This happens because features like Apache's mod_autoindex, Nginx's autoindex, or IIS Directory Browsing are enabled by default. When such a directory is requested (e.g., /assets/), the server responds with a list of files, their sizes, timestamps, and subdirectories. This exposes sensitive information that can aid attackers in reconnaissance, targeted brute-force attacks, and discovery of overlooked administrative endpoints or	Disable directory listing on all publicly accessible paths. Here's how you can implement this for common web servers: For Apache (in .htaccess or server config): Options -Indexes For Nginx (in server configuration): autoindex off; If the application uses Node.js with Express and serves static files, ensure directory listing is disabled in the middleware: const express = require('express'); const path = require('path'); const app = express(); // Disable directory listing for static files app.use(express.static(path.join(__dirname, 'public'), { index: false // Prevents directory listing }));

				sensitive files like configuration backups or credentials. The penetration test report indicates this is an immediate issue affecting the FTP component (https://juice-shop.lab/ftp), which increases the risk by exposing internal file structures and potentially sensitive content.	For FTP servers (e.g., if using Apache FTPServer or similar), configure the server to disable directory listing in its settings. For example, in a configuration file: <pre><LimitWrite> DenyAll </LimitWrite> <Directory> Options -Index </Directory></pre>
M3: Directory Listing Enabled	Qwen 2.5- coder-14b	Pentest rapport SchutzPunkt.p df	Den mest sannolika grundorsaken till sårbarheten är att webbserven är konfigurerad för att tillåta kataloglistning på ett eller flera offentliga sökvägar.	Sårbarheten uppstår eftersom webbserven är inställd för att visa en automatiskt genererad indexsida för en katalog som saknar en standardindexfil, till exempel index.html. Detta kan orsakas av aktiverade funktioner som Apache mod_autoindex, Nginx autoindex eller IIS Directory Browsing, eller på grund av standardkonfigurationer som inte har hårdkodats. När detta är aktiverat returnerar en begäran till en katalogsökväg (till exempel /assets/) en sida med en lista över filer, storlekar, tidsstämplar och underkataloger. Dessa listor visar interna filnamn, byggartefakter, säkerhetskopior, temporära filer och konfigurationsfragment som inte var avsedda att vara offentligt tillgängliga.	Förklaring: Sårbarheten uppstår eftersom webbserven är inställd för att visa en automatiskt genererad indexsida för en katalog som saknar en standardindexfil, till exempel index.html. Detta kan orsakas av aktiverade funktioner som Apache mod_autoindex, Nginx autoindex eller IIS Directory Browsing, eller på grund av standardkonfigurationer som inte har hårdkodats. När detta är aktiverat returnerar en begäran till en katalogsökväg (till exempel /assets/) en sida med en lista över filer, storlekar, tidsstämplar och underkataloger. Dessa listor visar interna filnamn, byggartefakter, säkerhetskopior, temporära filer och konfigurationsfragment som inte var avsedda att vara offentligt tillgängliga. Föreslagen åtgärd med kod: En möjlig lösning är att inaktivera kataloglistning på alla offentligt tillgängliga webbsökvägar. Detta kan göras genom att lägga till följande kod i konfigurationen för din webbserver:

					<pre> // För Apache <Directory "/var/www/html"> Options -Indexes </Directory> // För Nginx location / { autoindex off; } // För IIS <configuration> <system.webServer> <directoryBrowse enabled="false" /> </system.webServer> </configuration> </pre> Denna kod inaktiverar kataloglistning på alla offentligt tillgängliga webbsökvägar, vilket minskar risken för informationsexponering och förenklar identifieringen av sårbara punkter.
--	--	--	--	--	--

Application is vulnerable to SQL Injection	Qwen 3.5 - 9B	Juice Shop - Web Application Security Assessment Report	Sårbarheten beror på direkt stränginterpolering av användarinput (e-postadress och lösenord) i en rå SQL-query-sträng som exekveras via <code>models.sequelize.query()</code>. Koden infogar variablerna (<code>req.body.email</code> och <code>hashed password</code>) direkt inuti query-mallen utan att använda parametriska platshållare eller ORM-funktioner som automatiskt hanterar flyttning av data från SQL-kod. Enligt penetrationstrappporten är applikationen sårbar för klassisk SQL-injection (CVSS 8.9). Rapporten visar att en payload såsom <code>' OR 1=1 --</code> i fältet "email" tillåter autentisering som administratör oavsett lösenord, eftersom databasen tolkar inputet som del av query-logiken snarare än data. I koden (<code>login.ts</code>) syns detta tydligt på raden: <code>models.sequelize.query(`SELECT * FROM Users WHERE email = '\${req.body.email}    '' AND password = ...`...)</code> Här infogas <code>req.body.email</code> direkt inuti backtickar (<code>`\${...}`</code>). Om en attacker skickar input som innehåller SQL-syntax, bryts query-strukturen och logiken kan manipuleras. Rapportens rekommendation är att använda parametriska queries (prepared statements) istället för strängkonkatenering i koden.	Byt ut den råa SQL-queryn mot en metod som använder parametrisk bindning, exempelvis genom att använda placeholders (<code>\$1</code>, <code>\$2</code>) och separata värden för inmatningen, eller använd ORM-metoder (som <code>findOne</code> om möjligt) som hanterar data isolerat från koden. Här visas hur man fixar den specifika raden med parametriska platshållare: <pre>// Föreslagen säker version av queryn med parametrisk bindning models.sequelize.query('SELECT * FROM Users WHERE email = \$1 AND password = \$2', { model: UserModel, plain: true }, [req.body.email '', security.hash(req.body.password '')] // Värden skickas separat för att inte bli tolkade som SQL-kod) Alternativt kan man använda modellens metoder om möjligt (om Sequelize-versionen stöder det): // Alternativ: Använd ORM-metoden istället av raw queryn const user = await UserModel.findOne({ where: { email: req.body.email '' } }) // Därefter jämför hash separat eller använd built-in auth logik om tillgänglig</pre>
--	---------------	---	---	--

Application is vulnerable to SQL Injection	Qwen 2.5-coder-7b	Juice Shop - Web Application Security Assessment Report	SQL-injektion	Sårbarheten uppstår eftersom koden använder string concatenation för att skapa SQL-frågor, vilket gör det möjligt för en angripare att införa och manipulera SQL-kod. I specifikt fall i login.ts filen används models.sequelize.query med en sträng som innehåller user-provided input (req.body.email och req.body.password) utan några mekanismer för att förhindra SQL-injektioner. Detta gör det möjligt för en angripare att skicka in en payload som kan manipulera SQL-frågan, till exempel ' OR 1=1 -- i email-fältet, vilket skulle resultera i att alla användare kunde logga in.	Ersätt string concatenation med parameterized queries (förberedda uttryck). Detta kan göras genom att använda Sequelize's query-metod med bindparametrar snarare än att skicka in SQL-koden som en sträng. Här är ett exempel på hur det kan se ut: <pre> return (req: Request, res: Response, next: NextFunction) => { verifyPreLoginChallenges(req); // vuln-code-snippet hide-line models.sequelize.query(`SELECT * FROM Users WHERE email = :email     AND password = :password AND deletedAt IS NULL`, { model: UserModel, plain: true, replacements: { email: req.body.email "", password: security.hash(req.body.password "") } } // vuln-code-snippet neutral-line loginAdminChallenge loginBenderChallenge loginJimChallenge) .then((authenticatedUser) => { // vuln-code-snippet neutral-line loginAdminChallenge loginBenderChallenge loginJimChallenge const user = utils.queryResultToJson(authenticatedUser); if (user.data?.id && user.data.totpSecret !== "") { res.status(401).json({ status: 'totp_token_required', data: { tmpToken: security.authorize({ userId: user.data.id, type: 'password_valid_needs_second_factor_token' }) } }) } }) } </pre>
--	-------------------	---	---------------	---	--

					<pre>}); } else if (user.data?.id) { // @ts-expect-error FIXME some properties missing in user - vuln-code-snippet hide-line afterLogin(user, res, next); } else { res.status(401).send(res.__('Invalid email or password.')); } }).catch((error: Error) => { next(error); }); });</pre> Genom att använda replacements-objektet i Sequelize's query-metod, kan vi förhindra SQL-injektioner genom att separera SQL-koden från de user-provided input-värdena. Detta säkerställer att värdena skickas som bindparametrar och inte som en del av SQL-frågan, vilket förhindrar att angripare kan manipulera SQL-koden.
Application is vulnerable to SQL Injection	Ministry of Justice	Juice Shop - Web Application Security Assessment Report	Direkt interpolering av användarindata i SQL-frågor utan parameterisering eller sanering. Användarens e-postadress (req.body.email) sätts direkt in i SQL-frågan som en sträng, vilket gör det möjligt för angripare att injicera maliciös SQL-kod.	Sårbarheten uppstår när användarindata (e-postadress) direkt interpoleras i SQL-frågan utan någon form av parameterisering eller sanering. I koden <pre>models.sequelize.query(\SELECT * FROM Users WHERE email = '\${req.body.email} '' ...') </pre> sätts användarens e-postadress in direkt i SQL-strängen. Detta gör att angripare kan manipulera frågans logik genom att införa payloads som ' OR 1=1 --, vilket resulterar i en alltid sann villkor (e.g., email = ' OR 1=1), och därmed omgår autentiseringen. Rapporten bekräftar detta genom att visa hur en angripare kan logga in som	Använd parameteriserade frågor istället för direkt stränginterpolering. Här är ett förslag på hur koden kan omvandlas: <pre>// Original sårbar kod: models.sequelize.query(`SELECT * FROM Users WHERE email = '\${req.body.email}    ''' AND password = '\${security.hash(req.body.password    '')}' AND deletedAt IS NULL`, { model: UserModel, plain: true }) // Säkrad version med parameterisering: const email = req.body.email ''; const hashedPassword = security.hash(req.body.password ''); models.sequelize.query(`SELECT * FROM Users WHERE email = :email AND password = :password AND deletedAt IS NULL`, { replacements: { email, password: hashedPassword },</pre>

				admin eller dumpa lösenords_hashar från databasen med hjälp av SQLMap.	<pre> model: UserModel, plain: true }) Alternativt kan du använda Sequelize's inbyggda metoder för att undvika manuell SQL-konstruktion: UserModel.findOne({ where: { email: req.body.email "", password: security.hash(req.body.password ""), deletedAt: null } }); Dessa ändringar säkerställer att användarindata behandlas som data och inte som utförbar SQL-kod, vilket förhindrar SQL-injektion. Parameteriserade frågor är den mest effektiva metoden för att förebygga denna typ av sårbarhet. </pre>
Application is vulnerable to SQL Injection	Qwen 2.5-coder-14b	Juice Shop - Web Application Security Assessment Report	Den mest sannolika grundorsaken till sårbarheten är att applikationen använder oskyddade användarindata i SQL-frågor utan att korrekt neutralisera specialtecken som kan ändra den avsedda SQL-kommandot.	Sårbarheten uppstår eftersom applikationen konstruerar en SQL-kommando med externt påverkade indata från användaren, men utan att neutralisera specialtecken som kan ändra den avsedda SQL-kommandot när det skickas till databaskomponenten. Detta leder till att användarinmatningen kan tolkas som SQL-syntax i stället för vanlig användardata, vilket kan användas för att manipulera frågelogiken och eventuellt exekvera systemkommandon.	En möjlig lösning är att använda parameteriserade frågor (också kallade för förberedda instruktioner) istället för strängkonkatenering i frågan. Här är ett exempel på hur detta kan implementeras: <pre> // Före: Osäker kod med strängkonkatenering const query = "SELECT * FROM users WHERE email = " + req.body.email + " AND password = " + req.body.password + """; connection.query(query, (error, results) => { if (error) throw error; console.log(results); }); // Efter: Säker kod med parameteriserade frågor const query = "SELECT * FROM users WHERE email = ? AND password = ?"; connection.query(query, [req.body.email, req.body.password], (error, results) => { if (error) throw error; </pre>

					<pre>console.log(results); });</pre> Genom att använda parameteriserade frågor kan vi säkerställa att användarinmatningen behandlas som data och inte som SQL-syntax. Detta förhindrar SQL-injektioner och ökar säkerheten för applikationen.
DOM-Based Cross Site Scripting (XSS)	Qwen 3.5 - 9B	Juice Shop - Web Application Security Assessment Report	Användning av metoden DomSanitizer.bypassSecurityTrustHtml() på data från en användarstyrd källa (URL-parametern 'q') kombinerat med rendering via [innerHTML] i HTML-mallen utan att utföra någon kodning (encoding) eller validering. Detta omgår Angulars inbyggda skydd mot skadlig kod och tillåter direkt inkapsling av användarinput som HTML-kod, vilket leder till DOM-baserade XSS-attacker.	Enligt penetrationstrappporten uppstår sårbarheten vid sökfunktionen (/search?q=). I search-result.component.ts hämtas sökfrågan från URL-parametern via this.route.snapshot.queryParams.q. Denna sträng, som är kontrollerad av användaren (attacker), behandlas sedan explicit som "säkert" HTML genom anropet .bypassSecurityTrustHtml(queryParam), vilket markerar innehållet för Angular så att det inte filtreras. I search-result.component.html bindes denna variabel till [innerHTML] i en -tagg (). När användaren sätter in ett skadligt payload, t.ex. <script>alert('xss')</script>, som sökterm, kommer detta att renderas	Ta bort anropet bypassSecurityTrustHtml i TypeScript-koden eftersom sökfrågan är ren text och inte HTML-innehåll som kräver innerHTML-rendering för struktur. Ändra även bindningen i HTML-mallen från [innerHTML] till interpolation ({{ }}) eller en annan säker metod, vilket tvingar Angular att automatiskt koda om HTML-tecken (t.ex. < blir &lt;). search-result.component.ts: <pre>// ... existerande kod ovanför metoden filterTable() ... filterTable() { let queryParam: string = this.route.snapshot.queryParams.q if (queryParam) { queryParam = queryParam.trim() // Använd inte bypassSecurityTrustHtml för // användarinput som ska visas som text/titel const searchValueString = queryParam this.dataSource.filter = queryParam.toLowerCase() } }</pre>

				och köras direkt av webbläsaren eftersom det har omgått sanitiseringsprocessen. Rapportens PoC bekräftar att payloads exekveras vid denna ruta (CVSS Score 6.8). <pre> this.searchValue = searchValueString // Uppdatera variabeln till vanlig sträng eller behåll typen men ändra bindning i HTML this.gridDataSource.subscribe((result: any) => { if (result.length === 0) { this.emptyState = true } else { this.emptyState = false } }) } else { this.dataSource.filter = "" this.searchValue = undefined this.emptyState = false } } // ... resten av koden ... search-result.component.html: <!-- Ändra bindningen från [innerHTML] till interpolation för att säkerställa HTML-encoding --> <div class="center-align"> <div class="table-container custom-slate"> <!-- ... existerande kod ovanför heading div ... --> <div class="heading mat-elevation-z6"> @if (searchValue) { <div> {{"TITLE_SEARCH_RESULTS" translate}} - <!-- Ändrat från [innerHTML]="searchValue" till {{ searchValue }} för säker textrendering --> {{ searchValue }} </div> } @else { </pre>
--	--	--	--	---

					<pre> <div>{{"TITLE_ALL_PRODUCTS" translate}}</div> } </div> <!-- ... resten av koden ... --> </pre>
DOM- Based Cross Site Scripting (XSS)	Qwen 2.5- coder-7b	Juice Shop - Web Application Security Assessment Report	DOM-baserad Cross Site Scripting (XSS)	Sårbarheten uppstår eftersom applikationen inte korrekt kodar dynamiskt inmatade data innan den infogas i utdata. I koden ser vi att produktbeskrivningarna tas emot från servern och sedan direkt tilldelas till tableData utan någon form av kodning. Sedan används innerHTML för att rendera dessa beskrivningar, vilket gör det möjligt för skadliga skript att införlivas om de innehåller HTML eller JavaScript.	För att förhindra DOM-baserade XSS-sårbarheter bör alla dynamiska data kodas innan den infogas i utdata. I koden kan vi använda DomSanitizer för att säkerställa att inmatade data kodas korrekt. Ett möjligt lösning skulle kunna se ut så här: <pre> import { DomSanitizer, SafeHtml } from '@angular/platform-browser'; // ... trustProductDescription(tableData: any[]) { for (let i = 0; i < tableData.length; i++) { tableData[i].description = this.sanitizer.bypassSecurityTrustHtml(tableData[i].desc ription); } } </pre> Detta kodar alla produktbeskrivningar innan de tilldelas till tableData, vilket förhindrar skadliga skript från att införlivas. Dessutom bör vi kontrollera och kodning av

					alla andra dynamiska data i applikationen för att säkerställa att ingen annan sårbarhet uppstår.
DOM-Based Cross Site Scripting (XSS)	Ministr al 3 14B Reasoning	Juice Shop - Web Application Security Assessment Report	The root cause of the vulnerability is the misuse of Angular's DomSanitizer.bypassSecurityTrustHtml method on user-controlled input from the URL query parameter (q). This bypasses Angular's built-in security protections, allowing malicious scripts to be executed in the context of the victim's browser without proper validation or sanitization.	The DOM-based XSS vulnerability occurs because: The application retrieves a search query parameter (q) from the URL, which is user-controlled and potentially malicious. This input is directly passed to bypassSecurityTrustHtml, which tells Angular to trust this HTML content as if it came from a trusted source. An attacker can craft a malicious payload (e.g., <iframe src="javascript:alert('xss')">) in the search query, which gets executed when the page renders because the application blindly trusts the input without validation or sanitization. This is a DOM-based XSS because the exploit happens entirely on the client side without server	The immediate fix is to stop using bypassSecurityTrustHtml for user input and instead treat the search query as plain text. Here's how you can modify the code: In search-result.component.ts, modify the filterTable method: <pre>filterTable() { let queryParams: string = this.route.snapshot.queryParams.q; if (queryParams) { queryParams = queryParams.trim(); this.ngZone.runOutsideAngular(() => { this.io.socket().emit('verifyLocalXssChallenge', queryParams); }); this.dataSource.filter = queryParams.toLowerCase(); // Treat search value as plain text, not HTML this.searchValue = queryParams; // Angular will escape this automatically in the template this.gridDataSource.subscribe((result: any) => { if (result.length === 0) {</pre>

				involvement, making traditional defenses like WAFs ineffective against it. <pre> this.emptyState = true; } else { this.emptyState = false; } }); } else { this.dataSource.filter = ""; this.searchValue = undefined; this.emptyState = false; } } </pre> In the template (search-result.component.html), change the binding to use text interpolation instead of innerHTML: <pre> @if (searchValue) { <div> {{"TITLE_SEARCH_RESULTS" translate}} <!-- Use text interpolation instead of innerHTML --> {{ searchValue }} </div> } @else { <div>{{"TITLE_ALL_PRODUCTS" translate}}</div> } </pre> Additional Recommendations: Input Validation: Implement strict validation for the q parameter to ensure it only contains expected characters (e.g., alphanumeric, spaces, and a few safe symbols like - , _ , etc.). CSP Implementation: Add a Content Security Policy (CSP) header to your application that restricts inline scripts and external resources. Use Angular's Built-in Sanitization: For product descriptions or any other dynamic content that might come from untrusted sources, use proper sanitization
--	--	--	--	--

					before trusting HTML. Avoid using <code>bypassSecurityTrustHtml</code> unless you are absolutely sure the content is safe (e.g., comes from a trusted admin interface with proper sanitization). Output Encoding: Ensure all dynamic data is properly encoded based on its context (HTML, attribute, JavaScript, etc.) before being embedded in the page. By making these changes, you eliminate the DOM-based XSS vulnerability by ensuring that user input is treated as plain text and not executable HTML or JavaScript. This aligns with the recommendations in the penetration test report to fully encode all dynamic data and implement a Content Security Policy (CSP) for added protection.
DOM-Based Cross Site Scripting (XSS)	Qwen 2.5-coder-14b	Juice Shop - Web Application Security Assessment Report	Den mest sannolika grundorsaken till sårbarheten är att programmet inte korrekt hanterar och korregerar skadliga inmatningar i användargränssnittet, specifikt i sökfunktionen.	Sårbarheten uppstår eftersom programmet använder <code>bypassSecurityTrustHtml</code> för att markera produktbeskrivningar som säkra HTML utan att korrekt validera eller rensa inmatningen. Detta gör att eventuella skadliga skript som ingår i beskrivningarna kan köras i webbläsaren, vilket leder till en DOM-baserad XSS-attack.	En möjlig lösning är att använda <code>DomSanitizer</code> för att korrekt rensa och validera inmatningen innan den bäddas in i HTML. Här är ett exempel på hur detta kan implementeras: <pre>trustProductDescription (tableData: any[]) { for (let i = 0; i < tableData.length; i++) { // Använd DomSanitizer för att rensa beskrivningen const safeDescription = this.sanitizer.sanitize(SecurityContext.HTML, tableData[i].description); if (safeDescription) { tableData[i].description = safeDescription; } else { tableData[i].description = ""; // Om beskrivningen är osäker, sätt den till tom } } }</pre> Denna förbättring säkerställer att inga skadliga skript kan

					körs i webbläsaren genom att korrekt rensa och validera inmatningen.
--	--	--	--	--	--



HÖGSKOLAN I BORÅS

Besöksadress: Allégatan 1 · Postadress: 501 90 Borås · Tfn: 033-435 40 00 · E-post: registrator@hb.se · Webb: www.hb.se